

About The Trainer

❖ **Cristopher Perando**

- Senior Software Developer with over 20 years of experience in software engineering, server administration, database architecture, and cybersecurity.
- Brings 8 years of teaching experience across local and international training environments, delivering technical education and hands-on industry expertise.
- Experienced in Artificial Intelligence technologies and recognized for strong collaboration, leadership, and an engaging professional approach.

Training Schedule

Start	Coffee Break (15mins)	Lunch Break	Continued	Coffee Break (15mins)	End
9:00 am	10:30 am – 10:45 am	12 noon – 1 pm	1pm	3:00 pm - 3:15 pm	5:00 pm

Beginner **Playwright Automation Testing**

- This hands-on training program introduces participants to modern UI automation using **Playwright for .NET** and **xUnit** within a **.NET 10** environment.
- The course emphasizes practical implementation using **real-world application behavior**, particularly modern **React.js applications** and **Material UI components**, where dynamic rendering, asynchronous loading, and complex DOM structures often introduce automation challenges.

Training Objectives

- Build Playwright automation projects using .NET 10 and xUnit
- Create maintainable UI tests using modern Playwright practices
- Automate forms, dialogs, tables, menus, and interactive Material UI components
- Build reliable selector strategies using role-based locators and test IDs
- Implement the Page Object Model (POM) for maintainable test architecture
- Debug tests using traces, screenshots, videos, and Playwright tooling
- Reduce flaky tests using synchronization and auto-waiting capabilities

Training Audience

- Manual Testers transitioning to Automation Testing
- QA Engineers working with .NET and React-based systems
- Junior Developers interested in test automation
- SDETs (Software Development Engineers in Test) – beginners
- IT Professionals involved in system quality assurance

Training Pre- requisites

- Basic understanding of C#
- Basic understanding of HTML and browser behavior
- Familiarity with web applications
- Experience using Visual Studio or VS Code

Course Content

Day 1 – UI Automation Foundations with Playwright .NET

➤ Module 1: Automation Testing Fundamentals

- Manual vs automation testing
- Role of UI automation in Agile and DevOps
- Test pyramid with emphasis on UI testing
- When UI automation is useful and when it is not
- Playwright overview and browser automation benefits

➤ Module 2: Playwright with C#, .NET 10, and xUnit Setup

- Installing .NET 10 SDK
- Creating an xUnit test project
- Installing Playwright for .NET
- Understanding project structure
- Running the first browser test

Course Content

Day 1 – UI Automation Foundations with Playwright .NET

➤ **Module 3: UI Automation Basics**

- Browser, context, page concepts
- Navigating pages
- Locators and selector strategy
- Buttons, textboxes, forms, links
- Assertions and validation

➤ **Module 4: Testing React.js + Material UI Applications**

- Challenges of modern React UI testing
- Dynamic DOM behavior
- MUI buttons, dialogs, menus, tables, dropdowns
- Auto-waiting and synchronization

Course Content

- **Day 2 – Advanced UI Automation Design**

- **Module 5: Reliable Locator Strategy**

- Role-based locators
 - Text locators
 - Test IDs
 - Avoiding brittle CSS/XPath selectors
 - Handling dynamic elements

- **Module 6: Forms, Tables, Dialogs, and Validation Testing**

- Form input automation
 - Client-side validation testing
 - MUI dialog automation
 - MUI table search/filter/pagination testing

Course Content

Day 2 – Advanced UI Automation Design

➤ **Module 7: Page Object Model with xUnit**

- Why use POM
- Page classes
- Reusable actions
- Reusable assertions
- Separating test logic from page interaction

➤ **Module 8: Test Data and UI State Management**

- Test data strategy
- Positive and negative test cases
- Browser storage and cookies
- Authentication state reuse
- Data-driven testing basics with xUnit

Course Content

Day 3 – End-to-End UI Testing, Debugging, and CI/CD

➤ **Module 9: End-to-End UI Test Scenarios**

- Designing realistic E2E UI workflows
- Combining multiple pages
- Verifying business flows from UI

➤ **Module 10: Debugging, Tracing, Screenshots, and Reporting**

- Debugging Playwright tests
- Headed vs headless execution
- Screenshots on failure
- Video recording
- Trace viewer
- Handling flaky tests

Course Content

Day 3 – End-to-End UI Testing, Debugging, and CI/CD

➤ **Module 11: Test Project Structure and Best Practices**

- Folder structure
- Naming conventions
- Test isolation
- Reusable fixtures
- Test cleanup
- Maintainability guidelines

➤ **Module 12: CI/CD Overview for UI Tests**

- Running Playwright tests with dotnet test
- Pipeline execution concept
- GitHub Actions overview
- Test artifacts: screenshots, videos, traces

Tools & Technologies

- .NET 10
- C#
- xUnit
- Playwright for .NET
- React.js
- Material UI
- Visual Studio / VS Code
- GitHub Actions

Module 1

Automation Testing Fundamentals

Manual vs automation testing

Area	Manual Testing	Automation Testing
Execution	Human	Tool/script
Speed	Slow	Fast
Cost initially	Low	Higher
Cost long-term	High	Lower
Repetitive tasks	Poor	Excellent
Human observation	Excellent	Limited
Regression testing	Difficult	Excellent
CI/CD integration	No	Yes
Parallel execution	No	Yes
Programming required	No	Yes

Manual Testing

- Manual testing is performed by a human tester who executes test cases without scripts or automation tools.
- **Process**
 1. Open the application
 2. Perform actions manually
 3. Observe behavior
 4. Compare actual results with expected results
 5. Record defects

Manual Testing

- **Example:** Testing a login page manually
- **Steps**
 1. Open browser
 2. Navigate to login page
 3. Enter username: admin
 4. Enter password: 123456
 5. Click Login
 6. Verify dashboard appears

Manual Testing

Advantages

- Good for exploratory testing
- Useful for UI/UX evaluation
- Human intuition can discover unexpected issues
- No coding skills required

Disadvantages

- Slow for repetitive tasks
- Time-consuming
- Prone to human mistakes
- Difficult for large regression suites
- Not ideal for CI/CD pipelines

Automation Testing

- Automation testing uses scripts and tools to execute test cases automatically.
- Examples of tools
 - Playwright
 - Selenium
 - Cypress
 - Appium

Automation Testing

Advantages

- Fast execution
- Repeatable
- Suitable for regression testing
- Integrates with CI/CD pipelines
- Supports parallel execution
- Reusable test scripts

Disadvantages

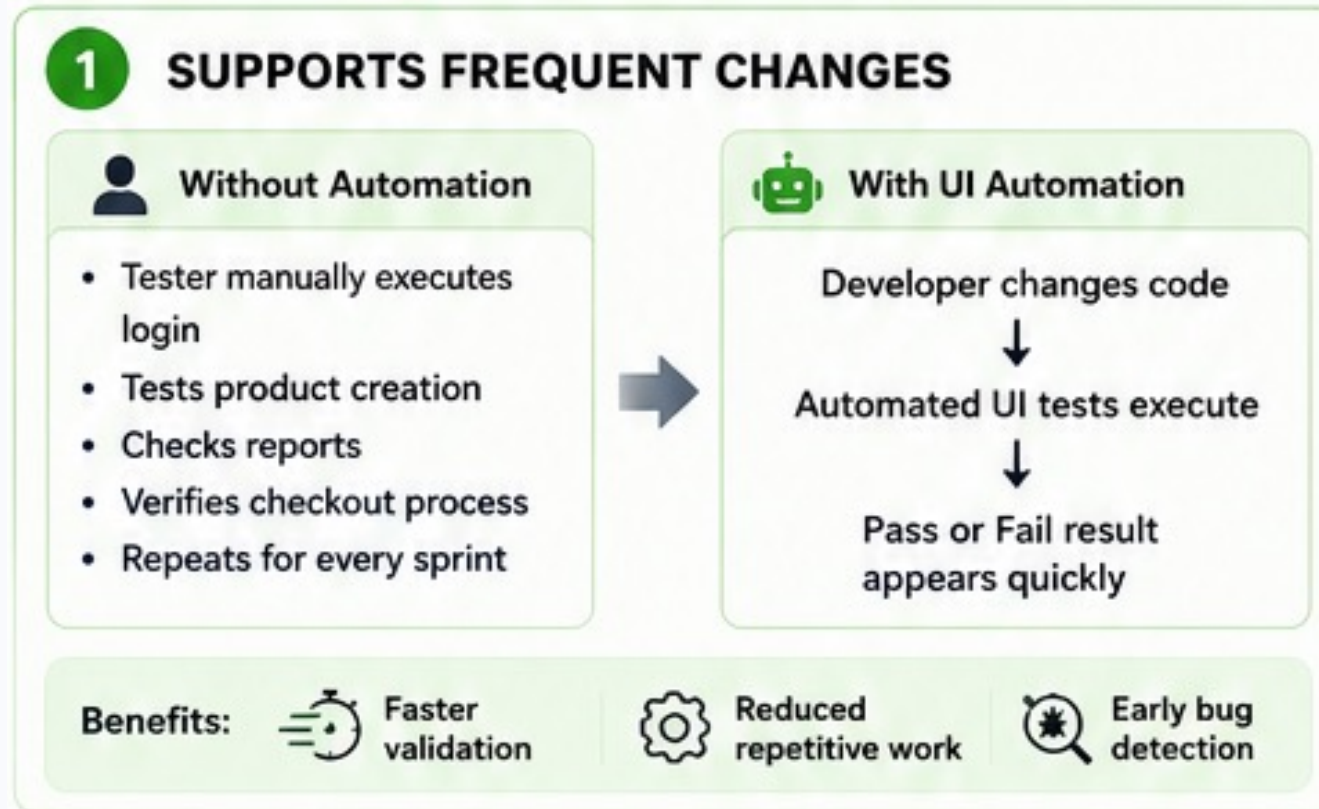
- Initial setup cost
- Requires programming skills
- Test maintenance effort
- Poorly designed tests can become unstable

AGILE

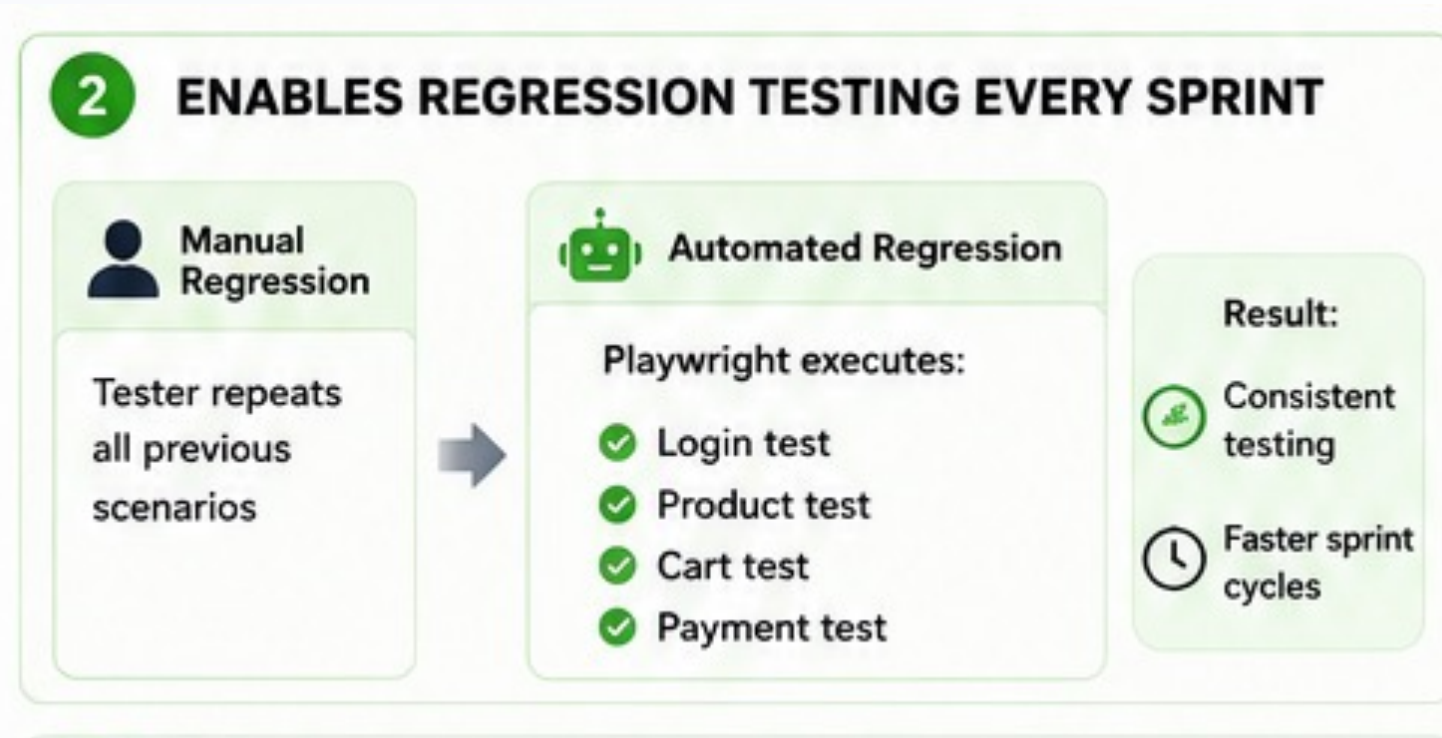
Short Iterations. Frequent Changes.
Continuous Feedback.



Role of UI Automation in Agile



Role of UI Automation in Agile



Role of UI Automation in Agile

3 PROVIDES QUICK FEEDBACK TO DEVELOPERS



★ The earlier a bug is found: Lower fixing cost • Less impact • Faster development

DEVOPS

Automate. Integrate.
Deliver Continuously.



TYPICAL DEVOPS PIPELINE



Role of UI Automation in DevOps

UI AUTOMATION IN CI/CD PIPELINE



Role of UI Automation in DevOps

CATCH ISSUES BEFORE PRODUCTION

Without UI Automation



Bug reaches
production



Users cannot login



With UI Automation



Playwright detects:
Button not found



Deployment stops

FASTER RELEASE CYCLES



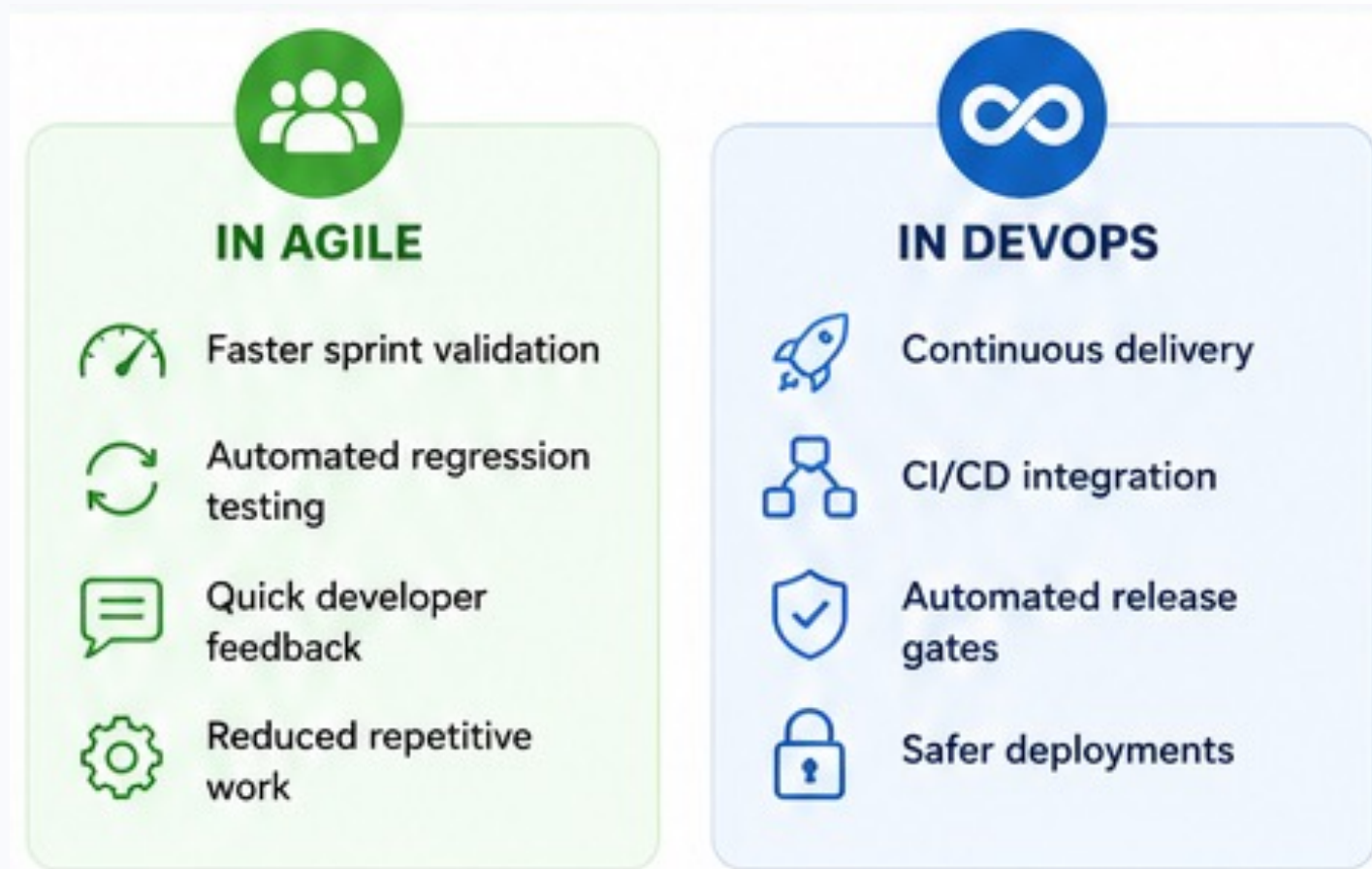
Manual Testing

3 – 5 hours
validation

Automated Testing

10 – 20 minutes
validation

Benefits of Automation in Agile and DevOps



When UI Automation Is Useful

Use UI Automation When	Example
The feature is frequently tested	Login, logout, registration
The workflow is critical to the business	Checkout, payment, booking, order submission
The UI is stable	Mature pages with minimal design changes
The test must run often	Smoke tests after every build
The same test needs multiple data sets	Login with valid/invalid users
Cross-browser testing is needed	Chrome, Edge, Firefox testing
Regression testing is required	Confirm old features still work after updates
Manual testing is time-consuming	Long multi-step forms or repeated validations

When UI Automation Is Not Useful

Avoid UI Automation When	Better Approach
The UI changes frequently	Manual testing or wait until stable
Testing business logic only	Unit test
Testing API response rules	API/integration test
Testing database calculations	Database/unit test
Checking exact visual appearance	Visual testing/manual review
Testing one-time features	Manual testing
The test is too fragile	Refactor the test or move lower in test pyramid
It takes too long to run	Use API or unit tests instead

A general rule...

- A good UI automation test answers

Can the user complete an important task successfully?

- A poor UI automation test tries to check

Is every button, label, color, and calculation correct through the browser?

Playwright

- Playwright is a modern browser automation framework developed by Microsoft for end-to-end (E2E), UI, and cross-browser testing.\
- It allows testers and developers to automate interactions with web applications using real browsers.

Playwright supports

- Chromium
- Chrome
- Microsoft Edge
- Firefox
- Safari/WebKit

Playwright for modern web applications

- React
- Angular
- Vue
- Single Page Applications (SPA)
- Material UI applications
- Dynamic JavaScript-heavy sites



BROWSER AUTOMATION BENEFITS

Automate Today. Deliver Quality Every Day.

1 FASTER TESTING



Automate repetitive tasks and run tests in minutes instead of hours.

Save Time. Increase Productivity.

2 REPEATABILITY



Automation performs the same steps every time with consistent results.

Same Steps. Same Results.

3 REDUCED HUMAN ERROR



Eliminate mistakes caused by manual testing and improve accuracy.

Reliable. Accurate. Trustworthy.





BROWSER AUTOMATION BENEFITS

Automate Today. Deliver Quality Every Day.

4 PARALLEL EXECUTION



Run multiple tests simultaneously to complete suites much faster.

More Tests. Less Time.

5 CROSS-BROWSER TESTING



Run the same tests on different browsers to ensure consistent user experience.

One Test. Many Browsers.

6 HEADLESS EXECUTION



Execute tests in headless mode to save resources and run faster, especially in CI/CD.

Faster. Lighter. CI/CD Ready.





BROWSER AUTOMATION BENEFITS

Automate Today. Deliver Quality Every Day.

7

AUTOMATIC WAITING

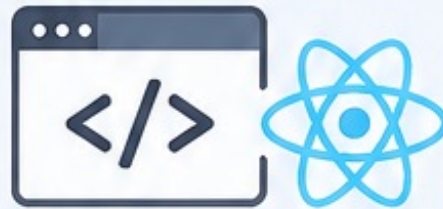


Built-in smart waiting for elements, reducing flaky tests and false failures.

Smarter Waiting. Stable Tests.

8

BETTER SUPPORT FOR MODERN WEB APPS



Handles dynamic content, SPAs, React, and modern UI frameworks with ease.

Modern Apps. No Problem.

9

IDEAL FOR CI/CD PIPELINES



Integrates seamlessly with CI/CD to deliver quick feedback and continuous quality.

Automate. Integrate. Accelerate.



Installing .NET 10 SDK

1. Download latest SDK <https://dotnet.microsoft.com/en-us/download/dotnet/10.0>

Build apps - SDK ⓘ

SDK 10.0.300

OS	Installers	Binaries
Linux	Package manager instructions	Arm32 Arm32 Alpine Arm64 Arm64 Alpine x64 x64 Alpine
macOS	Arm64 x64	Arm64 x64
Windows	x64 x86 Arm64 winget instructions	x64 x86 Arm64
All	dotnet-install scripts	

Installing .NET 10 SDK

2. On top level of your root project, enforce the version

```
dotnet new globaljson --sdk-version 10.0.300 --roll-forward disable
```

A new file global.json

```
{  
  "sdk": {  
    "rollForward": "disable",  
    "version": "10.0.300"  
  }  
}
```

Installing .NET 10 SDK

3. Check the installation

```
PS D:\PlaywrightActivities> dotnet --version  
10.0.300  
PS D:\PlaywrightActivities>
```


Creating an xUnit test with Playwright project

1. Search for available **Microsoft.DotNet.Test.ProjectTemplates** that is match to you current **.NET SDK**:

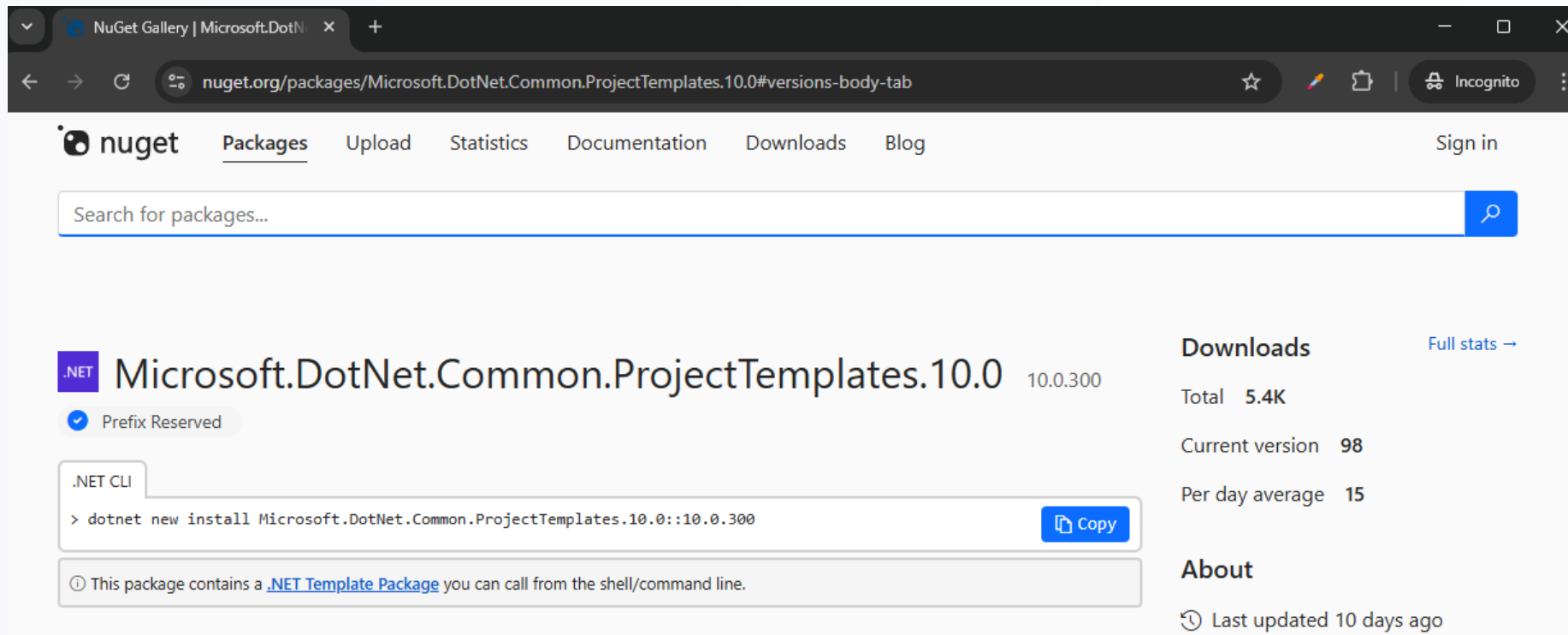
dotnet package search Microsoft.DotNet.Common.ProjectTemplates.10.*

```
PS D:\PlaywrightActivities\Module2> dotnet package search Microsoft.DotNet.Common.ProjectTemplates.10.*
*****
Source: Microsoft Visual Studio Offline Packages (file:///C:/Program Files (x86)/Microsoft SDKs/NuGetPackages/)
No results found.
*****
Source: nuget.org (https://api.nuget.org/v3/index.json)
```

Package ID	Latest Version	Owners	Total Downloads
Microsoft.DotNet.Common.ProjectTemplates.10.0	10.0.300	dotnetframework, Microsoft	5,417

Creating an xUnit test with Playwright project

- Nuget packages are available in the <https://www.nuget.org> repositories



Creating an xUnit test with Playwright project

2. Install the official template package:

`dotnet new install Microsoft.DotNet.Common.ProjectTemplates.10.0@10.0.300`

```
PS D:\PlaywrightActivities\Module2> dotnet new install Microsoft.DotNet.Common.ProjectTemplates.10.0@10.0.300
The following template packages will be installed:
  Microsoft.DotNet.Common.ProjectTemplates.10.0@10.0.300

Installing the template package(s) will override the available template package(s).
The following template package(s) are already available:
  Microsoft.DotNet.Common.ProjectTemplates.10.0@10.0.300

To install the template package(s) anyway, apply '--force' option:
  dotnet new install Microsoft.DotNet.Common.ProjectTemplates.10.0@10.0.300 --force

For details on the exit code, refer to https://aka.ms/templating-exit-codes#106
PS D:\PlaywrightActivities\Module2> dotnet new sln -n Module2.PlaywrightSetup
The template "Solution File" was created successfully.

PS D:\PlaywrightActivities\Module2> dotnet new xunit -n Module2.PlaywrightSetup.Tests
The template "xUnit Test Project" was created successfully.
```


Creating an xUnit test with Playwright project

3. Create the solution

```
dotnet new sln -n Module2.PlaywrightSetup
```

4. Create the xUnit test project

```
dotnet new xunit -n Module2.PlaywrightSetup.Tests
```

5. Add the test project to the solution

```
dotnet sln Module2.PlaywrightSetup.slnx add  
.\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj
```

Creating an xUnit test with Playwright project

```
PS D:\PlaywrightActivities\Module2> dotnet new sln -n Module2.PlaywrightSetup
The template "Solution File" was created successfully.

PS D:\PlaywrightActivities\Module2> dotnet new xunit -n Module2.PlaywrightSetup.Tests
The template "xUnit Test Project" was created successfully.

Processing post-creation actions...
Restoring D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj:
Restore succeeded.

PS D:\PlaywrightActivities\Module2> dotnet sln Module2.PlaywrightSetup.slnx add .\Module2.PlaywrightSetup.Tests\Module2.
PlaywrightSetup.Tests.csproj
Project `Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj` added to the solution.
PS D:\PlaywrightActivities\Module2> code .
```

Creating an xUnit test with Playwright project

6. Install Stable Playwright NuGet Package

```
cd Module2.PlaywrightSetup.Tests
```

```
dotnet add package Microsoft.Playwright --version 1.55.0
```

7. Build the project (This generates playwright.ps1)

```
dotnet build
```

```

PS D:\PlaywrightActivities\Module2> cd Module2.PlaywrightSetup.Tests
PS D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests> dotnet add package Microsoft.Playwright --version 1.55.0
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.Playwright' into project 'D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj'.
info : Restoring packages for D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj...
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.playwright/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.playwright/index.json 242ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.playwright/1.55.0/microsoft.playwright.1.55.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.playwright/1.55.0/microsoft.playwright.1.55.0.nupkg 51ms
info : Installed Microsoft.Playwright 1.55.0 from https://api.nuget.org/v3/index.json to C:\Users\crisp\.nuget\packages\microsoft.playwright\1.55.0 with content hash iPVHeRI5EZB1gK43MC/qnINvGUV3pYh/PJn5wNmeaiZ4ARLFAfK3azf46c7/kx/LmVaknZ5rSNT1F8SZgSaWSQ==.
info :   GET https://api.nuget.org/v3/vulnerabilities/index.json
info :   OK https://api.nuget.org/v3/vulnerabilities/index.json 219ms
info :   GET https://api.nuget.org/v3-vulnerabilities/2026.05.22.05.36.46/vulnerability.base.json
info :   GET https://api.nuget.org/v3-vulnerabilities/2026.05.22.05.36.46/2026.05.22.05.36.46/vulnerability.update.json
info :   OK https://api.nuget.org/v3-vulnerabilities/2026.05.22.05.36.46/vulnerability.base.json 207ms
info :   OK https://api.nuget.org/v3-vulnerabilities/2026.05.22.05.36.46/2026.05.22.05.36.46/vulnerability.update.json 260ms
info : Package 'Microsoft.Playwright' is compatible with all the specified frameworks in project 'D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj'.
info : PackageReference for package 'Microsoft.Playwright' version '1.55.0' added to file 'D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj'.
info : Generating MSBuild file D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\obj\Module2.PlaywrightSetup.Tests.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\obj\project.assets.json
log  : Restored D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests\Module2.PlaywrightSetup.Tests.csproj (in 23.98 sec).

```

Creating an xUnit test with Playwright project

8. Install Playwright Browser Binaries

```
pwsh bin/Debug/net10.0/playwright.ps1 install
```

- This installs:
 - Chromium
 - Firefox
 - WebKit

```
PS D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests> pwsh bin/Debug/net10.0/playwright.ps1 install
Downloading Chromium 140.0.7339.16 (playwright build v1187) from https://cdn.playwright.dev/dbazure/download/playwright/builds/chromium/1187/chromium-win64.zip
148.9 MiB [=====] 100% 0.0s
Chromium 140.0.7339.16 (playwright build v1187) downloaded to C:\Users\crisp\AppData\Local\ms-playwright\chromium-1187
Downloading Chromium Headless Shell 140.0.7339.16 (playwright build v1187) from https://cdn.playwright.dev/dbazure/download/playwright/builds/chromium/1187/chromium-headless-shell-win64.zip
91.3 MiB [=====] 100% 0.0s
Chromium Headless Shell 140.0.7339.16 (playwright build v1187) downloaded to C:\Users\crisp\AppData\Local\ms-playwright\chromium_headless_shell-1187
Downloading Firefox 141.0 (playwright build v1490) from https://cdn.playwright.dev/dbazure/download/playwright/builds/firefox/1490/firefox-win64.zip
104.4 MiB [=====] 100% 0.0s
Firefox 141.0 (playwright build v1490) downloaded to C:\Users\crisp\AppData\Local\ms-playwright\firefox-1490
Downloading Webkit 26.0 (playwright build v2203) from https://cdn.playwright.dev/dbazure/download/playwright/builds/webkit/2203/webkit-win64.zip
57 MiB [=====] 100% 0.0s
Webkit 26.0 (playwright build v2203) downloaded to C:\Users\crisp\AppData\Local\ms-playwright\webkit-2203
```



Creating an xUnit test with Playwright project

9. Verify the installation

dotnet list package

```
PS D:\PlaywrightActivities\Module2\Module2.PlaywrightSetup.Tests> dotnet list package
Restore complete (0.2s)

Build succeeded in 0.6s
Project 'Module2.PlaywrightSetup.Tests' has the following package references
[net10.0]:
Top-level Package           Requested   Resolved
> coverlet.collector        6.0.4      6.0.4
> Microsoft.NET.Test.Sdk    17.14.1    17.14.1
> Microsoft.Playwright      1.55.0     1.55.0
> xunit                     2.9.3      2.9.3
> xunit.runner.visualstudio 3.1.4      3.1.4
```

Creating an xUnit test with Playwright project

10. Restore Browsers Only - Sometimes NuGet works but browser binaries are corrupted.

```
pwsh .\bin\Debug\net10.0\playwright.ps1 install --force
```


Creating an xUnit test with Playwright project

11. Full Nuclear Reset - If there is a problem with the .NET packages, run **Clean** and **Restore**. You can execute these either at the **project level** or at the **solution level**, especially when multiple projects have corrupted or missing packages.

dotnet clean

dotnet restore

dotnet build

pwsh .\bin\Debug\net10.0\playwright.ps1 install

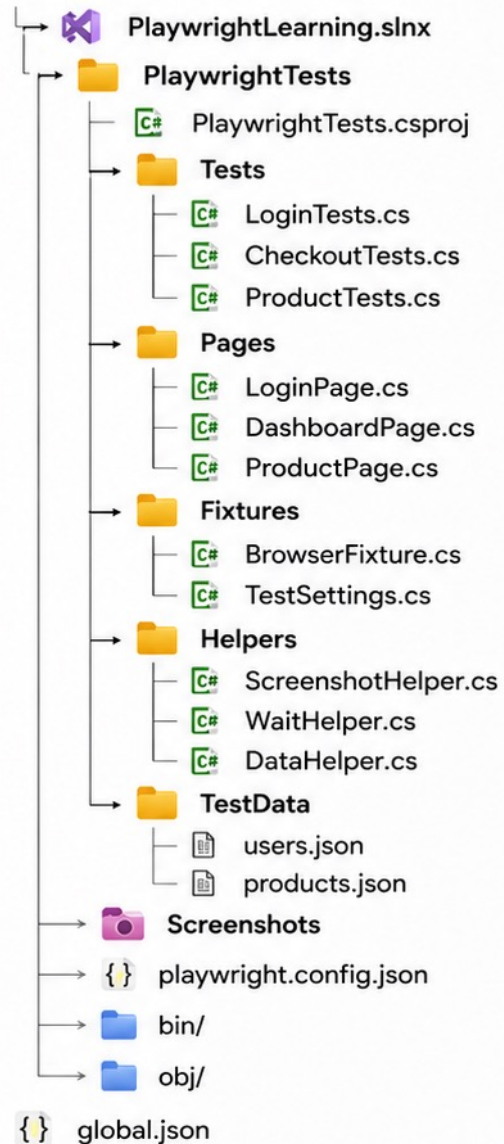
```
PS D:\PlaywrightActivities\Module2> dotnet clean  
  
Build succeeded in 0.8s  
PS D:\PlaywrightActivities\Module2> dotnet restore  
Restore complete (0.5s)  
  
Build succeeded in 0.6s
```

Most Common Problems and Solutions

Problem	Fix
Missing package	<code>dotnet restore</code>
Corrupted cache	<code>dotnet nuget locals all --clear</code>
Missing playwright.ps1	<code>dotnet build</code>
Browser missing	<code>playwright.ps1 install</code>
Broken build	<code>delete bin/obj</code>
Wrong SDK	<code>check global.json</code>

PLAYWRIGHT + XUNIT (.NET 10) PROJECT STRUCTURE

D:\PlaywrightActivities\Module2\



ITEM	PURPOSE
 PlaywrightLearning.slnx	Solution file that groups all projects.
 PlaywrightTests.csproj	Project file that defines dependencies and build settings.
 Tests/	Contains actual test cases using xUnit. (e.g., LoginTests.cs , CheckoutTests.cs)
 Pages/	Page Object Model classes that store UI elements and actions. (e.g., LoginPage.cs , DashboardPage.cs)
 Fixtures/	Shared setup and configuration for tests. (e.g., BrowserFixture.cs , TestSettings.cs)
 Helpers/	Utility/helper classes for common functions. (e.g., ScreenshotHelper.cs , WaitHelper.cs)
 TestData/	External test data files (JSON, CSV, etc.) for data-driven testing. (e.g., users.json , products.json)
 Screenshots/	Stores screenshots captured during test execution, especially on failures.
 playwright.config.json	Playwright configuration file (browser, headless, timeouts, etc.).
 bin/	Auto-generated build output (compiled binaries).
 obj/	Auto-generated temporary build artifacts.
 global.json	(Optional) Locks the .NET SDK version for the project.



Test Naming Standard

1. Test Class Naming

➤ Use:

- **<FeatureName>Tests**

➤ Examples:

LoginTests

RegisterTests

CheckoutTests

NavigationTests

DashboardTests

ProfileTests

Test Naming Standard

2. Test Method Naming - Highly readable

➤ Use:

Should_<ExpectedBehavior>_When_<Condition>()

➤ Examples:

➤ Login

public async Task *Should_LoginSuccessfully_When_ValidCredentialsAreProvided()*

public async Task *Should_DisplayError_When_InvalidPasswordIsEntered()*

➤ Registration

public async Task *Should_RegisterNewUser_When_ValidInformationIsProvided()*

public async Task *Should_ShowValidationMessage_When_EmailsEmpty()*

➤ Navigation

public async Task *Should_OpenDashboard_When_UserLogsInSuccessfully()*

Test Naming Standard

3. File Naming Convention - Match the class name.

➤ Example:

LoginTests.cs

RegisterTests.cs

DashboardTests.cs

➤ Avoid:

test1.cs

sample.cs

playwright.cs

unit1.cs

Test Naming Standard

4. Folder Naming Convention - Organize by feature.

➤ Recommended:

```
Tests/  
├── Authentication/  
│   ├── LoginTests.cs  
│   └── RegisterTests.cs  
├── Dashboard/  
│   └── DashboardTests.cs  
├── Orders/  
│   └── OrderTests.cs
```


Test Naming Standard

5. Namespace Naming Convention - Match project structure.

➤ Example:

```
namespace TrainingApp.UiTests.Authentication;
```

➤ or

```
namespace TrainingApp.UiTests.Tests.Authentication;
```

Test Naming Standard

6. Variable Naming Convention - Use meaningful names.

➤ Good:

IPage loginPage
string validEmail
string invalidPassword

➤ Avoid:

var x
var data
string a

Test Naming Standard

7. Locator Naming Convention

➤ If using Page Object Model

```
private ILocator txtEmail;  
private ILocator txtPassword;  
private ILocator btnLogin;  
private ILocator lblErrorMessage;
```

Activity 1: Running the First Browser Test

- By the end, learners can:
 - Start a prepared React + Node.js + MongoDB training app.
 - Create a Playwright .NET xUnit test project from zero.
 - Install Playwright for .NET.
 - Open a browser using Playwright.
 - Navigate to the training app.
 - Verify that the Register/Login page is visible.
 - Run the first automated browser test.

Activity 1: Running the First Browser Test

Step 1: Create module folder

```
cd D:\PlaywrightActivities\Module2  
mkdir Activity1  
cd Activity1
```

Step 2: Create Docker Compose file for the prepared app

```
code docker-compose.yml
```

Add:

```
services:
  mongo:
    image: mongo:7
    container_name: trainingapp-mongo
    ports:
      - "27017:27017"
    volumes:
      - trainingapp-mongo-data:/data/db

  backend:
    image: crisperando/trainingapp-backend:1.2
    container_name: trainingapp-backend
    ports:
      - "5000:5000"
    environment:
      - MONGO_URI=mongodb://mongo:27017/trainingapp
      - JWT_SECRET=trainingapp_secret_key
      - PORT=5000
    depends_on:
      - mongo
```



Continuation:

```
frontend:
  image: crisperando/trainingapp-frontend:1.0
  container_name: trainingapp-frontend
  ports:
    - "3000:80"
  depends_on:
    - backend
```

```
volumes:
  trainingapp-mongo-data:
```

Activity 1: Running the First Browser Test

Step 2: Create Docker Compose file for the prepared app

➤ Run:

`docker compose up -d`

➤ Check containers:

`docker ps`

➤ Open:

`http://localhost:3000`


```
PowerShell
[+] up 5/5
✓Network activity1_default          Created
    0.2s
✓Volume activity1_trainingapp-mongo-data Created
    0.0s
✓Container trainingapp-mongo        Started
    0.5s
✓Container trainingapp-backend      Started
    0.7s
✓Container trainingapp-frontend     Started
    0.8s

What's next:
    Filter, search, and stream logs from all your Compose services
    in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=activity1
PS D:\PlaywrightActivities\Module2\Activity1> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
US            PORTS                               NAMES
af60b4dec939   crisperando/trainingapp-frontend:1.0 "/docker-entrypoint...." 11 seconds ago Up 9
seconds       0.0.0.0:3000->80/tcp, [::]:3000->80/tcp trainingapp-frontend
ea80d7952c18   crisperando/trainingapp-backend:1.0 "docker-entrypoint.s..." 11 seconds ago Up 9
seconds       0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp trainingapp-backend
ea215aa4aacb   mongo:7                             "docker-entrypoint.s..." 11 seconds ago Up 1
0 seconds     0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp trainingapp-mongo
PS D:\PlaywrightActivities\Module2\Activity1> |
```



Sign up

[Already have an account?](#)

First Name*

John

Last Name*

Doe

Company

Demo Inc.

Email Address*

demo@company.com

Password



Poor

By Signing up, you agree to our [Terms of Service](#) and [Privacy Policy](#)

Create Account



Activity 1: Running the First Browser Test

Step 3: Create the xUnit test project

```
cd D:\PlaywrightActivities\Module2\Activity1  
dotnet new sln -n PlaywrightActivity1
```

- NET SDK generates .slnx, expect
➤ **PlaywrightActivity1.slnx**

Activity 1: Running the First Browser Test

Step 3: Create the xUnit test project

- Create the test project

```
dotnet new xunit -n TrainingApp.UiTests
```

- Add the project to the solution

```
dotnet sln PlaywrightActivity1.slnx add  
.\TrainingApp.UiTests\TrainingApp.UiTests.csproj
```

- Go inside the test project

```
cd TrainingApp.UiTests
```

```
PowerShell
PS D:\PlaywrightActivities\Module2\Activity1> dotnet new sln -n PlaywrightActivity1
The template "Solution File" was created successfully.

PS D:\PlaywrightActivities\Module2\Activity1> dotnet new xunit -n TrainingApp.UiTests
The template "xUnit Test Project" was created successfully.

Processing post-creation actions...
Restoring D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\TrainingApp.UiTests.csproj:
Restore succeeded.

PS D:\PlaywrightActivities\Module2\Activity1> dotnet sln PlaywrightActivity1.slnx add .\TrainingApp.
UiTests\TrainingApp.UiTests.csproj
Project `TrainingApp.UiTests\TrainingApp.UiTests.csproj` added to the solution.
PS D:\PlaywrightActivities\Module2\Activity1> cd TrainingApp.UiTests
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> |
```

Activity 1: Running the First Browser Test

Step 4: Install Playwright package

```
dotnet add package Microsoft.Playwright --version 1.55.0
```

- Restore and build:

```
dotnet restore  
dotnet build
```

- Install browsers:

```
pwsh bin\Debug\net10.0\playwright.ps1 install
```

```
PowerShell
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> dotnet add package Microsoft.Playwright --version 1.55.0
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.Playwright' into project 'D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\TrainingApp.UiTests.csproj'.
info : Restoring packages for D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\TrainingApp.UiTests.csproj...
info :   CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.05.22.23.39.40/vulnerability.base.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.05.22.23.39.40/2026.05.22.23.39.40/vulnerability.update.json
info : Package 'Microsoft.Playwright' is compatible with all the specified frameworks in project 'D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\TrainingApp.UiTests.csproj'.
info : PackageReference for package 'Microsoft.Playwright' version '1.55.0' added to file 'D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\TrainingApp.UiTests.csproj'.
info : Generating MSBuild file D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\obj\TrainingApp.UiTests.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\obj\project.assets.json
log  : Restored D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests\TrainingApp.UiTests.csproj (in 237 ms).
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> dotnet restore
Restore complete (0.5s)

Build succeeded in 0.8s
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> dotnet build
Restore complete (0.4s)
  TrainingApp.UiTests net10.0 succeeded (3.0s) → bin\Debug\net10.0\TrainingApp.UiTests.dll

Build succeeded in 3.9s
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> pwsh bin\Debug\net10.0\playwright.ps1 install
```

Activity 1: Running the First Browser Test

Step 5: Create the first browser test

- Delete the **default UnitTest1.cs** and **Create FirstBrowserTest.cs**
code FirstBrowserTest.cs

Add:

```
using Microsoft.Playwright;
using Xunit;

namespace TrainingApp.UiTests;

public class FirstBrowserTest
{
    [Fact]
    public async Task Should_Open_Training_App_Register_Page()
    {
        using IPlaywright playwright = await Playwright.CreateAsync();
        await using IBrowser browser = await playwright.Chromium.LaunchAsync(
            new BrowserTypeLaunchOptions
            {
                Headless = false,
                SlowMo = 500
            });
        IPage page = await browser.NewPageAsync();
        await page.GotoAsync("http://localhost:3000");
        await ExpectPageToContainText(page, "Create Account");
    }
}
```

continuation:

```
private static async Task ExpectPageToContainText(IPage page, string text)
{
    var locator = page.GetByText(text);
    await locator.WaitForAsync();
    Assert.True(await locator.IsVisibleAsync());
}
}
```

Activity 1: Running the First Browser Test

Step 6: Run the test

`dotnet test`

➤ Expected result:

Passed! - Failed: 0, Passed: 1

```
PowerShell
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> code FirstBrowserTest.cs
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> dotnet test
Restore complete (0.4s)
  TrainingApp.UiTests net10.0 succeeded (2.5s) → bin\Debug\net10.0\TrainingApp.UiTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.06]   Discovering: TrainingApp.UiTests
[xUnit.net 00:00:00.08]   Discovered: TrainingApp.UiTests
[xUnit.net 00:00:00.10]   Starting: TrainingApp.UiTests
[xUnit.net 00:00:04.16]   Finished: TrainingApp.UiTests
  TrainingApp.UiTests test net10.0 succeeded (5.0s)

Test summary: total: 1, failed: 0, succeeded: 1, skipped: 0, duration: 4.8s
Build succeeded in 8.4s
```

Activity 1: Running the First Browser Test

Step 7: Registration flow test

- Create new file **RegisterFlowTest.cs**

Add:

```
using Microsoft.Playwright;
using Xunit;

namespace TrainingApp.UiTests;

public class RegisterFlowTest
{
    [Fact]
    public async Task Should_Register_New_User_And_Show_Login()
    {
        using IPlaywright playwright = await Playwright.CreateAsync();
        await using IBrowser browser = await playwright.Chromium.LaunchAsync(
            new BrowserTypeLaunchOptions
            {
                Headless = false,
                SlowMo = 500
            });
        IPage page = await browser.NewPageAsync();
        await page.GotoAsync("http://localhost:3000/register");
        string uniqueEmail = $"testuser{DateTime.Now:yyyyMMddHHmmss}@demo.com";
    }
}
```

Add:

```
await page.GetByPlaceholder("John").FillAsync("John");
await page.GetByPlaceholder("Doe").FillAsync("Doe");
await page.GetByPlaceholder("Demo Inc.").FillAsync("Demo Company");
await page.GetByPlaceholder("demo@company.com").FillAsync(uniqueEmail);
await page.GetByPlaceholder("*****").FillAsync("Password");

await page.GetByRole(
    AriaRole.Button,
    new() { Name = "Create Account" }
).ClickAsync();

ILocator loginHeading = page.GetByRole(
    AriaRole.Heading,
    new() { Name = "Login" }
);

await loginHeading.WaitForAsync();
Assert.True(await loginHeading.IsVisibleAsync());
}
}
```

```
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> dotnet test
Restore complete (0.3s)
  TrainingApp.UiTests net10.0 succeeded (0.3s) → bin\Debug\net10.0\TrainingApp.UiTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.05]   Discovering: TrainingApp.UiTests
[xUnit.net 00:00:00.08]   Discovered:   TrainingApp.UiTests
[xUnit.net 00:00:00.10]   Starting:     TrainingApp.UiTests
[xUnit.net 00:00:06.03]   Finished:      TrainingApp.UiTests
  TrainingApp.UiTests test net10.0 succeeded (6.6s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 6.6s
Build succeeded in 7.6s
PS D:\PlaywrightActivities\Module2\Activity1\TrainingApp.UiTests> |
```


xUnit Lifecycle

Case 1: Basic Lifecycle

1. Create Test Class Instance
2. Execute Constructor
 - Constructor run before each test
 - Test Isolation
 - No shared state
 - Parallel-safe execution
 - Predictable behavior
3. Run Test Method

Activity 2 - xUnit and Playwright Lifecycle

```
1.Start > terminal
2.mkdir Module2/PlaywrightLifecycle
3.cd Module2/PlaywrightLifecycle
4.dotnet new sln -n PlaywrightLifecycle
5.dotnet new xunit -n PlaywrightLifecycleTests
6.dotnet sln PlaywrightLifecycle.slnx add PlaywrightLifecycleTests
7.cd PlaywrightLifecycleTests
8.dotnet add package Microsoft.Playwright --version 1.55.0
9.dotnet build
10.pwsh bin/Debug/net10.0/playwright.ps1 install
```

Activity 2 - xUnit and Playwright Lifecycle

9. Create new class file **Case1BasicLifecycleTest**

```
namespace TrainingApp.UiTests;

public class Case1BasicLifecycleTest {
    public Case1BasicLifecycleTest()
    {
        Console.WriteLine("Constructor - Setup/Initialization");
    }

    [Fact]
    public void Test1()
    {
        Console.WriteLine("Normal unit tests only");
    }
}
```



Activity 2 - xUnit and Playwright Lifecycle

9. Create new class file **Case1BasicLifecycleTest**

```
[Fact]
public async Task Test2()
{
    Console.WriteLine(@"
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
    ");
}
```

Activity 2 - xUnit and Playwright Lifecycle

10. Run the test

dotnet test --filter "TrainingApp.UiTests.Case1BasicLifecycleTest"

```
Restore complete (0.4s)
  PlaywrightLifecycleTests net10.0 succeeded (0.5s) → bin\Debug\net10.0\PlaywrightLifecycleTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.06]   Discovering: PlaywrightLifecycleTests
[xUnit.net 00:00:00.09]   Discovered:   PlaywrightLifecycleTests
[xUnit.net 00:00:00.11]   Starting:    PlaywrightLifecycleTests
Constructor - Setup/Initialization
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
Constructor - Setup/Initialization
Normal unit tests only
[xUnit.net 00:00:00.14]   Finished:    PlaywrightLifecycleTests
  PlaywrightLifecycleTests test net10.0 succeeded (0.8s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 0.8s
Build succeeded in 2.2s
```

Case 2: Setup + Clean

1. Create Test Class Instance
2. Execute Constructor
3. Run Test Method
4. Execute Dispose Method
 - Inherit the **IDisposable**
 - Run clean-up after each test

Activity 2 - xUnit and Playwright Lifecycle

11. Create new class file **Case2SetupClean**

```
namespace TrainingApp.UiTests;

public class Case2SetupClean : IDisposable
{
    public Case2SetupClean()
    {
        Console.WriteLine("Constructor - Setup/Initialization");
    }

    [Fact]
    public void Test1()
    {
        Console.WriteLine("Normal unit tests only");
    }
}
```



Activity 2 - xUnit and Playwright Lifecycle

11. Create new class file **Case2SetupClean**

```
[Fact]
public async Task Test2()
{
    Console.WriteLine(@"
Database/API calls,
Playwright UI tests,
Selenium old-style tests,
Modern browser automation
");
}

public void Dispose()
{
    Console.WriteLine("Dispose - Cleanup");
}
}
```



Activity 2 - xUnit and Playwright Lifecycle

12. Run the test

```
dotnet test --filter "TrainingApp.UiTests. Case2SetupClean"
```

```
Constructor - Setup/Initialization
Normal unit tests only
Dispose - Cleanup
Constructor - Setup/Initialization
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
Dispose - Cleanup
[xUnit.net 00:00:00.21] Finished:    PlaywrightLifecycleTests
    PlaywrightLifecycleTests test net10.0 succeeded (1.0s)
```

Case 3: Shared setup across tests

1. Create Fixture Class
2. Inherit the Fixture in the Test Class Instance
3. Inject the Fixture in the Constructor
4. Make the fixture as Read Only
5. Run Test Method

Activity 2 - xUnit and Playwright Lifecycle

13. Create new class file Case3SharedSetup

```
namespace TrainingApp.UiTests;

public class DatabaseFixture
{
    public DatabaseFixture()
    {
        Console.WriteLine("Database Connected");
    }
}
```

Activity 2 - xUnit and Playwright Lifecycle

13. Create new class file Case3SharedSetup

```
public class Case3SharedSetup : IClassFixture<DatabaseFixture>
{
    private readonly DatabaseFixture _fixture;

    public Case3SharedSetup(DatabaseFixture fixture)
    {
        _fixture = fixture;
    }
}
```

Activity 2 - xUnit and Playwright Lifecycle

13. Create new class file Case3SharedSetup

```
[Fact]
public void Test1()
{
    Console.WriteLine("Normal unit tests only");
}
```

Activity 2 - xUnit and Playwright Lifecycle

13. Create new class file Case3SharedSetup

```
[Fact]
public async Task Test2()
{
    Console.WriteLine(@"
Database/API calls,
Playwright UI tests,
Selenium old-style tests,
Modern browser automation
");
}
```

Activity 2 - xUnit and Playwright Lifecycle

14. Run the test

```
dotnet test --filter "TrainingApp.UiTests.Case3SharedSetup"
```

```
Database Connected
Normal unit tests only
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
[xUnit.net 00:00:00.14] Finished:    PlaywrightLifecycleTests
    PlaywrightLifecycleTests test net10.0 succeeded (0.8s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 0.8s
Build succeeded in 2.2s
```


Case 4: Shared Across Multiple Classes

1. Create Fixture Class

- Define the resource or state that needs to be shared.
- You can implement **IDisposable** or **IAsyncLifetime** to handle clean-up.

2. Declare the Collection Definition

- Create an empty definition class.
- This class acts as an anchor for the **[CollectionDefinition]** attribute and maps your DatabaseFixture to a unique collection name.

3. Inject and Use in Test Classes

- Decorate your test classes with **[Collection(" CollectionDefinition ")]**.
- You can then request the fixture instance directly via the class constructor.

Activity 2 - xUnit and Playwright Lifecycle

15. Create new class file DbFixture

```
public class DbFixture : IDisposable
{
    public string ConnectionString { get; private set; }
    public DbFixture()
    {
        // One-time setup: initialize database connection or docker containers
        ConnectionString = "Server=localhost;Database=TestDb;";
        Console.WriteLine("Database setup initialized.");
    }
    public void Dispose()
    {
        // One-time teardown: clean up database resources
        Console.WriteLine("Database cleaned up and disposed.");
    }
}
```

Activity 2 - xUnit and Playwright Lifecycle

16. Create new class file DatabaseCollection

```
[CollectionDefinition("DatabaseCollection")]  
public class DatabaseCollection : ICollectionFixture<DbFixture>  
{  
    // This class has no code, and is never created. Its purpose is simply  
    // to be anchor to apply [CollectionDefinition] and ICollectionFixture<>.  
}
```

Activity 2 - xUnit and Playwright Lifecycle

17. Create new class file Case4CustomerTests

```
[Collection("DatabaseCollection")]
public class Case4CustomerTests
{
    private readonly DbFixture _fixture;

    public Case4CustomerTests(DbFixture fixture)
    {
        _fixture = fixture; // Injected instance is shared with other classes
    }
    [Fact]
    public void Test_Can_Read_Customers()
    {
        var connStr = _fixture.ConnectionString;
        Console.WriteLine($"Test_Can_Read_Customers: {connStr}");
    }
}
```



Activity 2 - xUnit and Playwright Lifecycle

18. Create new class file Case4OrderTests

```
[Collection("DatabaseCollection")]
public class Case4OrderTests
{
    private readonly DbFixture _fixture;
    public Case4OrderTests(DbFixture fixture)
    {
        _fixture = fixture; // Exactly the same instance as CustomerTests
    }

    [Fact]
    public void Test_Can_Create_Order()
    {
        var connStr = _fixture.ConnectionString;
        Console.WriteLine($"Test_Can_Create_Order: {connStr}");
    }
}
```



Activity 2 - xUnit and Playwright Lifecycle

19. dotnet test

```
[xUnit.net 00:00:00.16] Starting: PlaywrightLifecycleTests
Database setup initialized.
Database Connected
Constructor - Setup/Initialization
Constructor - Setup/Initialization
Normal unit tests only
Normal unit tests only
Test_Can_Create_Order: Server=localhost;Database=TestDb;
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
Dispose - Cleanup
Constructor - Setup/Initialization
Constructor - Setup/Initialization
Normal unit tests only
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
Dispose - Cleanup
    Database/API calls,
    Playwright UI tests,
    Selenium old-style tests,
    Modern browser automation
Test_Can_Read_Customers: Server=localhost;Database=TestDb;
Database cleaned up and disposed.
[xUnit.net 00:00:00.21] Finished: PlaywrightLifecycleTests
```

Case 5: Async Lifecycle

1. Create class that inherit **IAsyncLifetime**
2. Create async InitializeAsync() for all test initialization
3. Create async test method
4. Create async DisposeAsync() for all test disposal

Activity 2 - xUnit and Playwright Lifecycle

20. Create new class file **Case5AsyncLifetime**

```
namespace TrainingApp.UiTests;
public class Case5AsyncLifetime : IAsyncLifetime
{
    public async Task InitializeAsync()
    {
        Console.WriteLine("Async Setup");
        await Task.Delay(1000);
    }
    [Fact]
    public async Task Test1()
    {
        Console.WriteLine("Test 1");
        await Task.Delay(500);
    }
}
```



Activity 2 - xUnit and Playwright Lifecycle

20. Create new class file **Case5AsyncLifetime**

```
[Fact]
public async Task Test2()
{
    Console.WriteLine("Test 2");
    await Task.Delay(500);
}

public async Task DisposeAsync()
{
    Console.WriteLine("Async Cleanup");
    await Task.Delay(1000);
}
}
```



Activity 2 - xUnit and Playwright Lifecycle

21. `dotnet test --filter "TrainingApp.UiTests.Case5AsyncLifetime"`

```
[xUnit.net 00:00:00.12] Starting: PlaywrightLifecycleTests
Async Setup
Test 2
Async Cleanup
Async Setup
Test 1
Async Cleanup
[xUnit.net 00:00:05.22] Finished: PlaywrightLifecycleTests
    PlaywrightLifecycleTests test net10.0 succeeded (5.9s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 5.9s
Build succeeded in 7.0s
```

Activity 3 - Fixture + Async Set-up

In this activity, you will how to:

- Create shareable expensive resources
- Async lifetime
- Isolated cookies
- Isolated sessions
- Isolated local storage
- Isolated authentication
- Parallel-safe execution
- Without reopening the browser every test

Activity 3 - Fixture + Async Set-up

1. Create new solution and project

```
mkdir Module2/Activity3
cd Module2/Activity3
dotnet new sln -n Activity3
dotnet new xunit -n UiTests
dotnet sln Activity3.slnx add UiTests
cd UiTests
dotnet add package Microsoft.Playwright --version 1.55.0
dotnet build
pwsh bin/Debug/net10.0/playwright.ps1 install
```



Activity 3 - Fixture + Async Set-up

2. Create new class file **PlaywrightFixture**

```
using Microsoft.Playwright;
namespace Activity3.UiTests;

public class PlaywrightFixture : IAsyncLifetime
{
    public IPlaywright Pw { get; private set; } = null!;
    public IBrowser Browser { get; private set; } = null!;
    public string BaseUrl { get; } = "http://localhost:3000";
}
```

Activity 3 - Fixture + Async Set-up

2. Create new class file **PlaywrightFixture**

```
public async Task InitializeAsync()
{
    Pw = await Playwright.CreateAsync();
    Browser = await Pw.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions
        {
            Headless = false,
            SlowMo = 500
        });
}
public async Task DisposeAsync()
{
    await Browser.CloseAsync();
    Pw.Dispose();
}
}
```



Activity 3 - Fixture + Async Set-up

3. Create new class file RegisterFlowTest

```
using Microsoft.Playwright;
namespace Activity3.UiTests;
public class RegisterFlowTest : IClassFixture<PlaywrightFixture>, IAsyncLifetime
{
    private readonly PlaywrightFixture _fixture;
    private IBrowserContext _context = null!;
    private IPage _page = null!;

    public RegisterFlowTest(PlaywrightFixture fixture)
    {
        _fixture = fixture;
    }
}
```

Activity 3 - Fixture + Async Set-up

3. Create new class file **RegisterFlowTest**

```
public async Task InitializeAsync()
{
    _context = await _fixture.Browser.NewContextAsync();
    _page = await _context.NewPageAsync();
}
```


Activity 3 - Fixture + Async Set-up

```
[Fact]
public async Task Should_Register_New_User_And_Show_Login()
{
    await _page.GotoAsync($"({_fixture.BaseUrl})/register");
    string uniqueEmail = $"testuser{DateTime.Now:yyyyMMddHHmmss}@demo.com";
    await _page.GetByPlaceholder("John").FillAsync("John");
    await _page.GetByPlaceholder("Doe").FillAsync("Doe");
    await _page.GetByPlaceholder("Demo Inc.").FillAsync("Demo Company");
    await _page.GetByPlaceholder("demo@company.com").FillAsync(uniqueEmail);
    await _page.GetByPlaceholder("*****").FillAsync("Password");
    await _page.GetByRole(AriaRole.Button, new() { Name = "Create Account" })
        .ClickAsync();
    ILocator loginHeading = _page.GetByRole(
        AriaRole.Heading,
        new() { Name = "Login" }
    );
    await Assertions.Expect(loginHeading).ToBeVisibleAsync();
}
```



Activity 3 - Fixture + Async Set-up

```
public async Task DisposeAsync()
{
    await _context.CloseAsync();
}
}
```

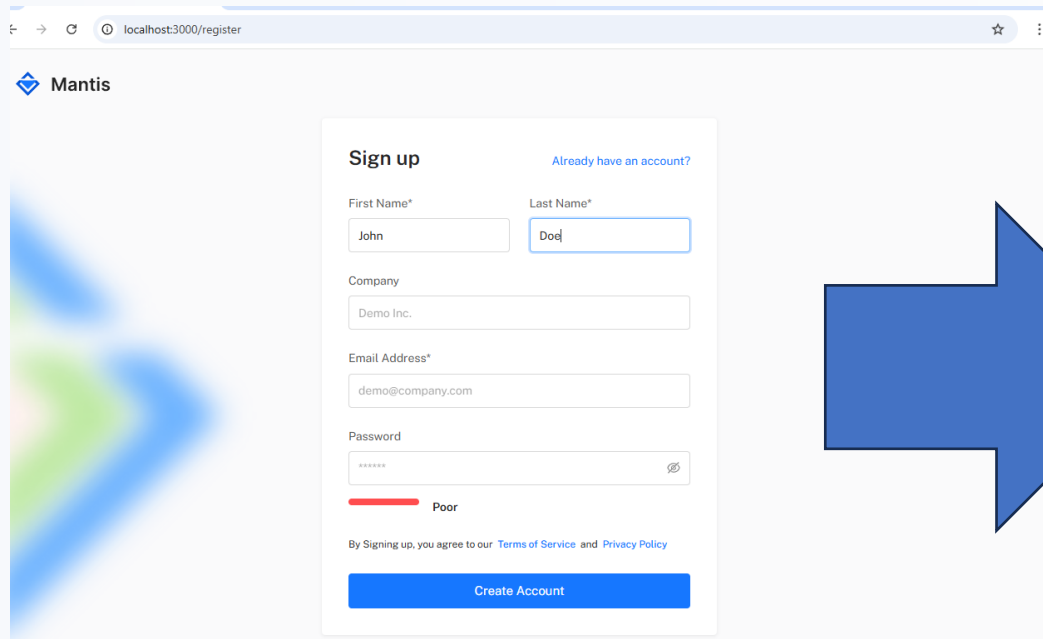
Activity 3 - Fixture + Async Set-up

4. Run the Docker

☐	▼	●	activity1	-	-	-
☐		●	trainingapp-fi	af60b4dec939	crisperand	3000:80 ↗
☐		●	trainingapp-b	ea80d7952c18	crisperand	5000:5000 ↗
☐		●	trainingapp-n	ea215aa4aacb	mongo:7	27017:27017 ↗

Activity 3 - Fixture + Async Set-up

5. Run the test **dotnet test**



localhost:3000/register

Mantis

Sign up [Already have an account?](#)

First Name* Last Name*

Company

Email Address*

Password 

Poor

By Signing up, you agree to our [Terms of Service](#) and [Privacy Policy](#)

Create Account

```
Restore complete (0.4s)
  UiTests net10.0 succeeded (2.4s) → bin\Debug\net10.0\UiTests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit)
[xUnit.net 00:00:00.06]   Discovering: UiTests
[xUnit.net 00:00:00.08]   Discovered:   UiTests
[xUnit.net 00:00:00.10]   Starting:    UiTests
[xUnit.net 00:00:05.73]   Finished:   UiTests
  UiTests test net10.0 succeeded (6.5s)

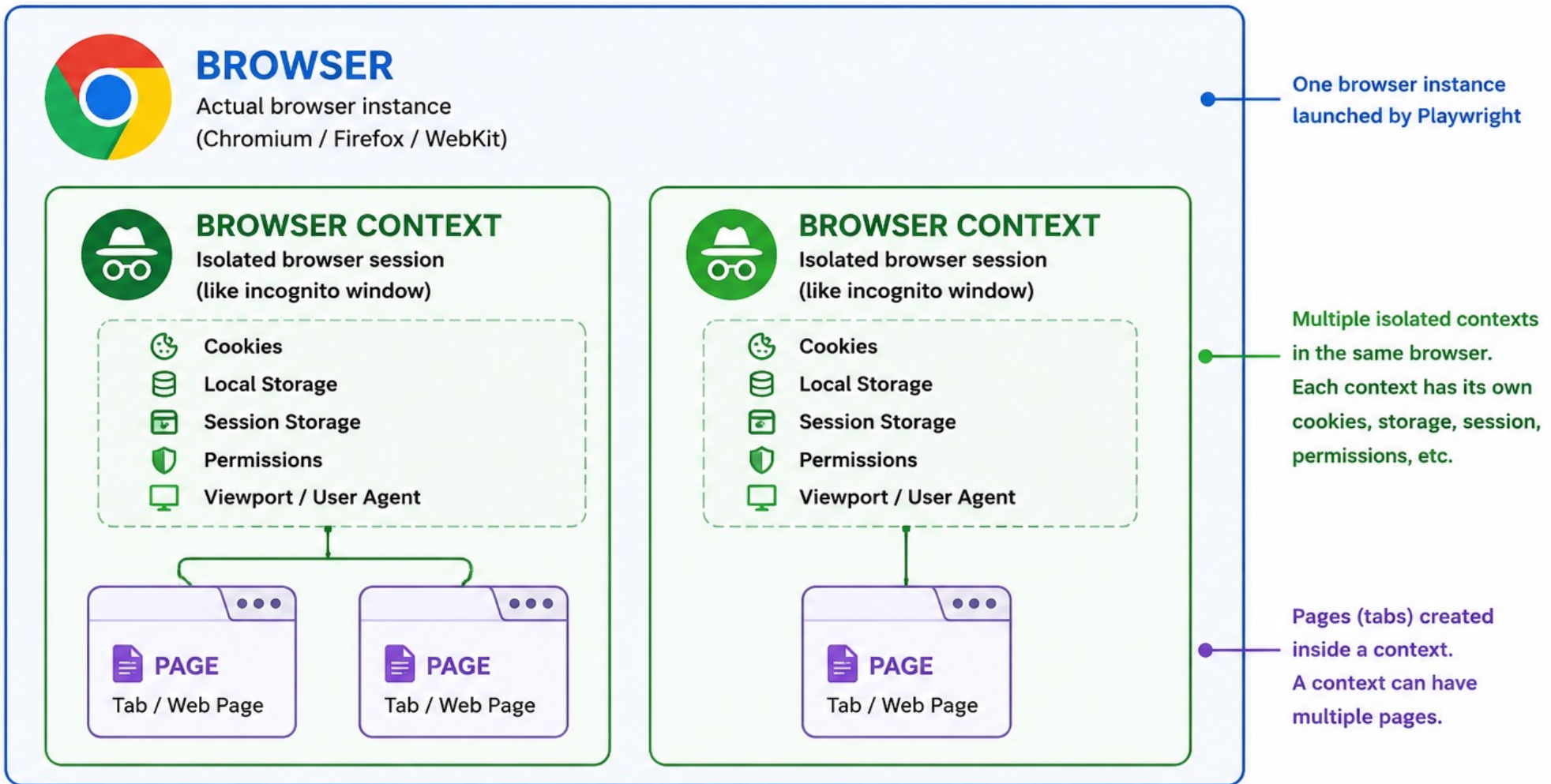
Test summary: total: 1, failed: 0, succeeded: 1, skipped: 0, duration: 6.4s
Build succeeded in 9.7s
```

Module 3

UI Automation Basics

Playwright Architecture

Browser → Context → Page



Browser, context, page concepts

1. **Browser** is the actual browser instance.

➤ This opens Chromium, Firefox, or WebKit.

➤ Example:

```
IBrowser browser = await playwright.Chromium.LaunchAsync(  
    new BrowserTypeLaunchOptions  
    {  
        Headless = false,  
        SlowMo = 500  
    });
```

Browser, context, page concepts

2. **BrowserContext** is an isolated browser session.

➤ It is like an incognito/private browser window.

➤ Each context has its own:

- Cookies
- Local storage
- Session storage
- Login session
- Permissions
- Viewport settings

➤ Example:

```
IBrowserContext adminContext = await browser.NewContextAsync();
```

```
IBrowserContext userContext = await browser.NewContextAsync();
```


Browser, context, page concepts

3. Page is one browser tab.

➤ A browser context can contain multiple pages or tabs.

➤ You use Page to:

- Open URLs
- Click buttons
- Fill inputs
- Read text
- Assert visible elements
- Take screenshots

➤ Example:

```
await page.GotoAsync("http://localhost:3000/register");  
await page.GetByRole(AriaRole.Button, new() { Name = "Create Account" }).ClickAsync();
```

Browser, context, page concepts

3. Page is one browser tab.

➤ Recommended Structure

➤ Instead of this:

```
IPage page = await browser.NewPageAsync();
```

➤ Use this:

```
IBrowserContext context = await browser.NewContextAsync();
```

```
IPage page = await context.NewPageAsync();
```

Navigating pages

- **Common Navigation Methods**

Method	Purpose
GotoAsync()	Open URL
WaitForURLAsync()	Wait for redirect
GoBackAsync()	Browser back
GoForwardAsync()	Browser forward
ReloadAsync()	Refresh page

Navigating pages

- Basic: simply open URL

```
await page.GotoAsync("https://example.com");
```

- Wait a page redirects after clicking a button

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Create Account" }  
) ClickAsync();  
  
await page.WaitForURLAsync("**/login");
```

Navigating pages

- **Verify Current URL**

`Assert.Contains("/login", page.Url);`

- **Equivalent to clicking browser Back**

`await page.GoBackAsync();`

- **Browser Refresh**

`await page.ReloadAsync();`

Locators and Selector Strategy

- **Locator** is an object used to
 - Find elements
 - Wait automatically
 - Interact with UI controls
 - Retry until the element is ready

Locators and Selector Strategy

```
ILocator loginButton = page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Login" });  
  
await loginButton.ClickAsync();
```

Why Playwright Locators Are Better Than Traditional Selectors?

- Traditional automation tools often use

`"#btnLogin"`

`".submit-button"`

`"xpath=//div[2]/button"`

- These are fragile because
 - UI layout changes break tests
 - CSS classes often change
 - XPath becomes difficult to maintain

Why Playwright Locators Are Better Than Traditional Selectors?

- Playwright uses
 - Role-based locators
 - Text-based locators
 - Label-based locators
 - Placeholder-based locators
 - Test ID locators
- These imitate how real users identify elements.

Buttons

- Case 1: Different tags, but still a button

<button>Submit Form</button>

<input type="submit" value="Submit Form" />

Buttons

Case 1: Different tags, but still a button

`<button>Create Account</button>`

`<input type="submit" value="Submit Form" />`

- To target the button

```
await page.GetByRole(AriaRole.Button, new() { Name = "Create Account" }).ClickAsync();  
await page.GetByRole(AriaRole.Button, new() { Name = "Submit Form" }).ClickAsync();
```

Buttons

Case 2: Two buttons both say Submit Form.

```
<form id="registerForm">  
  <input type="submit" value="Submit Form" />  
</form>
```

```
<form id="contactForm">  
  <input type="submit" value="Submit Form" />  
</form>
```

Buttons

Case 2: Two buttons both say Submit Form.

- To target the button

```
await page
    .Locator("#registerForm")
    .GetByRole(AriaRole.Button, new() { Name = "Submit Form" })
    .ClickAsync();
```

```
await page
    .Locator("#contactForm")
    .GetByRole(AriaRole.Button, new() { Name = "Submit Form" })
    .ClickAsync();
```

Textboxes

Case 1: Textbox with aria-label

```
<input type="email" aria-label="Email" />
```

To target:

```
await page.GetByRole(AriaRole.Textbox,  
new() { Name = "Email" }).FillAsync("user@test.com");
```

Textboxes

Case 2: Textbox Within the Label Tag

```
<label>  
    Username  
    <input type="text" />  
</label>
```

To target:

```
await page.GetByLabel("Username")  
    .FillAsync("admin");
```

Textboxes

Case 3: Textbox can be target using the placeholder

```
<input type="text" placeholder="Username" />
```

To target:

```
await page.GetByPlaceholder("Username")  
    .FillAsync("admin");
```


Textboxes

Case 4: Textbox can be target using the id

```
<input type="text" id="username" />
```

To target:

```
await page.Locator("#username")  
    .FillAsync("admin");
```

Textboxes

Case 4: Textbox can be target by name attribute

```
<input type="text" name="username" />
```

To target:

```
await page.Locator("[name='username']")  
    .FillAsync("admin");
```

Forms, links

- Given we have the following form:

```
<form>  
<input aria-label="First Name" />  
<input aria-label="Last Name" />  
<button>Submit</button>  
</form>
```

Forms, links

- A group of input fields usually submitted together.

```
await page.GetByLabel("First Name").FillAsync("John");  
await page.GetByLabel("Last Name").FillAsync("Doe");  
await page.GetByRole(AriaRole.Button, new() { Name = "Submit" }).ClickAsync();
```

Links

- Given the following link
`<a href="/login">Login</a>`

To target:

```
await page.GetByRole(AriaRole.Link, new() { Name = "Login" }).ClickAsync();
```

Assertions and validation

- In Playwright with xUnit, we commonly validate using:

`Assert.True(condition);`

`Assert.Equal(expected, actual);`

`Assert.Contains(expectedText, actualText);`

Assertions and validation

Example 1: Validate Text Is Visible

- This checks if **Create Account** appears on the page.

```
var heading = page.GetText("Create Account");
```

```
await heading.WaitForAsync();
```

```
Assert.True(await heading.IsVisibleAsync());
```

Assertions and validation

Example 2: Validate Page URL

- This checks if the browser is currently on the register page.

```
await page.GotoAsync("http://localhost:3000/register");
```

```
Assert.Contains("/register", page.Url);
```


Assertions and validation

Example 3: Validate Input Value

- This confirms that the textbox really contains the expected email.

```
var emailTextbox = page.GetByPlaceholder("demo@company.com");
```

```
await emailTextbox.FillAsync("testuser@demo.com");
```

```
string value = await emailTextbox.InputValueAsync();
```

```
Assert.Equal("testuser@demo.com", value);
```

Assertions and validation

- **Example 5: Validate After Clicking Create Account**
- This validates that after registration, the application shows the **Login** page or login section.

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Create Account" }  
).ClickAsync();
```

```
var loginText = page.GetText("Login");
```

```
await loginText.WaitForAsync();
```

```
Assert.True(await loginText.IsVisibleAsync());
```

Module 4

Testing React.js + Material UI Applications

Challenges of modern React UI testing

1. Dynamic rendering

- React updates the UI without reloading the page.

- Example challenge

- ```
await page.GetText("Saved successfully").IsVisibleAsync();
```

- This may fail if the text appears after an API call.

- Better (Playwright waits automatically)

- ```
await Expect(page.GetText("Saved successfully")).ToBeVisibleAsync();
```

Challenges of modern React UI testing

2. Components generate complex HTML

- A simple Material UI button may produce several nested elements.

```
<button class="MuiButton-root">  
  <span>Create Account</span>  
</button>
```

➤ Avoid targeting generated CSS classes like
`page.Locator(".MuiButton-root")`

➤ Better

```
page.GetByRole(AriaRole.Button, new() { Name = "Create Account" })
```

Challenges of modern React UI testing

3. Repeated text or repeated buttons

- React pages often reuse components.

➤ Example

```
<button>Edit</button>
```

```
<button>Edit</button>
```

```
<button>Edit</button>
```

➤ Better

```
var row = page.GetByRole(AriaRole.Row, new() { Name = "John Doe" });  
await row.GetByRole(AriaRole.Button, new() { Name = "Edit" }).ClickAsync();
```

Challenges of modern React UI testing

4. Asynchronous API calls

- The UI may depend on backend data.

➤ Common issue (Fails because data has not loaded yet)

```
await page.GetByText("Product A").ClickAsync();
```

➤ Better

```
await Expect(page.GetByText("Product A")).ToBeVisibleAsync();  
await page.GetByText("Product A").ClickAsync();
```

Challenges of modern React UI testing

5. Unstable selectors

➤ Avoid selectors like:

`page.Locator("div:nth-child(3) > button")`

➤ Better options:

`page.GetByRole(...)`

`page.GetByLabel(...)`

`page.GetByPlaceholder(...)`

`page.GetByText(...)`

`page.GetByTestId(...)`

Challenges of modern React UI testing

6. State changes after user actions

- Buttons may become disabled, dialogs may open, or validation messages may appear.

➤ Example:

```
await page.GetByRole(AriaRole.Button, new() { Name = "Submit" }).ClickAsync();
```

```
await Expect(page.GetByText("Email is required")).ToBeVisibleAsync();
```

Challenges of modern React UI testing

7. Styling libraries hide real behavior

- Material UI, Bootstrap, and custom components may look like buttons or textboxes but are not always simple HTML controls.

➤ Example issue:

```
<div role="button">Save</div>
```

➤ Use role-based locator:

```
await page.GetByRole(AriaRole.Button, new() { Name = "Save" }).ClickAsync();
```

Playwright Locator / Selector in IPage

Method	Purpose	Best Used For	Example
page.GetByRole()	Finds elements using ARIA roles and accessible names	Buttons, links, dialogs, menus, checkboxes	page.GetByRole(AriaRole.Button, new() { Name = "Save" })
page.GetByText()	Finds elements by visible text	Labels, messages, headings, buttons	page.GetByText("Login Successful")
page.GetByLabel()	Finds form controls associated with a label	Textboxes, dropdowns, checkboxes	page.GetByLabel("Username")
page.GetByPlaceholder()	Finds inputs by placeholder text	Forms using placeholders	page.GetByPlaceholder("Enter email")
page.GetByAltText()	Finds elements using image alt text	Images and icons	page.GetByAltText("Company Logo")
page.GetByTitle()	Finds elements using title attribute	Tooltip-based elements	page.GetByTitle("Close")

Playwright Locator / Selector in IPage

Method	Purpose	Best Used For	Example
page.GetByTestId()	Finds elements using data-testid attribute	Stable automation testing	page .GetByTestId("submit-button")
page.Locator()	Generic locator using CSS/XPath/text selectors	Advanced/custom selection	page .Locator(".login-form button")
page.FrameLocator()	Access elements inside iframe	iframe automation	page .FrameLocator("#user-frame")
page.QuerySelectorAsync()	Returns first matching element (legacy style)	Older style element handling	await page .QuerySelectorAsync("#email")
page.QuerySelectorAllAsync()	Returns all matching elements	Multiple element processing	await page .QuerySelectorAllAsync(".card")

Dynamic DOM behavior

Dynamic behavior	Meaning	Playwright concern
Conditional rendering	Appears only after login, click, or API result	Locator must wait for it
Re-rendering	React destroys and recreates elements	Avoid storing raw element handles
Loading state	Spinner shown first, real content later	Assert final state
Disabled to enabled	Button becomes clickable after valid input	Playwright waits before click
Modal/dropdown creation	Element is inserted only when opened	Open first, then locate inside
List updates	Table rows/cards change after search/filter	Use scoped locators and assertions

Dynamic DOM behavior

- Bad Pattern

```
await page
  .GetByRole(AriaRole.Button, new() { Name = "Create Account" })
  .ClickAsync();

Assert.True(await page.GetText("Login").IsVisibleAsync());
```

Dynamic DOM behavior

- Bad Pattern Timeline

Time	What happens
T1	Click button
T2	API request starts
T3	React updates state
T4	Login page renders

Dynamic DOM behavior

- Fixes for previous bad pattern code

```
await page
  .GetByRole(AriaRole.Button, new() { Name = "Create Account" })
  .ClickAsync();

await page.GetText("Login").WaitForAsync();
Assert.True(await page.GetText("Login").IsVisibleAsync());
```


Dynamic DOM behavior

- Good Pattern

```
await page
  .GetByRole(AriaRole.Button, new() { Name = "Create Account" })
  .ClickAsync();

await Expect(page.GetByText("Login")).ToBeVisibleAsync();
```

Expect() for Page Assertions

- Example

```
await Expect(page).ToHaveURLAsync("http://localhost:3000/login");
```

Assertion	Purpose
ToHaveTitleAsync()	Validate page title
ToHaveURLAsync()	Validate current URL
ToHaveScreenshotAsync()	Validate visual screenshot
ToMatchAriaSnapshotAsync()	Validate accessibility snapshot

Expect() for Visibility Assertions

- Example

```
await Expect(page.GetText("Login")).ToBeVisibleAsync();
```

Assertion	Purpose
<code>ToBeVisibleAsync()</code>	Element is visible
<code>ToBeHiddenAsync()</code>	Element is hidden
<code>ToBeInViewportAsync()</code>	Element is inside viewport

Expect() for State Assertions

- Example

```
await Expect(page.GetByRole(AriaRole.Button)).ToBeDisabledAsync();
```

Assertion	Purpose
ToBeEnabledAsync()	Element enabled
ToBeDisabledAsync()	Element disabled
ToBeEditableAsync()	Input editable
ToBeReadOnlyAsync()	Input readonly
ToBeCheckedAsync()	Checkbox/radio checked
ToBeEmptyAsync()	Empty element/input
ToBeFocusedAsync()	Element focused

Expect() for Text Assertions

- Example

```
await Expect(page.GetByRole(AriaRole.Heading)).ToContainTextAsync("Dashboard");
```

Assertion	Purpose
ToContainTextAsync()	Partial text match
ToHaveTextAsync()	Exact text match
ToHaveValueAsync()	Input value
ToHaveValuesAsync()	Multi-select values
ToHaveAccessibleNameAsync()	ARIA accessible name
ToHaveAccessibleDescriptionAsync()	ARIA description

Expect() for Attribute / CSS Assertions

- Example

```
await Expect(page.GetByRole(AriaRole.Heading)).ToContainTextAsync("Dashboard");
```

Assertion	Purpose
ToContainTextAsync()	Partial text match
ToHaveTextAsync()	Exact text match
ToHaveValueAsync()	Input value
ToHaveValuesAsync()	Multi-select values
ToHaveAccessibleNameAsync()	ARIA accessible name
ToHaveAccessibleDescriptionAsync()	ARIA description

MUI UI Elements and Playwright Locators

MUI Component	Common HTML behavior	Recommended Playwright Locator
Button	Usually renders as <button>	<code>GetByRole(AriaRole.Button, new() { Name = "Save" })</code>
Dialog	Renders with role="dialog"	<code>GetByRole(AriaRole.Dialog)</code>
Menu	Renders with role="menu"	<code>GetByRole(AriaRole.Menu)</code>
Menu item	Renders with role="menuitem"	<code>GetByRole(AriaRole.Menuitem, new() { Name = "Profile" })</code>
Table	Uses table roles	<code>GetByRole(AriaRole.Table)</code>
Table row	Uses row role	<code>GetByRole(AriaRole.Row)</code>
Dropdown / Select	Often renders as combobox or button	<code>GetByRole(AriaRole.Combobox)</code> or <code>GetByLabel("Status")</code>
TextField	Usually input	<code>GetByLabel("Email")</code>



To target: MUI Button

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Create Account" }  
).ClickAsync();
```


To target: MUI Dialog

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Open Dialog" }  
).ClickAsync();
```

```
await Assertions.Expect(  
    page.GetByRole(AriaRole.Dialog)  
).ToBeVisibleAsync();
```

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Confirm" }  
).ClickAsync();
```

To target: MUI Menu

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Account" }  
).ClickAsync();  
  
await Assertions.Expect(  
    page.GetByRole(AriaRole.Menu)  
).ToBeVisibleAsync();  
  
await page.GetByRole(  
    AriaRole.MenuItem,  
    new() { Name = "Logout" }  
).ClickAsync();
```

To target: MUI Table

```
var table = page.GetByRole(AriaRole.Table);  
  
await Assertions.Expect(table).ToBeVisibleAsync();  
  
await Assertions.Expect(  
    table.GetByText("John Doe")  
) .ToBeVisibleAsync();
```

To target: MUI Dropdown / Select

```
await page.GetByLabel("Status").ClickAsync();
```

```
await page.GetByRole(  
    AriaRole.Option,  
    new() { Name = "Active" }  
).ClickAsync();
```

To target: MUI Dropdown / Select

```
// Alternative for some MUI Select components
await page.GetByRole(
    AriaRole.Combobox,
    new() { Name = "Status" }
).ClickAsync();

await page.GetByRole(
    AriaRole.Option,
    new() { Name = "Active" }
).ClickAsync();
```

Auto-waiting and synchronization

- Playwright automatically waits for the element to become targetable before performing the event.
 - Present in DOM
 - Visible
 - Stable
 - Enabled
 - Ready to receive click events

Auto-waiting and synchronization

Default waiting time of Playwright

Type	Default Timeout
Actions (ClickAsync, FillAsync, etc.)	30 seconds
Assertions (Expect)	5 seconds
Navigation (GotoAsync)	30 seconds

Auto-waiting and synchronization

- Playwright will keep retrying/actionability-checking for up to **30 seconds** before throwing an error.

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Save" }  
).ClickAsync();
```


Auto-waiting and synchronization

- This retries for up to **5 seconds** by default.

```
await Assertions.Expect(  
    page.GetByText("Success")  
).ToBeVisibleAsync();
```

Change timeout globally & synchronization

- **Page / action timeout**

`page.SetDefaultTimeout(60000);`

Now all actions wait up to 60 seconds.

- **Navigation timeout**

`page.SetDefaultNavigationTimeout(60000);`

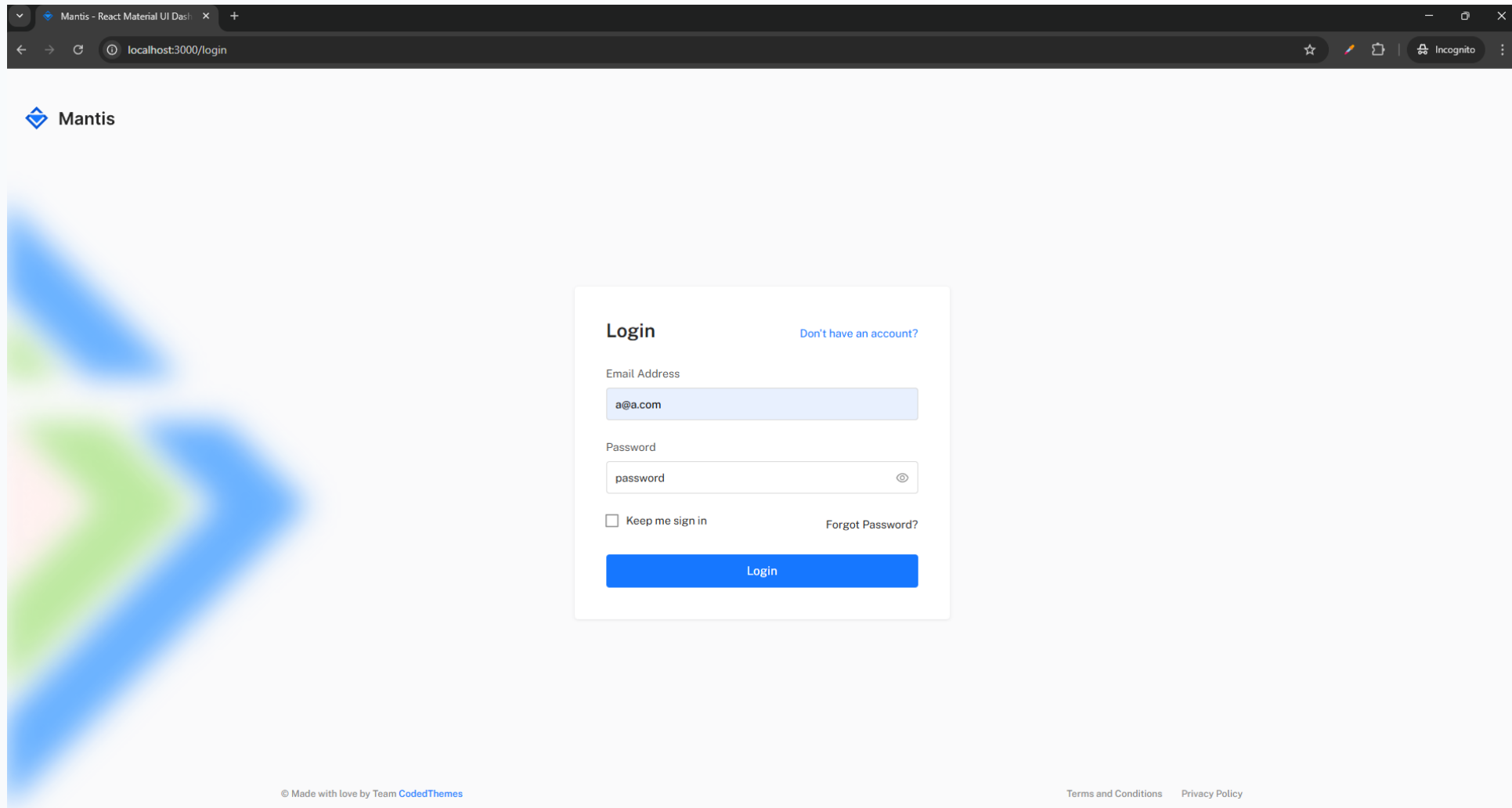
Change timeout globally & synchronization

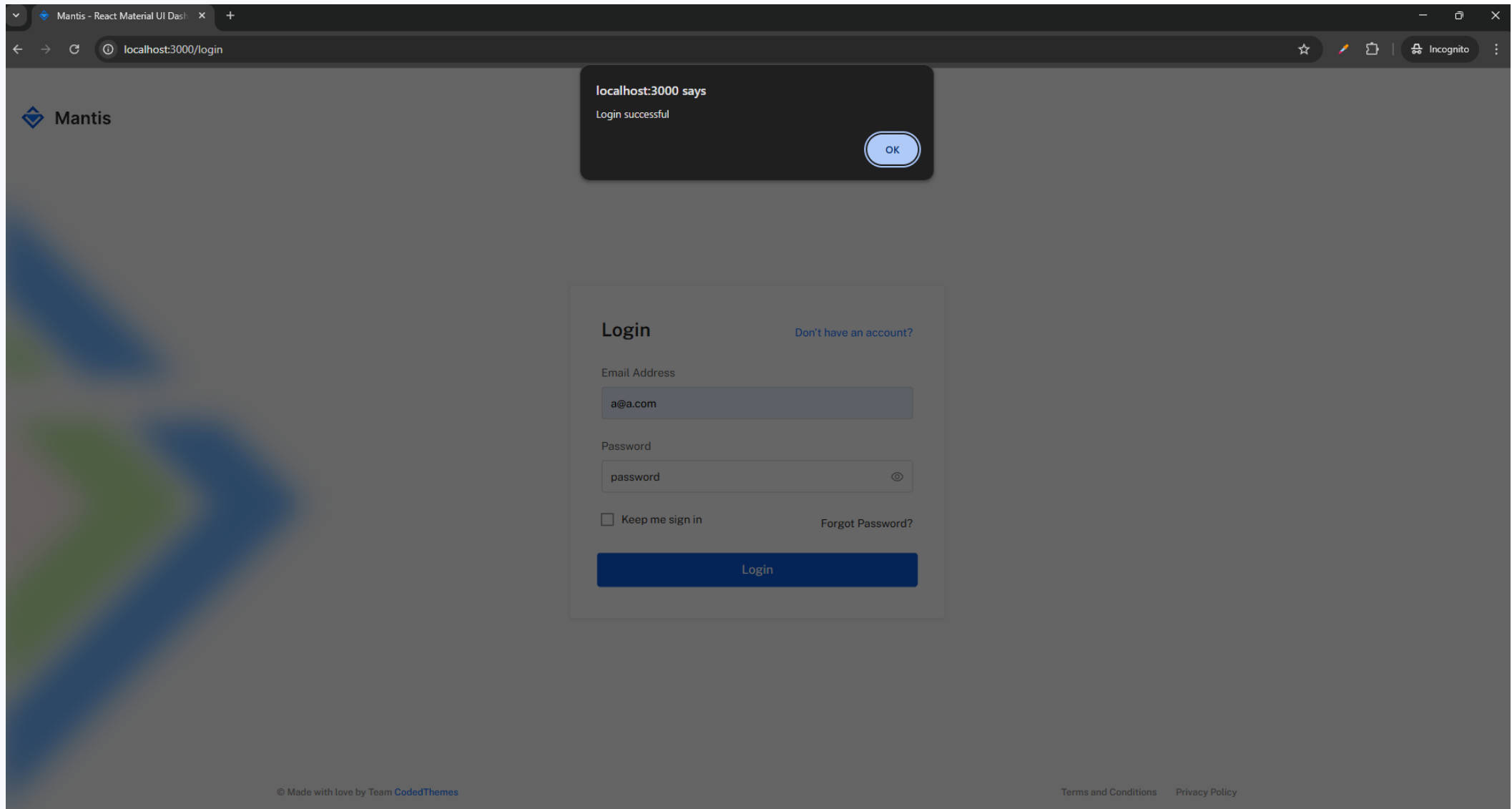
- **Change timeout per operation**
- This waits only 10 seconds.

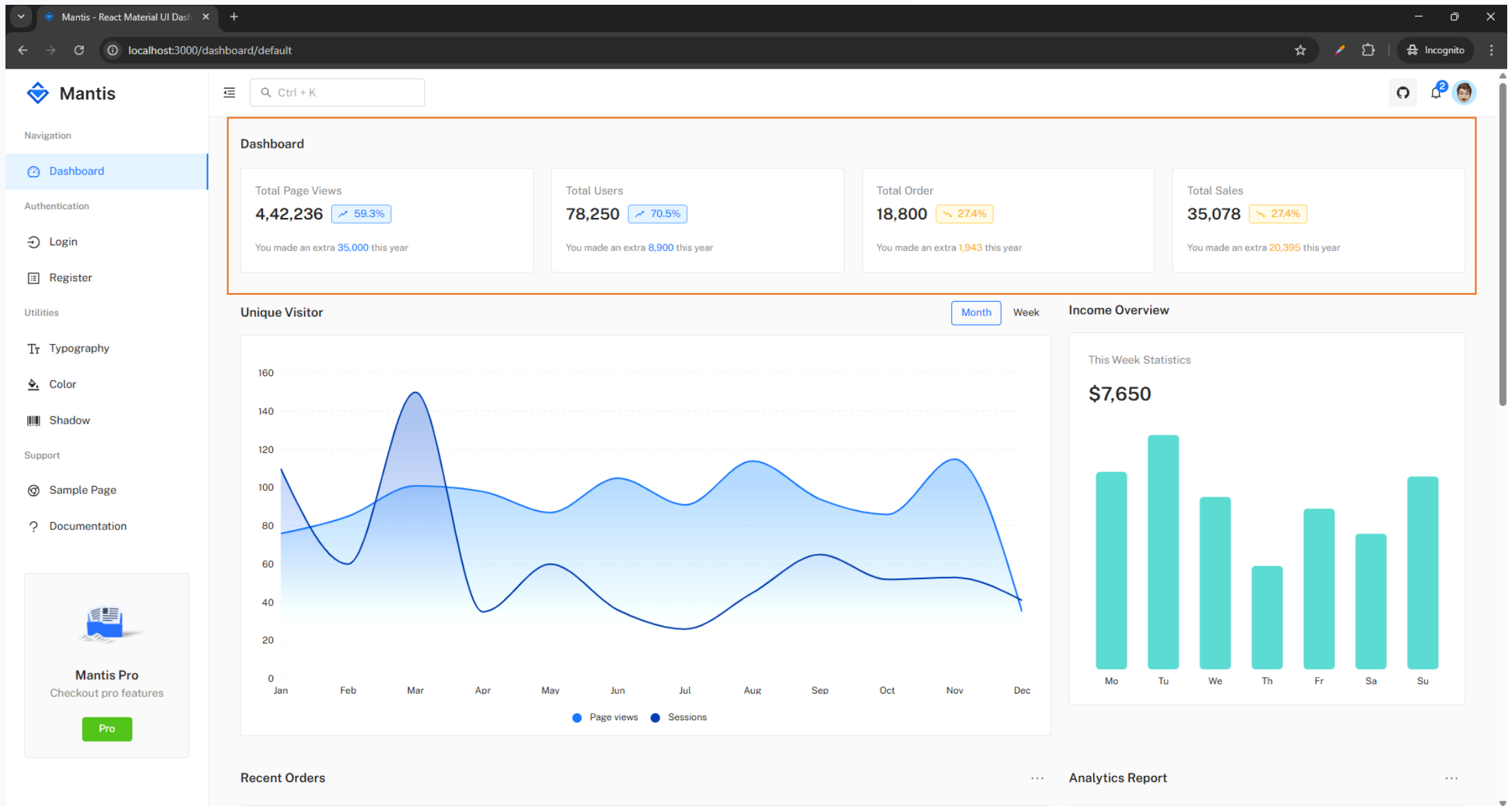
```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Submit" }  
) ClickAsync(new()  
{  
    Timeout = 10000  
});
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

- In this activity, you will learn how to:
 - Login using an existing account.
 - Handle a browser popup alert.
 - Validate successful navigation to the dashboard.
 - Check dashboard cards:
 - Total Page Views
 - Total Users
 - Total Order
 - Total Sales
 - Use Playwright Expect() assertions.
 - Use IClassFixture and IAsyncLifetime for shared setup.







Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

1. Create new solution and project

```
mkdir Activity4
cd Activity4
dotnet new sln -n Activity4.UiTesting
dotnet new xunit -n TrainingApp.UiTests
dotnet sln Activity4.UiTesting.slnx add .\TrainingApp.UiTests\TrainingApp.UiTests.csproj
cd .\TrainingApp.UiTests
dotnet add package Microsoft.Playwright --version 1.55.0
dotnet build
pwsh bin/Debug/net10.0/playwright.ps1 install
mkdir Fixtures
```



Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

2. Create Playwright Fixture \Fixtures\PlaywrightFixture.cs

```
using Microsoft.Playwright;
namespace TrainingApp.UiTests.Fixtures;

public class PlaywrightFixture : IAsyncLifetime
{
    public IPlaywright Playwright { get; private set; } = null!;
    public IBrowser Browser { get; private set; } = null!;
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

2. Create Playwright Fixture `\Fixtures\PlaywrightFixture.cs`

```
public async Task InitializeAsync()
{
    Playwright = await Microsoft.Playwright.Playwright.CreateAsync();

    Browser = await Playwright.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions
        {
            Headless = false,
            SlowMo = 300
        });
}
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

2. Create Playwright Fixture \Fixtures\PlaywrightFixture.cs

```
public async Task DisposeAsync()
{
    await Browser.CloseAsync();
    Playwright.Dispose();
}
}
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

3. Create Dashboard Test File \DashboardTests.cs

```
using Microsoft.Playwright;
using TrainingApp.UiTests.Fixtures;
using static Microsoft.Playwright.Assertions;
namespace TrainingApp.UiTests;

public class DashboardTests : IClassFixture<PlaywrightFixture>
{
    private readonly PlaywrightFixture _fixture;

    public DashboardTests(PlaywrightFixture fixture)
    {
        _fixture = fixture;
    }
}
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

3. Create Dashboard Test File \DashboardTests.cs

```
[Fact]
public async Task Should_Login_Handle_Popup_And_Check_Dashboard_Statistics()
{
    await using IBrowserContext context = await _fixture.Browser.NewContextAsync();
    IPage page = await context.NewPageAsync();

    await page.GotoAsync("http://localhost:3000/login");

    await page.GetByPlaceholder("Enter email address").FillAsync("a@a.com");
    await page.GetByPlaceholder("Enter password").FillAsync("password");
}
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

3. Create Dashboard Test File \DashboardTests.cs

```
string? dialogMessage = null;

page.Dialog += async (_, dialog) =>
{
    dialogMessage = dialog.Message;
    await dialog.AcceptAsync();
};

await page.GetByRole(
    AriaRole.Button,
    new() { Name = "Login" }
).ClickAsync();

Assert.Equal("Login successful", dialogMessage);
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

3. Create Dashboard Test File \DashboardTests.cs

```
await Expect(page.GetByRole(  
    AriaRole.Heading,  
    new() { Name = "Dashboard" }  
)).ToBeVisibleAsync();  
  
await Expect(  
    page.GetByRole(AriaRole.Heading, new() { Name = "Total Page Views" })  
).ToBeVisibleAsync();  
  
await Expect(page.GetByText("4,42,236")).ToBeVisibleAsync();
```

Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

3. Create Dashboard Test File \DashboardTests.cs

```
await Expect(  
    page.GetByRole(AriaRole.Heading, new() { Name = "Total Users" })  
).ToBeVisibleAsync();  
  
await Expect(page.GetText("78,250")).ToBeVisibleAsync();  
  
await Expect(  
    page.GetByRole(AriaRole.Heading, new() { Name = "Total Order" })  
).ToBeVisibleAsync();
```


Activity 4: Login, Handle Popup, and Validate Dashboard Statistics

3. Create Dashboard Test File \DashboardTests.cs

```
        await Expect(page.GetText("18,800")).ToBeVisibleAsync();

        await Expect(
            page.GetByRole(AriaRole.Heading, new() { Name = "Total Sales" })
        ).ToBeVisibleAsync();

        await Expect(page.GetText("35,078")).ToBeVisibleAsync();
    }
}
```

Module 5

Reliable Locator Strategy

Role-based locators

- A role defines the purpose of an element in the UI.

HTML	Role
<button>	button
	link
<input type="textbox">	textbox
<table>	table
<dialog>	dialog
	img

Instead of targeting fragile HTML structures like

```
div > button:nth-child(2)
```

or unstable classes like

```
.MuiButton-root.css-abcd123
```

Resolve UI element whose role is **Button** and whose visible accessible name is **"Save"**

```
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Save" }  
).ClickAsync();
```

Common Roles in Playwright

Role	Typical HTML
AriaRole.Button	<button>
AriaRole.Link	<a>
AriaRole.Textbox	<input>
AriaRole.Checkbox	<input type="checkbox">
AriaRole.Radio	radio buttons
AriaRole.Dialog	modal dialogs
AriaRole.Table	<table>
AriaRole.Row	table rows
AriaRole.Cell	table cells
AriaRole.Combobox	dropdowns
AriaRole.MenuItem	menus

Example 1 — Button

- HTML

- `<button>Create Account</button>`

- Playwright

- `await page.GetByRole(
 AriaRole.Button,
 new() { Name = "Create Account" }
).ClickAsync();`

Example 2 — Link

- HTML

```
<a href="/login">Login</a>
```

- Playwright

```
await page.GetByRole(  
    AriaRole.Link,  
    new() { Name = "Login" }  
).ClickAsync();
```

Example 3 — Textbox

- HTML

```
<input type="text" aria-label="Username" />
```

- Playwright:

```
await page.GetByRole(  
    AriaRole.Textbox,  
    new() { Name = "Username" }  
).FillAsync("admin");
```


Example 4 — Material UI Button

- In MUI or React apps
`<div role="button">Save</div>`
- Still works because it creates an accessibility role
`role="button"`
- Even though it is NOT a real `<button>` element
`await page.GetByRole(
 AriaRole.Button,
 new() { Name = "Save" }
) ClickAsync();`

Benefits of Role-Based Locators

Benefit	Explanation
Stable	Less affected by layout/CSS changes
Readable	Easy to understand
Accessibility-friendly	Encourages proper ARIA usage
Recommended by Playwright	Best practice
Works well with React/MUI	Handles dynamic components better

Text locators

- **Text locators** are selectors that find elements based on the visible text shown to the user.
 - They are useful when:
 - The UI text is stable
 - The element has no reliable ID/class
 - You want tests to behave like a real user interaction

Common Text Locator Methods

Method	Purpose	Example
GetByText()	Finds element by visible text	"Login"
Filter(new() { HasText = ... })	Filters locator containing text	tables/cards
Locator("text=...")	Old text engine syntax	legacy style
HasText	Match text inside another locator	nested elements
HasNotText	Exclude text	filtering

GetByText()

- This is the modern and stable approach.
 - Visible text
 - Partial match by default
 - Normalized whitespace
- Example:
`<button>Login</button>`
- Playwright finds it using
`await page.GetByText("Login").ClickAsync();`

Exact Match

- If multiple elements contain similar text:

```
<button>Login</button>
```

```
<button>Login Now</button>
```

- Use `Exact = true`.

```
await page.GetByText(  
    "Login",  
    new() { Exact = true }  
).ClickAsync();
```

Partial Text Match

- Example:

`<h1>Welcome to Training Portal</h1>`

- This works because Playwright supports partial matching.
- Text matching is generally case-insensitive.
`await page.GetByText("Training").ClickAsync();`

Using Filter + HasText

- Very useful for:
 - tables
 - cards
 - lists
 - repeated components

Using Filter + HasText

- Example HTML:

```
<tr>
  <td>John</td>
  <td>Admin</td>
</tr>
<tr>
  <td>Mary</td>
  <td>User</td>
</tr>
```

- Target only the row containing "John":

```
ILocator row = page
    .Locator("tr")
    .Filter(new() { HasText = "John" });

await row.ClickAsync();
```

Using Filter + HasText

- Another example:

```
<tr>
  <td>John</td>
  <td>Admin</td>
  <td><button>Edit</button></td>
</tr>
```

- Playwright:

```
ILocator row = page
  .GetByRole(AriaRole.Row)
  .Filter(new() { HasText = "John" });
```

```
await row.GetByRole(
  AriaRole.Button,
  new() { Name = "Edit" }
).ClickAsync();
```

Nested Text Matching

- Example:

```
<div class="card">  
  <h3>Product A</h3>  
  <button>Buy</button>  
</div>
```

- Find the **Buy** button inside **Product A** card:

```
ILocator card = page  
  .Locator(".card")  
  .Filter(new() { HasText = "Product A" });  
await card.GetByRole(AriaRole.Button, new() { Name = "Buy" })  
  .ClickAsync();
```

Test IDs

- Test IDs are **special attributes added to HTML elements** specifically for UI automation testing.
- They help make locators:
 - Stable
 - Predictable
 - Independent from UI text or styling
 - Resistant to UI redesigns

Common Test ID Attributes

- Most common
 - **data-testid**
- Other organizations use
 - **data-test**
 - **data-qa**
 - **data-cy**

Test ID

- HTML

`<button data-testid="save-button">Save</button>`

- Playwright

`await page.GetByTestId("save-button").ClickAsync();`

- Without Test ID (Fragile)

`await page.GetByText("Save").ClickAsync();`

Test ID Best Practices

- **Good** (Clear and meaningful)
`data-testid="register-button"`
- **Bad** (Too generic)
`data-testid="btn1"`

Combining Test IDs with Other Locators

- To target an element on specific Test ID, ARIA, and Case-sensitivity

```
await page
    .GetByTestId("user-table")
    .GetByRole(
        AriaRole.Button,
        new() { Name = "Edit", Exact = true }
    )
    .ClickAsync();
```


Avoiding brittle CSS/XPath selectors

- CSS selectors based on deep hierarchy
- XPath using indexes
- Auto-generated class names from React/MUI
- Selectors tied to layout or styling
- These are common causes of unstable UI tests.

Example of a Brittle CSS Selector

- Problem:
 - Any UI layout change breaks the test
 - Adding a new div can fail the locator
 - Hard to read and maintain

```
await page.Locator(  
    "div > div:nth-child(2) > form > button"  
).ClickAsync();
```

Example of a Brittle CSS Selector

- Problem:
 - Depends on element position
 - React/MUI rendering changes can break it
 - Difficult to debug

```
await page.Locator(  
    "//div[2]/form/button[1]"  
).ClickAsync();
```

Handling dynamic elements

- Dynamic elements are UI elements that **appear, disappear, update, enable, disable, or move after an action.**

Dynamic behavior	Example
Element appears later	success message, modal, menu
Element disappears	loading spinner
Text changes	status: Pending → Approved
Button becomes enabled	Submit button after validation
Table row is added	new customer appears
React re-renders element	MUI component reloads

Handling dynamic elements

- Use **condition-based waiting**, not fixed delays.

```
await Assertions.Expect(locator).ToBeVisibleAsync();  
await Assertions.Expect(locator).ToBeHiddenAsync();  
await Assertions.Expect(locator).ToBeEnabledAsync();  
await Assertions.Expect(locator).ToHaveTextAsync("Completed");  
await Assertions.Expect(locator).ToContainTextAsync("Success");
```

Handling dynamic elements

- **Bad pattern**

- **Task.Delay(3000)** waits blindly.
- It may be too short on slow machines or too long on fast machines.

```
await page.GetByRole(AriaRole.Button, new() { Name = "Save" }).ClickAsync();  
await Task.Delay(3000);  
Assert.True(await page.GetByText("Saved successfully").IsVisibleAsync());
```

Handling dynamic elements

- **Better pattern: use Expect**

- Playwright keeps checking until the condition becomes true or timeout is reached.

- **Wait for loading spinner to disappear**

```
ILocator loading = page.GetByText("Loading...");  
await Assertions.Expect/loading).ToBeHiddenAsync();
```

Handling dynamic elements

- Wait for button to become enabled

```
ILocator submitButton = page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Submit" }  
);  
  
await Assertions.Expect(submitButton).ToBeEnabledAsync();  
  
await submitButton.ClickAsync();
```


Handling dynamic elements

- Wait for table row to appear

```
ILocator userTable = page.GetByTestId("user-table");

await Assertions.Expect(
    userTable.GetByRole(AriaRole.Row, new() { Name = new Regex("John Doe") })
).ToBeVisibleAsync();
```

Module 6

Forms, Tables, Dialogs, and Validation Testing

Form input automation

```
// Textbox by label
await page.GetByLabel("Email").FillAsync("test@example.com");

// Textbox by placeholder
await page.GetByPlaceholder("Enter password").FillAsync("Password123!");

// Button by role
await page.GetByRole(
    AriaRole.Button,
    new() { Name = "Login" }
).ClickAsync();

// Checkbox
await page.GetByLabel("Remember me").CheckAsync();
```



Form input automation

```
// Radio button
await page.GetByLabel("Male").CheckAsync();

// Dropdown / select
await page.GetByLabel("Country").SelectOptionAsync("Philippines");

// Clear textbox
await page.GetByLabel("Email").ClearAsync();

// Type slowly, character by character
await page.GetByLabel("Search").PressSequentiallyAsync("Laptop");
```

Client-side validation testing

- Client-side validation testing means verifying **validation that runs in the browser before the request reaches the backend.**
- Example validations:
 - Required fields
 - Email format
 - Password minimum length
 - Number range
 - Invalid date
 - Disabled submit button
 - Error messages

Client-side validation testing

```
await page.GetByRole(AriaRole.Button, new() { Name = "Create Account" })
.ClickAsync();

await Assertions.Expect(page.GetByText("First name is required"))
.ToBeVisibleAsync();

await Assertions.Expect(page.GetByText("Last name is required"))
.ToBeVisibleAsync();

await Assertions.Expect(page.GetByText("Email is required"))
.ToBeVisibleAsync();

await Assertions.Expect(page.GetByText("Password is required"))
.ToBeVisibleAsync();
```

Better pattern: test one rule clearly

```
[Fact]
public async Task Should_Show_Invalid_Email_Message()
{
    using IPlaywright playwright = await Playwright.CreateAsync();
    await using IBrowser browser = await playwright.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions
        {
            Headless = false,
            SlowMo = 300
        });
    IPage page = await browser.NewPageAsync();
    await page.GotoAsync("http://localhost:3000/register");
    await page.GetByPlaceholder("demo@company.com").FillAsync("invalid-email");

    await page.GetByRole(AriaRole.Button, new() { Name = "Create Account" })
        .ClickAsync();

    await Assertions.Expect(page.GetText("Invalid email address"))
        .ToBeVisibleAsync();
}
```

Common client-side validation tests

Scenario	What to test
Empty form	Required error messages appear
Invalid email	Email format message appears
Short password	Password length message appears
Valid input	Error messages disappear
Invalid form	Submit button stays disabled
Valid form	Submit button becomes enabled

MUI dialog automation

- **MUI dialog automation** means testing how a Material UI dialog opens, displays content, accepts input, confirms/cancels, and closes using Playwright.

Activity 5 - MUI Dialog Automation

- This activity demonstrates:
 - We will automate this dialogs
<https://mantisdashboard.com/components-overview/dialogs>
 - Opening MUI dialogs from multiple buttons.
 - Using GetByRole(AriaRole.Dialog) for stable dialog detection.
 - Closing dialogs using common dialog buttons.
 - Falling back to Escape when no obvious close button exists.
 - Avoiding brittle CSS/XPath selectors.

Activity 5 - MUI Dialog Automation

1. Prepare the project

```
mkdir Activity5
cd Activity5
dotnet new sln -n Activity5.MuiDialogAutomation
dotnet new xunit -n Activity5.MuiDialogAutomation.Tests
dotnet sln Activity5.MuiDialogAutomation.slnx add Activity5.MuiDialogAutomation.Tests
cd .\Activity5.MuiDialogAutomation.Tests
dotnet add package Microsoft.Playwright --version 1.55.0
dotnet build
pwsh bin\Debug\net10.0\playwright.ps1 install
```



Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
using Microsoft.Playwright;
using System.Text.RegularExpressions;
using Xunit;
using static Microsoft.Playwright.Assertions;

namespace Activity5.MuiDialogAutomation.Tests;

public class MantisDialogTests
{
    private const string DialogsUrl =
        "https://mantisdashboard.com/components-overview/dialogs";
```



Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
[Fact]
public async Task Should_Automate_Common_Mui_Dialogs()
{
    using IPlaywright playwright = await Playwright.CreateAsync();

    await using IBrowser browser = await playwright.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions
        {
            Headless = false,
            SlowMo = 300
        });

    IPage page = await browser.NewPageAsync();

    await page.GotoAsync(DialogsUrl);
```



Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
await OpenDialogAsync(page, "Open Simple Dialog");  
await CloseDialogAsync(page);  
  
await OpenDialogAsync(page, "Open Alert Dialog");  
await ClickDialogButtonAsync(page, "Agree");  
  
await OpenDialogAsync(page, "Open Form Dialog");
```

Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
ILocator dialog = page.GetByRole(AriaRole.Dialog);

await dialog.GetByRole(
    AriaRole.Textbox,
    new() { Name = "Email Address" }
).FillAsync("student@test.com");

await ClickDialogButtonAsync(page, "Subscribe");

}
```

Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
private static async Task OpenDialogAsync(IPage page, string buttonName)
{
    await page.GetByRole(
        AriaRole.Button,
        new() { Name = buttonName }
    ).ClickAsync();

    await Expect(page.GetByRole(AriaRole.Dialog)).ToBeVisibleAsync();
}
```


Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
private static async Task ClickDialogButtonAsync(IPage page, string
buttonName)
{
    ILocator dialog = page.GetByRole(AriaRole.Dialog);
    ILocator button = dialog.GetByRole(
        AriaRole.Button,
        new()
        {
            Name = buttonName,
            Exact = true
        });

    await button.ClickAsync();
    await Expect(dialog).ToBeHiddenAsync();
}
```



Activity 5 - MUI Dialog Automation

2. Create the test file **MantisDialogTests**

```
private static async Task CloseDialogAsync(IPage page)
{
    ILocator dialog = page.GetByRole(AriaRole.Dialog);
    ILocator closeButton = dialog.GetByRole(
        AriaRole.Button,
        new()
        {
            NameRegex = new Regex("close", RegexOptions.IgnoreCase)
        });

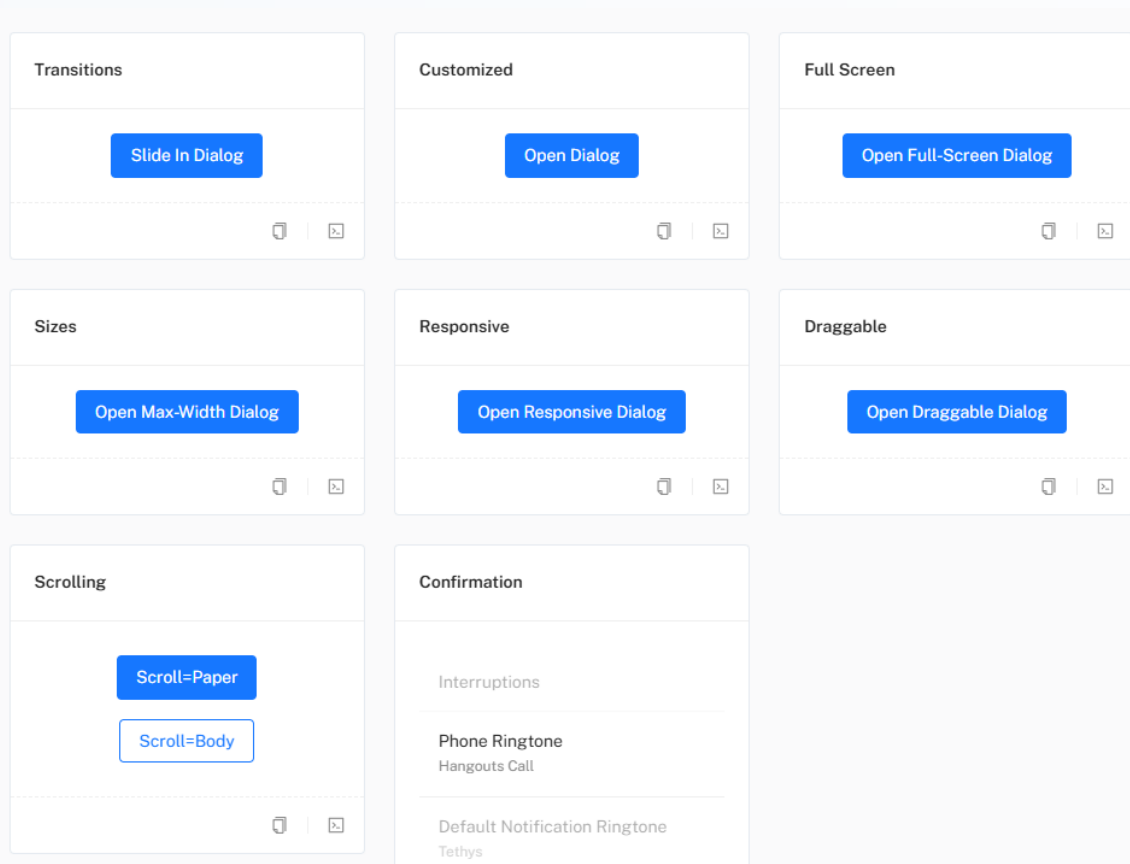
    await closeButton.ClickAsync();

    await Expect(dialog).ToBeHiddenAsync();
}
}
```



Activity 5 - MUI Dialog Automation

3. Your challenge! Automate the rest of the dialogs as seen on screenshot



Activity 6 - MUI table search/filter/pagination

- You will practice:
 - Logging in to a web application
 - Navigating directly to a protected page
 - Testing global table search
 - Testing column-based filtering
 - Testing dropdown filtering
 - Testing pagination buttons
 - Using stable Playwright locators
 - Validating table results using assertions

Activity 6 - MUI table search/filter/pagination

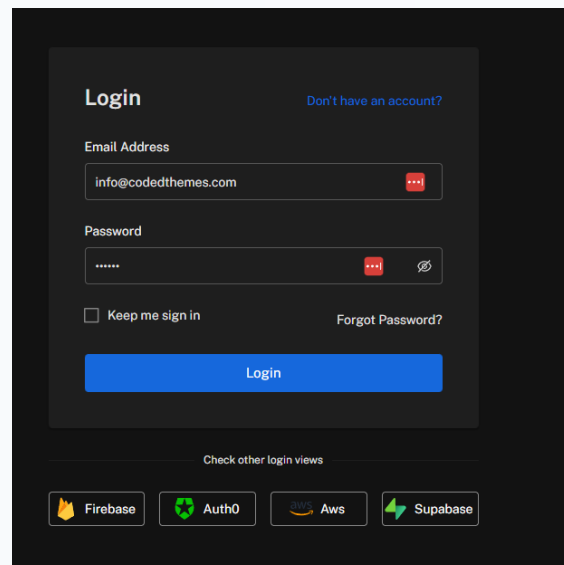
1. Prepare your project

```
mkdir Activity6  
cd Activity6  
dotnet new sln -n Activity6.MuiTableAutomation  
dotnet new xunit -n Activity6.MuiTableAutomation.Tests  
dotnet sln Activity6.MuiTableAutomation.slnx add Activity6.MuiTableAutomation.Tests  
cd .\Activity6.MuiTableAutomation.Tests  
dotnet add package Microsoft.Playwright --version 1.55.0  
dotnet build  
pwsh bin\Debug\net10.0\playwright.ps1 install
```

Activity 6 - MUI table search/filter/pagination

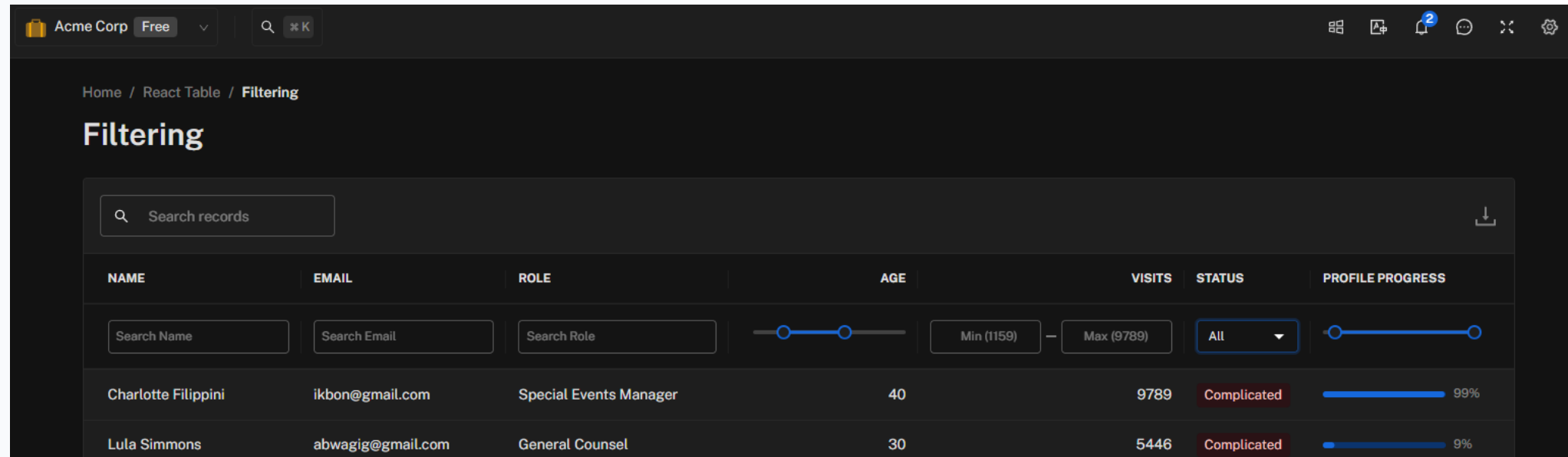
2. Automate the login <https://mantisdashboard.com/login>

3. Click login button



Activity 6 - MUI table search/filter/pagination

4. Go to <https://mantisdashboard.com/tables/react-table/filtering>
5. Automate the global search, search by name, email, role, and status



Acme Corp Free

Home / React Table / Filtering

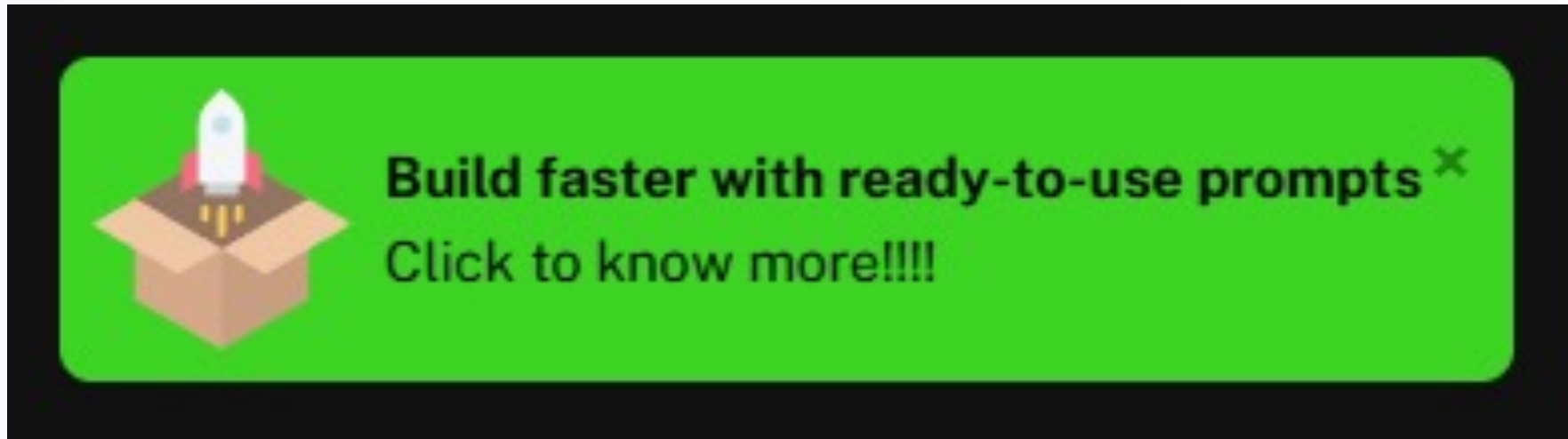
Filtering

Search records

NAME	EMAIL	ROLE	AGE	VISITS	STATUS	PROFILE PROGRESS
Search Name	Search Email	Search Role	<input type="range"/>	Min (1159) - Max (9789)	All	<input type="range"/>
Charlotte Filippini	ikbon@gmail.com	Special Events Manager	40	9789	Complicated	99%
Lula Simmons	abwagig@gmail.com	General Counsel	30	5446	Complicated	9%

Activity 6 - MUI table search/filter/pagination

6. Challenge! Close the Green Promo Box in <https://mantisdashboard.com/login>



Module 7

Page Object Model with xUnit

POM

- It is a design pattern where you separate
 - **UI element locators**
 - **Page actions**
 - **Test logic**

Why use POM

1. Improves Maintainability

- Without POM, locators are scattered everywhere in test files.
- If the UI changes, you must update ALL tests manually.

Example of Improves Maintainability

```
<input placeholder="Email Address">
```

```
// Without POM
await page.GetByPlaceholder("Enter email address").FillAsync("admin@test.com");
await page.GetByPlaceholder("Enter password").FillAsync("password");
await page.GetByRole(
    AriaRole.Button,
    new() { Name = "Login" }
).ClickAsync();
```

```
// With POM
await loginPage.LoginAsync(
    "admin@test.com",
    "password"
);
```

Why use POM

2. Reduces Duplicate Code

```
// Without POM: Repeated in many tests.  
await page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Login" }  
).ClickAsync();  
  
// With POM: Reusable everywhere.  
await loginPage.ClickLoginAsync();
```

Why use POM

3. Makes Tests Easier to Read

```
// Without POM: Harder to read business intent.  
await page.GetByPlaceholder("Email").FillAsync("a@a.com");  
await page.GetByPlaceholder("Password").FillAsync("password");  
await page.GetByRole(AriaRole.Button, new() { Name = "Login" }).ClickAsync();  
await Assertions.Expect(page.GetText("Dashboard")).ToBeVisibleAsync();
```

```
// With POM: Much cleaner and business-oriented.  
await loginPage.LoginAsync("a@a.com", "password");  
await dashboardPage.VerifyDashboardLoadedAsync();
```

Why use POM

4. Separates Responsibilities which result to cleaner architecture

Responsibility	Location
UI locators	Page class
UI actions	Page class
Assertions/business flow	Test class

Why use POM

6. Easier UI Change Management

- If MUI changes:

`<button>Submit</button>`

- to:

`<button>Save</button>`

- Only update inside the page object:

`GetByRole(AriaRole.Button,
new() { Name = "Save" })`

- *Tests remain unchanged.*

Why use POM

7. Encourages Stable Locator Strategy

➤ POM encourages use of stable Playwright locators:

- GetByRole
- GetByLabel
- GetByTestId
- GetByPlaceholder

```
public ILocator SubmitButton =>
    _page.GetByRole(
        AriaRole.Button,
        new() { Name = "Submit" }
    );
```

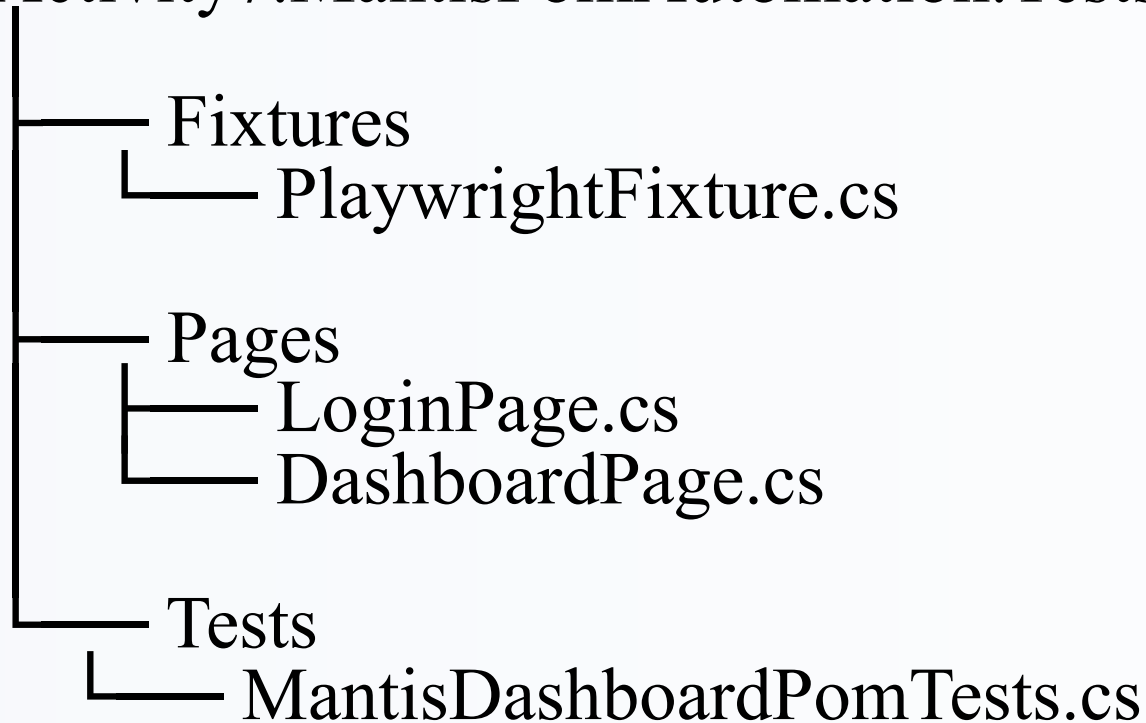
Activity 7: Page Object Model with xUnit

- For this activity, you will learn
 - Page classes
 - Reusable actions
 - Reusable assertions
 - Separating test logic from page interaction

Activity 7: Page Object Model with xUnit

- **Project Structure**

Activity7.MantisPomAutomation.Tests



Login

[Don't have an account?](#)

Email Address

a@a.com



Password

password



☐ Keep me sign in

[Forgot Password?](#)

Login

localhost:3000 says
Login successful

OK

Login

Don't have an account?

Email Address

a@a.com

Password

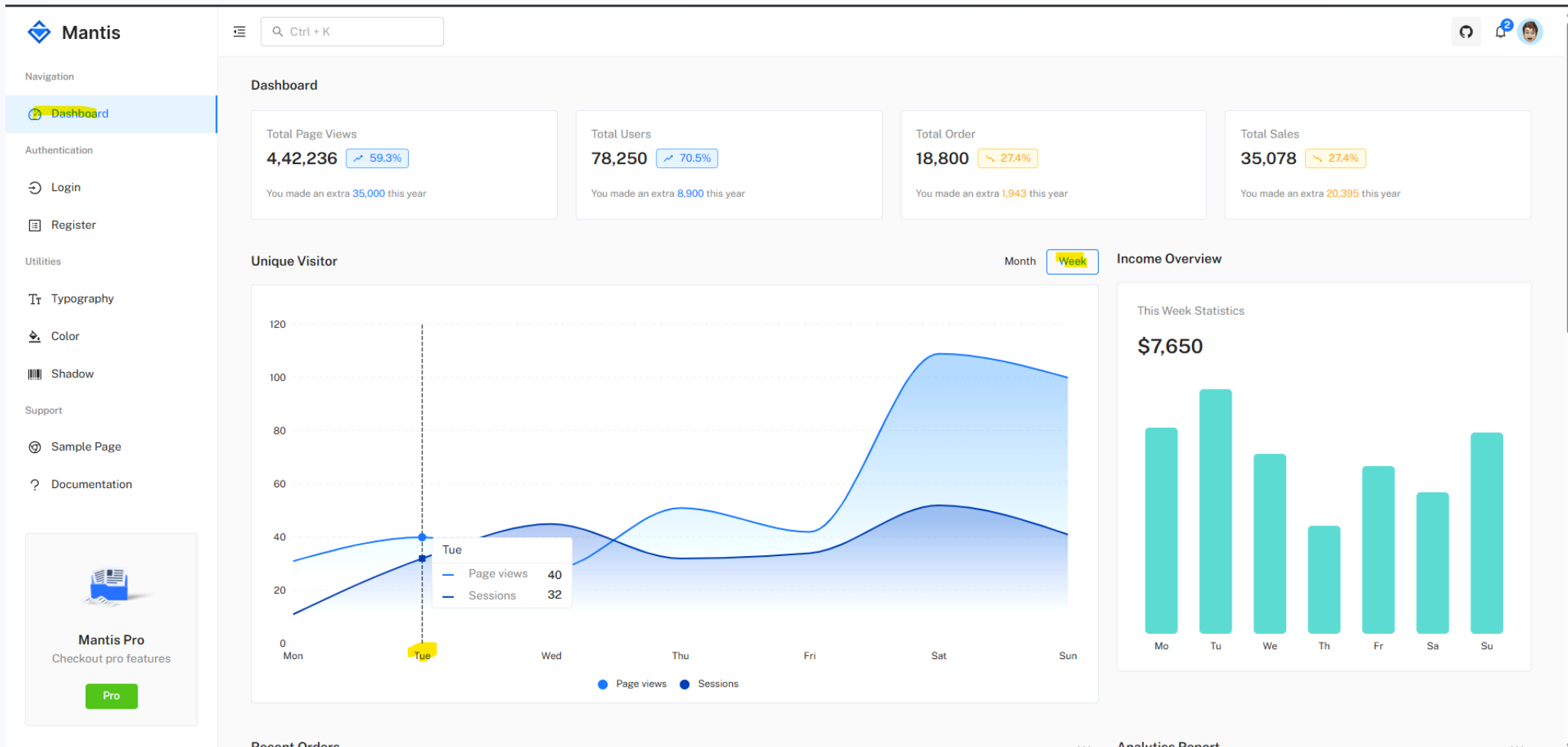
password

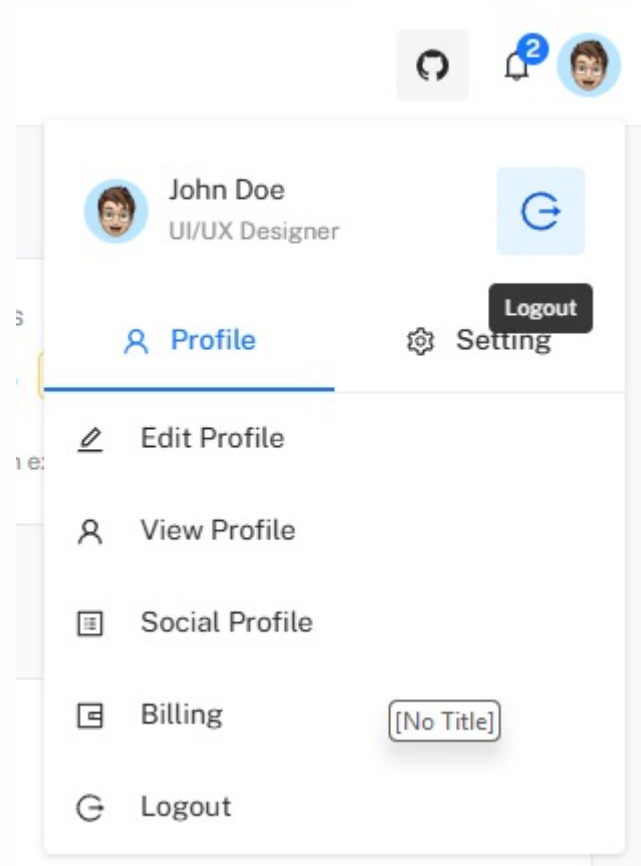
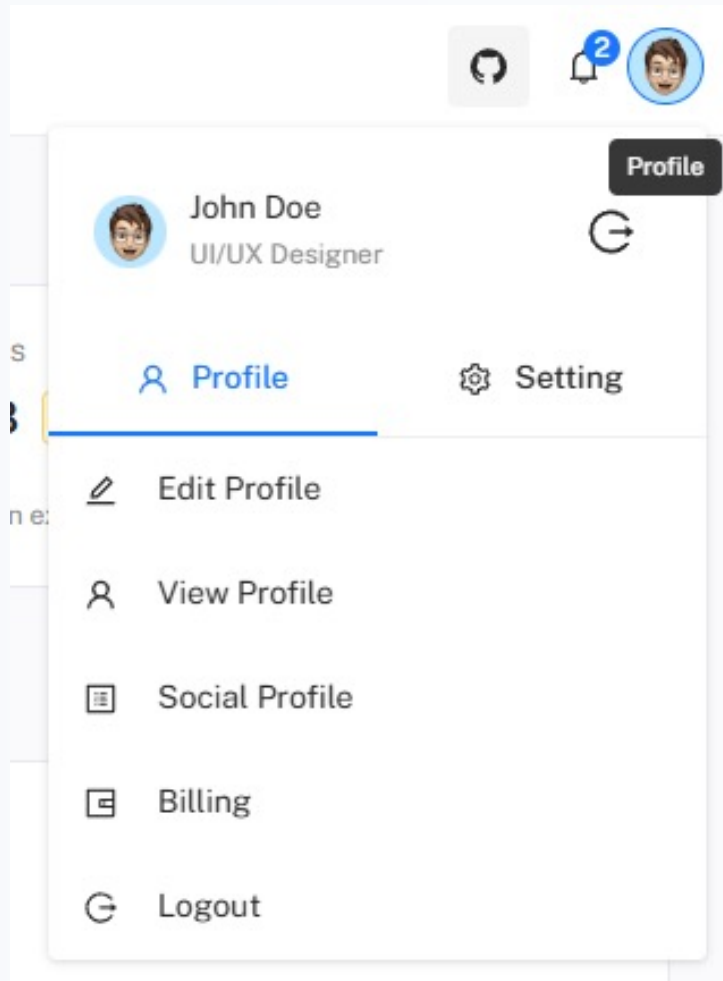
☐ Keep me sign in

Forgot Password?

Login







Module 8

Test Data and UI State Management

Test data strategy

- A **test data strategy** defines how test data is created, managed, reused, secured, and cleaned up during automated or manual testing.
- In UI automation using Playwright with C# and xUnit, a good test data strategy helps ensure:
 - Tests are repeatable
 - Tests are isolated
 - Tests are reliable
 - Tests do not fail randomly because of shared data
 - Parallel execution works correctly

Common Test Data Strategies

1. Hardcoded Test Data (Bad Practice)

```
await page.GetByPlaceholder("Email")  
    .FillAsync("john@test.com");
```

- Problems:
 - Duplicate conflicts
 - Cannot run in parallel
 - Difficult maintenance
 - State pollution

Common Test Data Strategies

3. Test Data Factory Pattern

- Centralized reusable test data generation.

```
public static class TestDataFactory
{
    public static string CreateUniqueEmail()
    {
        return $"user_{Guid.NewGuid():N}@test.com";
    }
}
```

- Usage:
 string email = TestDataFactory.CreateUniqueEmail();
- Benefits:
 - Reusable
 - Maintainable
 - Consistent

Common Test Data Strategies

4. API-Based Test Data Setup (Best Practice)

- Instead of creating data manually via UI:
 - Use API to create data
 - Use UI only for validation
- Benefits:
 - Faster tests
 - Less flaky
 - Better isolation

Common Test Data Strategies

5. Seeded Database Strategy

- Prepare known test records before execution.
- Good for:
 - Login tests
 - Authorization tests
 - Reporting tests
- Bad for:
 - Parallel execution
 - Mutation-heavy tests

Positive and negative test cases

1. Positive Test Case - checks if the application works correctly when the user enters **valid data** or follows the correct workflow.

```
[Fact]
public async Task Should_Login_With_Valid_Credentials()
{
    await page.GotoAsync("http://localhost:3000/login");
    await page.GetByLabel("Email").FillAsync("admin@test.com");
    await page.GetByLabel("Password").FillAsync("password123");
    await page.GetByRole(AriaRole.Button, new() { Name = "Login" })
        .ClickAsync();
    await Assertions.Expect(
        page.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })
    ).ToBeVisibleAsync();
}
```



Positive and negative test cases

2. Negative Test Case - checks if the application handles invalid input.

```
[Fact]
public async Task Should_Show_Error_For_Invalid_Login()
{
    await page.GotoAsync("http://localhost:3000/login");
    await page.GetByLabel("Email").FillAsync("wrong@test.com");
    await page.GetByLabel("Password").FillAsync("wrongpassword");
    await page.GetByRole(AriaRole.Button, new() { Name = "Login" })
        .ClickAsync();

    await Assertions.Expect(
        page.GetText("Invalid email or password")
    ).ToBeVisibleAsync();
}
```



Common Examples

Feature	Positive Test Case	Negative Test Case
Login	Valid user can log in	Wrong password shows error
Register	Valid account can be created	Empty email shows validation
Search	Existing record is found	Non-existing record shows “No results”
Form	Valid form submits successfully	Missing required field blocks submit
Upload	Valid file uploads	Invalid file type is rejected
Table	Pagination moves to next page	Disabled next button cannot be clicked

Browser storage and cookies

Storage type	Purpose	Scope
Cookies	Login/session tracking	Browser context/domain
Local Storage	Persistent browser data	Origin/site
Session Storage	Temporary tab data	Page/tab session

Cookies Example

```
var cookies = await context.CookiesAsync();
foreach (var cookie in cookies)
{
    Console.WriteLine($"{cookie.Name} = {cookie.Value}");
}
// Add cookie:
await context.AddCookiesAsync(new[]
{
    new Cookie
    {
        Name = "training_mode",
        Value = "enabled",
        Domain = "localhost",
        Path = "/"
    }
});
// Clear cookies:
await context.ClearCookiesAsync();
```



Local Storage Example

```
await page.EvaluateAsync("""
    localStorage.setItem('theme', 'dark');
""");

var theme = await page.EvaluateAsync<string>("localStorage.getItem('theme')");

Assert.Equal("dark", theme);

// Clear local storage:
await page.EvaluateAsync("localStorage.clear()");
```

Session Storage Example

```
await page.EvaluateAsync("""
    sessionStorage.setItem('currentStep', 'checkout');
""");

var step = await page.EvaluateAsync<string>(
    "sessionStorage.getItem('currentStep')"
);

Assert.Equal("checkout", step);
```

Why Authentication State Reuse

- Without authentication reuse
 - Every test does this
 - Open browser
 - Navigate to login page
 - Enter username/password
 - Click login
 - Wait for dashboard
 - This becomes
 - Slow
 - Repetitive
 - Brittle
 - Harder to maintain
 - With authentication reuse
 - Login once
 - Save browser storage/cookies
 - Reuse the authenticated session

Playwright Authentication Flow

Step 1 - Login Once

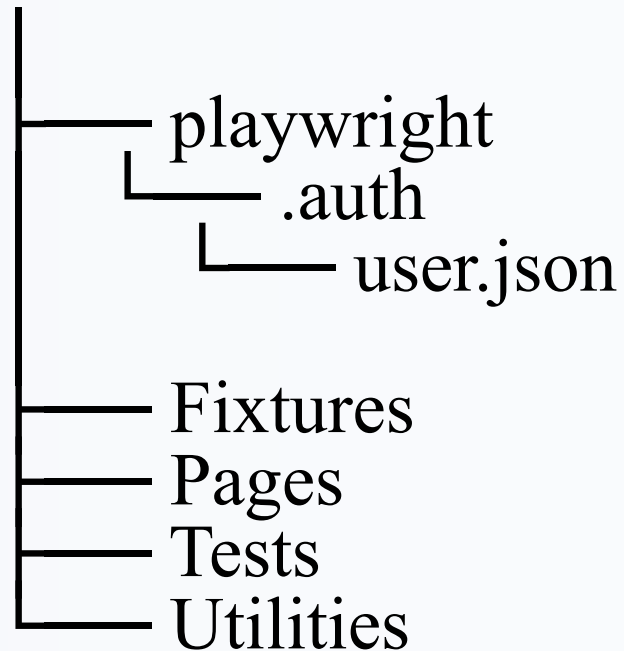
➤ Saves:

- Cookies
- JWT tokens
- Local storage

Step 2 - Reuse Authentication

Playwright Authentication Flow

- **Recommended Folder Structure**



When NOT to Reuse Authentication

- Login functionality itself
- Session expiration
- MFA
- Logout flows
- Security testing
- Permission boundary testing

Activity 8 - Test Data and UI State Management

- This activity teaches how professional UI automation frameworks handle:
- reusable test data
- positive and negative testing
- browser/session state
- authentication reuse
- scalable automated testing

Activity 8 - Test Data and UI State Management

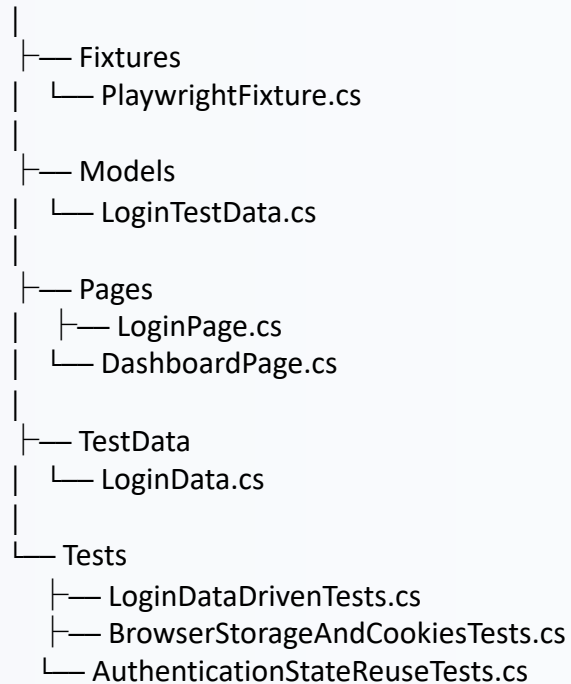
1. Prepare your project

```
mkdir Activity8  
cd Activity8  
dotnet new sln -n Activity8  
dotnet new xunit -n Activity8.TestDataUiState.Tests  
dotnet sln Activity8.slnx add .\Activity8.TestDataUiState.Tests\Activity8.TestDataUiState.Tests.csproj  
cd .\Activity8.TestDataUiState.Tests  
dotnet add package Microsoft.Playwright --version 1.55.0  
dotnet build  
pwsh bin\Debug\net10.0\playwright.ps1 install
```

Activity 8 - Test Data and UI State Management

2. Project structure

Activity8.TestDataUiState.Tests



Module 9

End-to-End UI Test Scenarios

Designing realistic E2E UI workflows

- A **realistic E2E workflow** tests a complete user journey, not just one button or one page.
- Example:
 - Login
 - Open dashboard
 - Search/filter table
 - Open record
 - Edit data
 - Save
 - Verify success message
 - Logout

Good E2E workflow design

1. Start from real user scenarios

➤ Bad:

- Click button
- Check dialog
- Close dialog

➤ Better:

- Admin logs in
- Admin opens user management
- Admin searches a user
- Admin edits user status
- Admin confirms update
- System shows updated status

Good E2E workflow design

2. Keep workflow meaningful, not too large

- Avoid testing the entire application in one test.

- Good test size:

 - Login → Create item → Verify item appears

- Too large:

 - Login → Create item → Edit item → Delete item → Export report → Logout

Good E2E workflow design

3. Use Page Object Model for readability

```
[Fact]
public async Task Admin_Should_Search_Filter_And_View_Order_Details()
{
    await LoginPage.GotoAsync();
    await LoginPage.LoginAsync("admin@test.com", "password");

    await DashboardPage.ExpectLoadedAsync();

    await OrdersPage.GotoAsync();
    await OrdersPage.SearchOrderAsync("ORD-1001");
    await OrdersPage.OpenOrderDetailsAsync("ORD-1001");

    await OrderDetailsPage.ExpectOrderVisibleAsync("ORD-1001");
}
```



Good E2E workflow design

4. Assert business result, not only UI movement

- Weak assertion:

```
await Assertions.Expect(page.GetByRole(AriaRole.Dialog))  
    .ToBeVisibleAsync();
```

- Better assertion:

```
await Assertions.Expect(page.GetText("Order ORD-1001 was updated  
successfully")).ToBeVisibleAsync();
```

Good E2E workflow design

5. Use test data intentionally

- For realistic E2E tests, prepare data first.
- Common strategy:
 1. Arrange: create test data
 2. Act: perform UI workflow
 3. Assert: verify result
 4. Cleanup: remove/reset test data

Realistic workflow for Mantis Dashboard

1. Open login page
2. Login as valid user
3. Verify dashboard loaded
4. Open customer/order table
5. Search specific record
6. Apply filter
7. Open details dialog/page
8. Verify correct record information
9. Close dialog/page
10. Logout

Combining multiple pages

- **Combining multiple pages** means your test controls more than one browser tab/window in the same scenario.
- Example use cases:
 - Login page opens dashboard in a new tab
 - Clicking “View Report” opens a new browser page
 - Payment gateway opens in another window
 - Admin page and user page are tested together
 - Email preview page opens separately

Basic Example: Handle New Page

```
[Fact]
public async Task Should_Handle_New_Page()
{
    using IPlaywright playwright = await Playwright.CreateAsync();
    await using IBrowser browser = await playwright.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions { Headless = false });
    IBrowserContext context = await browser.NewContextAsync();

    IPage mainPage = await context.NewPageAsync();
    await mainPage.GotoAsync("http://localhost:3000");

    IPage newPage = await context.RunAndWaitForPageAsync(async () =>
    {
        await mainPage.GetByRole(
            AriaRole.Link,
            new() { Name = "Open Dashboard" }
        ).ClickAsync();
    });
    await newPage.WaitForLoadStateAsync();
    await Assertions.Expect(newPage.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })).ToBeVisibleAsync();
    await mainPage.GetByRole(AriaRole.Button, new() { Name = "Logout" }).ClickAsync();
}
```

Example: Two Pages in One Test

```
[Fact]
public async Task Should_Use_Two_Pages_In_One_Context()
{
    using IPlaywright playwright = await Playwright.CreateAsync();
    await using IBrowser browser = await playwright.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions
        {
            Headless = false
        });

    IBrowserContext context = await browser.NewContextAsync();

    IPage adminPage = await context.NewPageAsync();
    IPage userPage = await context.NewPageAsync();

    await adminPage.GotoAsync("http://localhost:3000/admin");
    await userPage.GotoAsync("http://localhost:3000");

    await Assertions.Expect(adminPage).ToHaveURLAsync(new Regex("admin"));
    await Assertions.Expect(userPage).ToHaveURLAsync(new Regex("localhost:3000"));
}
```

Verifying business flows from UI

- Testing the **real user journey**, not just one button or one page.
- Example business flow:
 - Login
 - open dashboard
 - search customer/order/product
 - create/update record
 - verify success message
 - verify data appears in table
 - logout

Good UI business flow test structure

- Arrange → prepare test data / open app
- Act → perform real user steps
- Assert → verify business result
- Cleanup → logout / remove created data

Example: full business flow test

```
// 1. Open application
await page.GotoAsync("http://xyz.com");

// 2. Login
await page.GetByRole(AriaRole.Button, new() { Name = "Login" }).ClickAsync();

// 3. Verify dashboard
await Assertions.Expect(page.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })).ToBeVisibleAsync();

// 4. Open business page
await page.GetByRole(AriaRole.Link, new() { Name = "Products" }).ClickAsync();

// 5. Create new product
await page.GetByRole(AriaRole.Button, new() { Name = "Add Product" }).ClickAsync();

// 6. Verify success message
await Assertions.Expect(page.GetByText("Product created successfully")).ToBeVisibleAsync();

// 7. Verify product appears in table
await Assertions.Expect(page.GetByRole(AriaRole.Row, new() { Name = new Regex("Test Product 001")
})).ToBeVisibleAsync();

// 8. Logout
await page.GetByRole(AriaRole.MenuItem, new() { Name = "Logout" }).ClickAsync();

// 9. Verify back to login page
await Assertions.Expect(page.GetByRole(AriaRole.Button, new() { Name = "Login" })).ToBeVisibleAsync();
```

Activity 9 - Enterprise End-to-End UI Workflow Automation

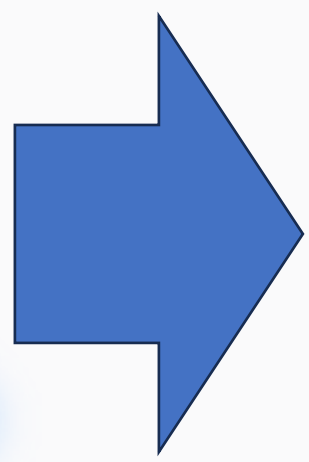
1. Enterprise End-to-End UI Workflow Automation
2. Combine Multiple Pages in One Automated Flow
3. Implement Enterprise-Level Page Object Model (POM)
4. Build Stable and Maintainable Playwright Tests
5. Separate UI Components from Pages
6. Implement Business Workflow Abstraction
7. Use Models for Test Data Management
8. Verify Business Flows Through UI Assertions

Project Structure

Activity9.RealisticE2EWorkflowAutomation.Tests

- |
 - |— Pages
 - |— Authentication
 - |— MantisLoginPage.cs
 - |— Profile
 - |— MantisProfilePage.cs
 - |— Forms
 - |— MantisFormsWizardPage.cs
 - |— Components
 - |— SidebarMenuComponent.cs
 - |— HeaderMenuComponent.cs
 - |— PromoPopupComponent.cs
 - |— Workflows
 - |— UserBusinessWorkflow.cs
 - |— Models
 - |— UserProfileModel.cs
 - |— ShippingAddressModel.cs
 - |— Fixtures
 - |— PlaywrightFixture.cs
 - |— Tests
 - |— MantisRealisticE2EWorkflowTests.cs
 - |— Utilities
 - |— TestDataFactory.cs
 - |— WaitHelpers.cs





Login

[Don't have an account?](#)

Email Address

info@codedthemes.com

Password

.....

☐ Keep me sign in

[Forgot Password?](#)

Login

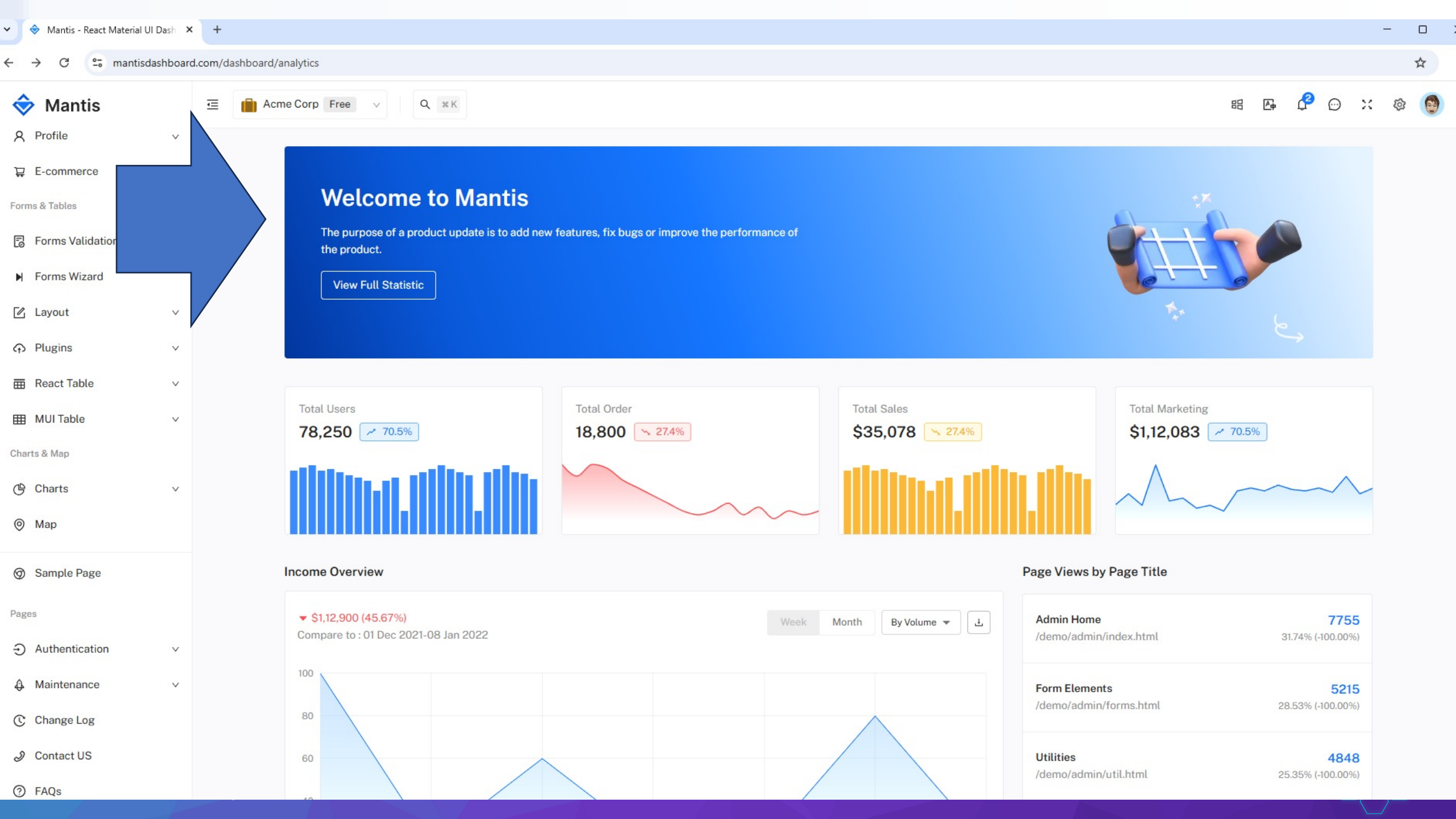
Check other login views

 Firebase

 Auth0

 Aws

 Supabase



-  **Mantis**
-  Profile

 E-commerce

Forms & Tables

 Forms Validation

 Forms Wizard

 Layout

 Plugins

 React Table

 MUI Table

Charts & Map

 Charts

 Map

 Sample Page

Pages

 Authentication

 Maintenance

 Change Log

 Contact US

 FAQs

 Acme Corp

Free



2











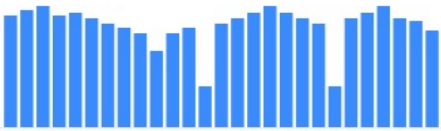
Welcome to Mantis

The purpose of a product update is to add new features, fix bugs or improve the performance of the product.

View Full Statistic

Total Users

78,250 +70.5%



Total Order

18,800 -27.4%



Total Sales

\$35,078 -27.4%



Total Marketing

\$1,12,083 +70.5%



Income Overview

▼ \$1,12,900 (45.67%)

Compare to : 01 Dec 2021-08 Jan 2022

Week

Month

By Volume





Page Views by Page Title

Admin Home /demo/admin/index.html	7755 31.74% (-100.00%)
Form Elements /demo/admin/forms.html	5215 28.53% (-100.00%)
Utilities /demo/admin/util.html	4848 25.35% (-100.00%)

86

40

4.5K

Personal Information

First Name

Stebin

Last Name

Ben

Email Address

stebin.ben@gmail.com

Date of Birth (+18)

March

10

1993

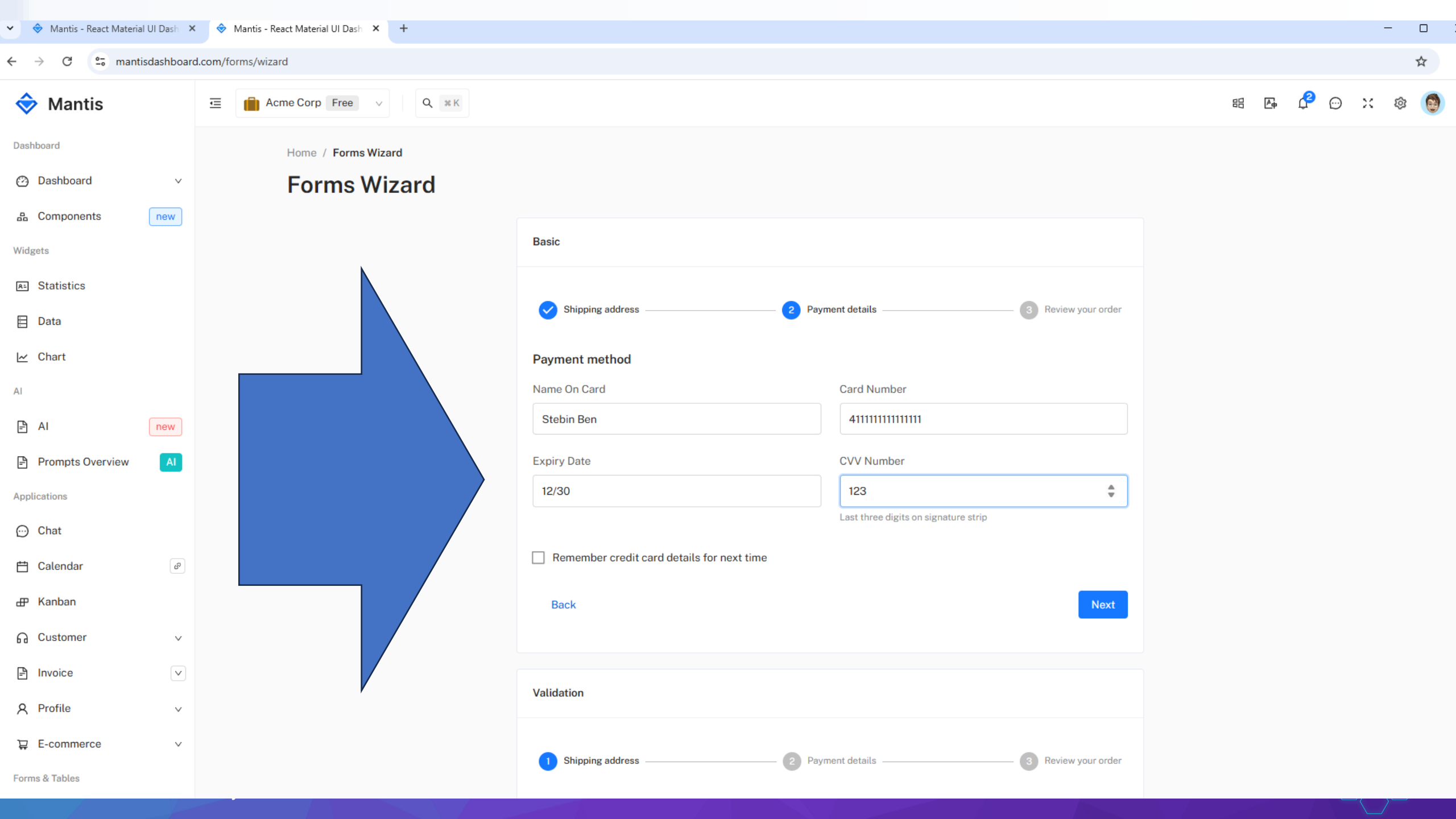
Phone Number

+91

9652364852

Designation

Full Stack Developer



Dashboard

Dashboard

Components

new

Widgets

Statistics

Data

Chart

AI

AI

new

Prompts Overview

AI

Applications

Chat

Calendar

🗳

Kanban

Customer

Invoice

▼

Profile

▼

E-commerce

▼

Forms & Tables

Home / Forms Wizard

Forms Wizard

Basic



Shipping address



Payment details



Review your order

Payment method

Name On Card

Stebin Ben

Card Number

4111111111111111

Expiry Date

12/30

CVV Number

123

Last three digits on signature strip



Remember credit card details for next time

Back

Next

Validation



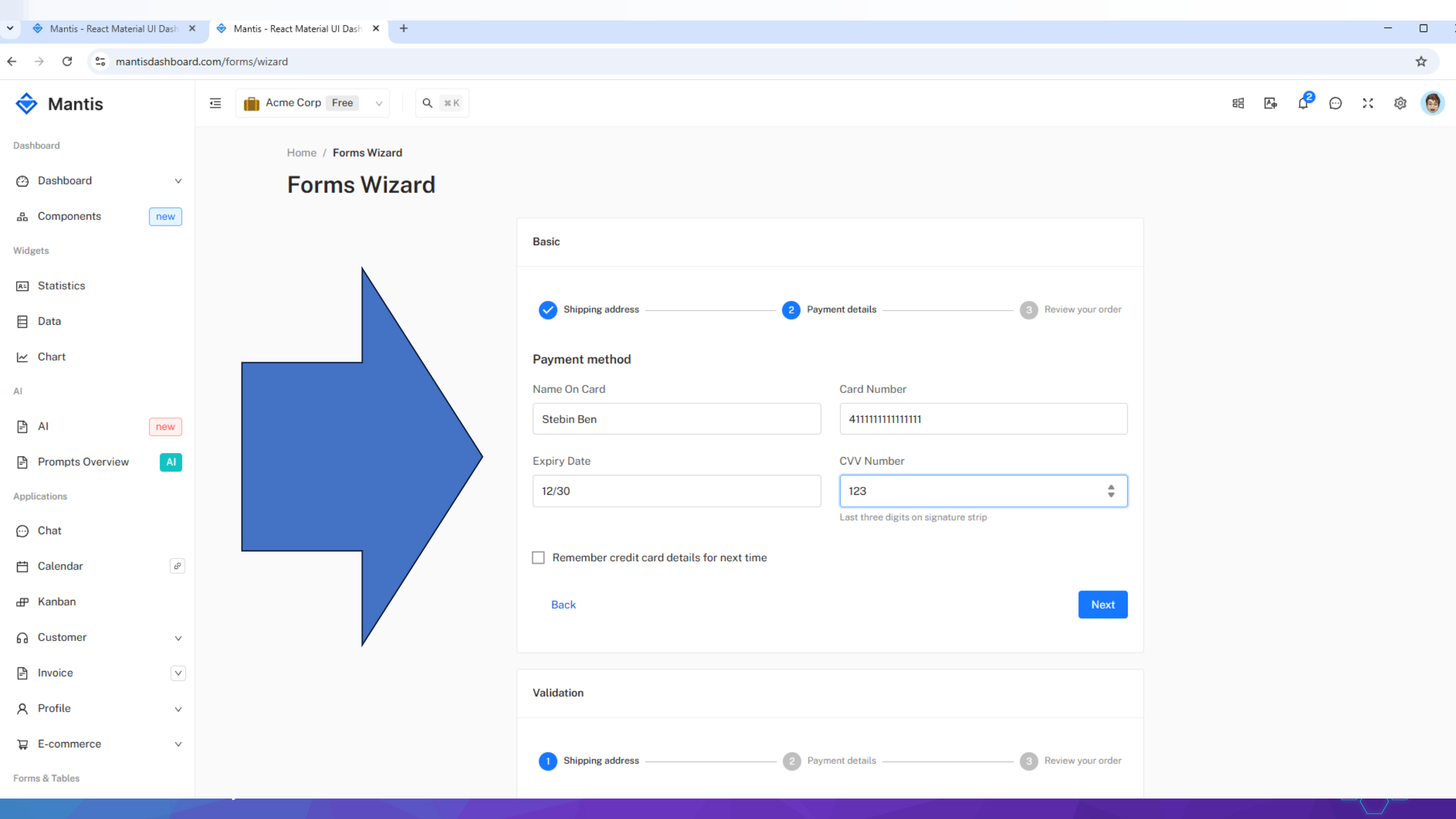
Shipping address



Payment details



Review your order




```

PS D:\PlaywrightActivities\Activity9\Activity9.RealisticE2EWorkflowAutomation.Tests> dotnet test
Restore complete (0.3s)
  Activity9.RealisticE2EWorkflowAutomation.Tests net10.0 succeeded (0.3s) → bin\Debug\net10.0\Activity9.RealisticE2EWorkflowAutomation.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.06]   Discovering: Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:00.09]   Discovered:   Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:00.10]   Starting:     Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:15.35]   Finished:     Activity9.RealisticE2EWorkflowAutomation.Tests
  Activity9.RealisticE2EWorkflowAutomation.Tests test net10.0 succeeded (16.0s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 15.9s
Build succeeded in 17.1s
PS D:\PlaywrightActivities\Activity9\Activity9.RealisticE2EWorkflowAutomation.Tests> dotnet test
Restore complete (0.4s)
  Activity9.RealisticE2EWorkflowAutomation.Tests net10.0 succeeded (0.3s) → bin\Debug\net10.0\Activity9.RealisticE2EWorkflowAutomation.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.07]   Discovering: Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:00.09]   Discovered:   Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:00.11]   Starting:     Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:43.68]   Finished:     Activity9.RealisticE2EWorkflowAutomation.Tests
  Activity9.RealisticE2EWorkflowAutomation.Tests test net10.0 succeeded (44.3s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 44.3s
Build succeeded in 45.5s
PS D:\PlaywrightActivities\Activity9\Activity9.RealisticE2EWorkflowAutomation.Tests> dotnet test
Restore complete (0.3s)
  Activity9.RealisticE2EWorkflowAutomation.Tests net10.0 succeeded (0.3s) → bin\Debug\net10.0\Activity9.RealisticE2EWorkflowAutomation.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.05]   Discovering: Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:00.08]   Discovered:   Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:00.10]   Starting:     Activity9.RealisticE2EWorkflowAutomation.Tests
[xUnit.net 00:00:49.50]   Finished:     Activity9.RealisticE2EWorkflowAutomation.Tests
  Activity9.RealisticE2EWorkflowAutomation.Tests test net10.0 succeeded (50.1s)

Test summary: total: 2, failed: 0, succeeded: 2, skipped: 0, duration: 50.1s
Build succeeded in 51.1s
PS D:\PlaywrightActivities\Activity9\Activity9.RealisticE2EWorkflowAutomation.Tests> |

```



Module 10

Debugging, Tracing, Screenshots, and Reporting

Debugging Playwright tests

- Run with visible browser

```
await using var browser = await playwright.Chromium.LaunchAsync(  
    new BrowserTypeLaunchOptions  
    {  
        Headless = false,  
        SlowMo = 300  
    });
```

- Pause during test

```
await page.PauseAsync();
```

Debugging Playwright tests

- Take screenshot when debugging

```
await page.ScreenshotAsync(new()  
{  
    Path = "debug-screenshot.png",  
    FullPage = true  
});
```
- Print current URL

```
Console.WriteLine(page.Url);
```

Debugging Playwright tests

- Print visible text

```
Console.WriteLine(await page.Locator("body").InnerTextAsync());
```

- Check locator count

```
var count = await page.GetByText("Total Order").CountAsync();  
Console.WriteLine($"Matched elements: {count}");
```

Debugging Playwright tests

- Debug strict mode issue

```
await Assertions.Expect(  
    page.GetByRole(AriaRole.Heading, new() { Name = "Total Order" })  
).ToBeVisibleAsync();
```

- Instead of

```
await Assertions.Expect(  
    page.GetByText("Total Order")  
).ToBeVisibleAsync();
```

Headed Execution

- The browser runs **without showing the GUI/window**.
- Example:
 - await using var browser = await playwright.Chromium.LaunchAsync(
 new BrowserTypeLaunchOptions
 {
 Headless = true
 });

Headed Advantages

Benefit	Explanation
Faster	No rendering overhead
Lightweight	Uses less CPU/RAM
Better for CI	Servers often have no GUI
Scalable	Easier parallel execution

Headed Disadvantages

Limitation	Explanation
Harder debugging	Cannot visually watch the test
UI animations harder to inspect	Visual behavior hidden
Beginner-unfriendly	You cannot “see” what failed

Headed Execution

- The browser runs with a **visible UI window**.

- Example:

```
await using var browser = await playwright.Chromium.LaunchAsync(  
    new BrowserTypeLaunchOptions  
    {  
        Headless = false  
    });
```

Headed Advantages

Benefit	Explanation
Easier debugging	You see every step
Better learning experience	Visual understanding
Great for demos	Stakeholders can watch
Easier locator troubleshooting	Observe UI behavior

Headed Disadvantages

Limitation	Explanation
Slower	UI rendering costs resources
More RAM/CPU	Visible browser consumes more
Less scalable	Many visible browsers are heavy

During Development

- Use:
 - Headless = false
- Why?
 - Easier debugging
 - Watch locators
 - Observe timing issues
 - Inspect dialogs/popups

During CI/CD

- Use:
 - Headless = true
- Why?
 - Faster
 - Stable in servers
 - Efficient parallel execution

Screenshots on failure

- **Screenshot on failure** means: when a test fails, automatically save a screenshot so you can inspect the UI state at the failure point.
- For **xUnit**, use try/catch inside the test

```
try
{
    await page.GotoAsync("https://mantisdashboard.com/login");
    await page.GetByPlaceholder("Email Address").FillAsync("wrong@email.com");
    await page.GetByPlaceholder("Password").FillAsync("wrongpassword");
    await page.GetByRole(AriaRole.Button, new() { Name = "Login" }).ClickAsync();
    await Assertions.Expect(
        page.GetByText("Dashboard")
    ).ToBeVisibleAsync();
}
catch
{
    await page.ScreenshotAsync(new PageScreenshotOptions
    {
        Path = "screenshots/login-failure.png",
        FullPage = true
    });
    throw;
}
```


Video recording

- Video recording allows you to automatically capture browser sessions during UI test execution.
- This is very useful for:
 - Debugging failed tests
 - Reviewing test execution steps
 - CI/CD pipeline artifacts
 - Training/demo purposes
 - Evidence for defects

How Video Recording Works

- Playwright records the browser context session, not individual pages.
- The recording starts when the browser context is created and is saved when the context is closed.

How to record the video

```
await using var context = await browser.NewContextAsync(  
    new BrowserNewContextOptions  
    {  
        RecordVideoDir = "videos/",  
        RecordVideoSize = new RecordVideoSize  
        {  
            Width = 1280,  
            Height = 720  
        }  
    });
```

Trace Viewer

- **Trace Viewer** is used to inspect a test execution step by step.
- It records:
 - Actions
 - Screenshots
 - DOM snapshots
 - network requests
 - console logs
 - Errors
 - Timing

To Enable the Trace Viewer

```
await context.Tracing.StartAsync(new()  
{  
    Screenshots = true,  
    Snapshots = true,  
    Sources = true  
});  
  
// test actions here  
  
await context.Tracing.StopAsync(new()  
{  
    Path = "trace.zip"  
});
```

To Open the Trace Viewer

```
pwsh bin/Debug/net10.0/playwright.ps1 show-trace trace.zip
```

Handling flaky tests

- A **flaky test** is a test that sometimes passes and sometimes fails without code changes.
 - Weak selectors
 - Missing waits
 - Hard-coded delays
 - Shared test data
 - Unstable UI state
 - Network/API timing
 - Tests depending on previous tests

Bad flaky example

Problem: `WaitForTimeoutAsync()` only waits for time, not for the UI to be ready.

```
[Fact]
public async Task Bad_Flaky_Test()
{
    await _page.GotoAsync("https://mantisdashboard.com/login");

    await _page.WaitForTimeoutAsync(3000); // bad

    await _page.GetByText("Login").ClickAsync();
}
```


Better version

```
[Fact]
public async Task Should_Login_Reliably()
{
    await _page.GotoAsync("https://mantisdashboard.com/login");
    await Assertions.Expect(
        _page.GetByRole(AriaRole.Heading, new() { Name = "Login" })
    ).ToBeVisibleAsync();
    await _page.GetByPlaceholder("Enter email address")
        .FillAsync("a@a.com");
    await _page.GetByPlaceholder("Enter password")
        .FillAsync("password");
    await _page.GetByRole(AriaRole.Button, new() { Name = "Login" })
        .ClickAsync();
    await Assertions.Expect(
        _page.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })
    ).ToBeVisibleAsync();
}
```

Common fixes

1. Use stable locators

- Prefer

```
_page.GetByRole(AriaRole.Button, new() { Name = "Submit" })
```

- or

```
_page.GetByTestId("submit-button")
```

- Avoid brittle selectors like

```
_page.Locator("div:nth-child(3) > button")
```

Common fixes

2. Replace hard waits with assertions

- Bad:

```
await _page.WaitForTimeoutAsync(5000);
```

- Good:

```
await Assertions.Expect(  
    _page.GetText("Order saved successfully")  
).ToBeVisibleAsync();
```

Common fixes

4. Avoid shared test dependency

- Bad:

```
[Fact]
public async Task Step1_Create_Order() { }
```

```
[Fact]
public async Task Step2_Edit_Order() { }
```

- Better: each test prepares its own data.

```
[Fact]
public async Task Should_Edit_Order()
{
    var order = TestDataFactory.ValidOrder();

    await CreateOrderAsync(order);
    await EditOrderAsync(order);
}
```

Common fixes

5. Capture trace, screenshot, and video on failure

- Use these when investigating flaky tests:

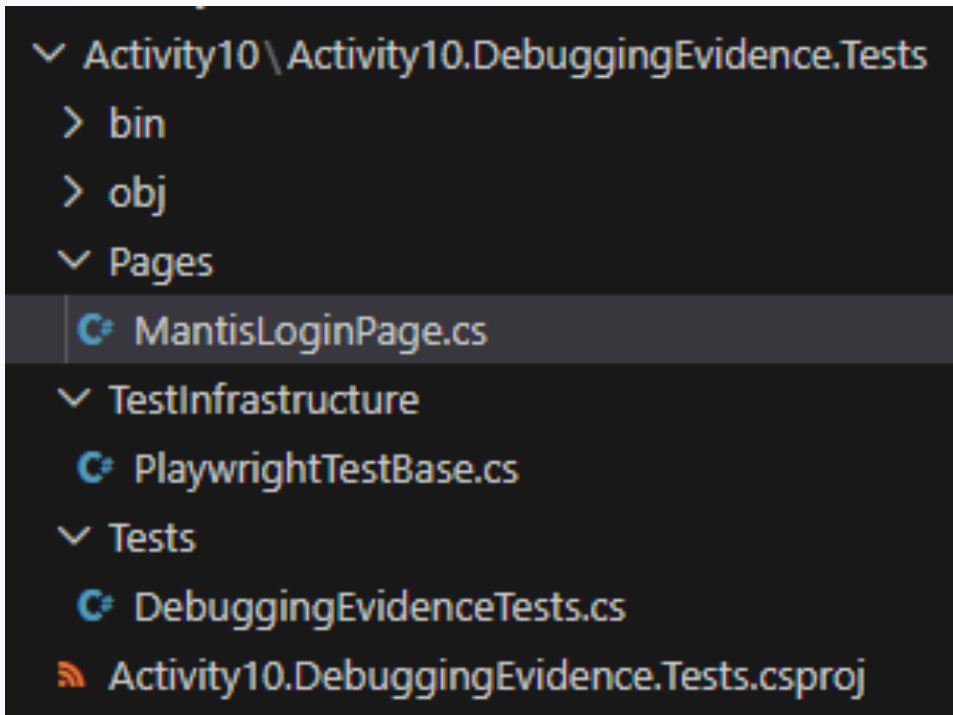
```
await context.Tracing.StartAsync(new()  
{  
    Screenshots = true,  
    Snapshots = true,  
    Sources = true  
});  
await context.Tracing.StopAsync(new()  
{  
    Path = "traces/flaky-test.zip"  
});
```

Activity 10: Debugging and Test Evidence

- This activity will understand better to
 - Debugging Playwright tests
 - Headed vs headless execution
 - Screenshots on failure
 - Video recording
 - Trace viewer
 - Handling flaky tests

Activity 10: Debugging and Test Evidence

Project structure



MantisLoginPage.cs

```
using Microsoft.Playwright;
namespace Activity10.DebuggingEvidence.Tests.Pages;
public sealed class MantisLoginPage
{
    private readonly IPage _page;
    public MantisLoginPage(IPage page)
    {
        _page = page;
    }
    public async Task GotoAsync()
    {
        await _page.GotoAsync(
            "http://localhost:3000",
            new PageGotoOptions
            {
                WaitUntil = WaitUntilState.DOMContentLoaded
            });
        await _page.GetByText("Already have an account?").ClickAsync();
    }
}
```



MantisLoginPage.cs

```
public async Task LoginAsync(string email, string password)
{
    await _page.GetByPlaceholder("Enter email address").FillAsync(email);
    await _page.GetByPlaceholder("Enter password").FillAsync(password);

    await _page.GetByRole(
        AriaRole.Button,
        new() { Name = "Login" }
    ).ClickAsync();
}

public async Task AssertDashboardLoadedAsync()
{
    await Assertions.Expect(
        _page.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })
    ).ToBeVisibleAsync();
}
}
```



PlaywrightTestBase.cs

```
using Microsoft.Playwright;

namespace Activity10.DebuggingEvidence.Tests.TestInfrastructure;

public abstract class PlaywrightTestBase : IAsyncLifetime
{
    protected IPlaywright Playwright = null!;
    protected IBrowser Browser = null!;
    protected IBrowserContext Context = null!;
    protected IPage Page = null!;

    protected virtual bool Headless => true;
```

PlaywrightTestBase.cs

```
public async Task InitializeAsync()
{
    Playwright = await Microsoft.Playwright.Playwright.CreateAsync();

    Browser = await Playwright.Chromium.LaunchAsync(
        new BrowserTypeLaunchOptions
        {
            Headless = Headless,
            SlowMo = Headless ? 0 : 300
        });
}
```

PlaywrightTestBase.cs

```
Context = await Browser.NewContextAsync(  
    new BrowserNewContextOptions  
    {  
        ViewportSize = new ViewportSize  
        {  
            Width = 1366,  
            Height = 768  
        },  
        RecordVideoDir = "videos/",  
        RecordVideoSize = new RecordVideoSize  
        {  
            Width = 1280,  
            Height = 720  
        }  
    }  
));
```

PlaywrightTestBase.cs

```
    await Context.Tracing.StartAsync(  
        new TracingStartOptions  
        {  
            Screenshots = true,  
            Snapshots = true,  
            Sources = true  
        });  
  
    Page = await Context.NewPageAsync();  
}
```

PlaywrightTestBase.cs

```
public async Task DisposeAsync()
{
    Directory.CreateDirectory("traces");

    await Context.Tracing.StopAsync(
        new TracingStopOptions
        {
            Path = $"traces/trace-{DateTime.Now:yyyyMMdd-HH:mm:ss}.zip"
        });

    await Context.CloseAsync();
    await Browser.CloseAsync();

    Playwright.Dispose();
}
```

PlaywrightTestBase.cs

```
protected async Task CaptureScreenshotAsync(string testName)
{
    Directory.CreateDirectory("screenshots");

    await Page.ScreenshotAsync(
        new PageScreenshotOptions
        {
            Path = $"screenshots/{testName}-{DateTime.Now:yyyyMMdd-HH:mm:ss}.png",
            FullPage = true
        });
}
```

DebuggingEvidenceTests.cs

```
using Activity10.DebuggingEvidence.Tests.Pages;
using Activity10.DebuggingEvidence.Tests.TestInfrastructure;
using Microsoft.Playwright;

namespace Activity10.DebuggingEvidence.Tests.Tests;

public sealed class DebuggingEvidenceTests : PlaywrightTestBase
{
    [Fact]
    public async Task Should_Login_And_Capture_Debugging_Evidence()
    {
        var loginPage = new MantisLoginPage(Page);
```


DebuggingEvidenceTests.cs

```
try
{
    await loginPage.GotoAsync();
    await loginPage.LoginAsync(
        "a@a.com",
        "password"
    );
    await loginPage.AssertDashboardLoadedAsync();
    await
CaptureScreenshotAsync(nameof(Should_Login_And_Capture_Debugging_Evidence));
}
catch
{
    await
CaptureScreenshotAsync($"{{nameof(Should_Login_And_Capture_Debugging_Evidence)}}-
FAILED");
    throw;
}
}
```

DebuggingEvidenceTests.cs

```
[Fact]
public async Task
Should_Handle_Flaky_Loading_Using_Expect_And_Retry_Style_Assertion()
{
    await Page.GotoAsync(
        "http://localhost:3000",
        new PageGotoOptions
        {
            WaitUntil = WaitUntilState.DOMContentLoaded
        });

    await Page.GetByText("Already have an account?").ClickAsync();

    await Page.GetByPlaceholder("Enter email address").FillAsync("a@a.com");
    await Page.GetByPlaceholder("Enter password").FillAsync("password");
}
```

DebuggingEvidenceTests.cs

```
await Page.GetByRole(  
    AriaRole.Button,  
    new() { Name = "Login" }  
).ClickAsync();  
  
await Assertions.Expect(  
    Page.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })  
).ToBeVisibleAsync(new LocatorAssertionsToBeVisibleOptions  
{  
    Timeout = 10000  
});  
  
await Assertions.Expect(  
    Page.GetByText("Total Order", new() { Exact = true }).First  
).ToBeVisibleAsync(new LocatorAssertionsToBeVisibleOptions  
{  
    Timeout = 10000  
});  
}
```

DebuggingEvidenceTests.cs

```
[Fact]
public async Task Should_Demonstrate_Debug_Pause_When_Running_Headed()
{
    await Page.GotoAsync("http://localhost:3000");
    await Page.GetByText("Already have an account?").ClickAsync();
    await Page.GetByPlaceholder("Enter email address").FillAsync("a@a.com");
    await Page.GetByPlaceholder("Enter password").FillAsync("password");
    await Page.GetByRole(
        AriaRole.Button,
        new() { Name = "Login" }
    ).ClickAsync();

    await Assertions.Expect(
        Page.GetByRole(AriaRole.Heading, new() { Name = "Dashboard" })
    ).ToBeVisibleAsync();
}
}
```

dotnet test

```
PowerShell
PS D:\PlaywrightActivities\Activity10\Activity10.DebuggingEvidence.Tests> dotnet test
Restore complete (0.3s)
  Activity10.DebuggingEvidence.Tests net10.0 succeeded (0.4s) → bin\Debug\net10.0\Activity10.DebuggingEvidence.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.06]   Discovering: Activity10.DebuggingEvidence.Tests
[xUnit.net 00:00:00.09]   Discovered:   Activity10.DebuggingEvidence.Tests
[xUnit.net 00:00:00.11]   Starting:    Activity10.DebuggingEvidence.Tests
[xUnit.net 00:00:09.21]   Finished:   Activity10.DebuggingEvidence.Tests
  Activity10.DebuggingEvidence.Tests test net10.0 succeeded (9.9s)

Test summary: total: 3, failed: 0, succeeded: 3, skipped: 0, duration: 9.9s
Build succeeded in 11.0s
```

Videos .\Activity10\Activity10.DebuggingEvidence.Tests\bin\Debug\net10.0\videos



5dffeac4210f7de9
7b84384825e6176
e.webm

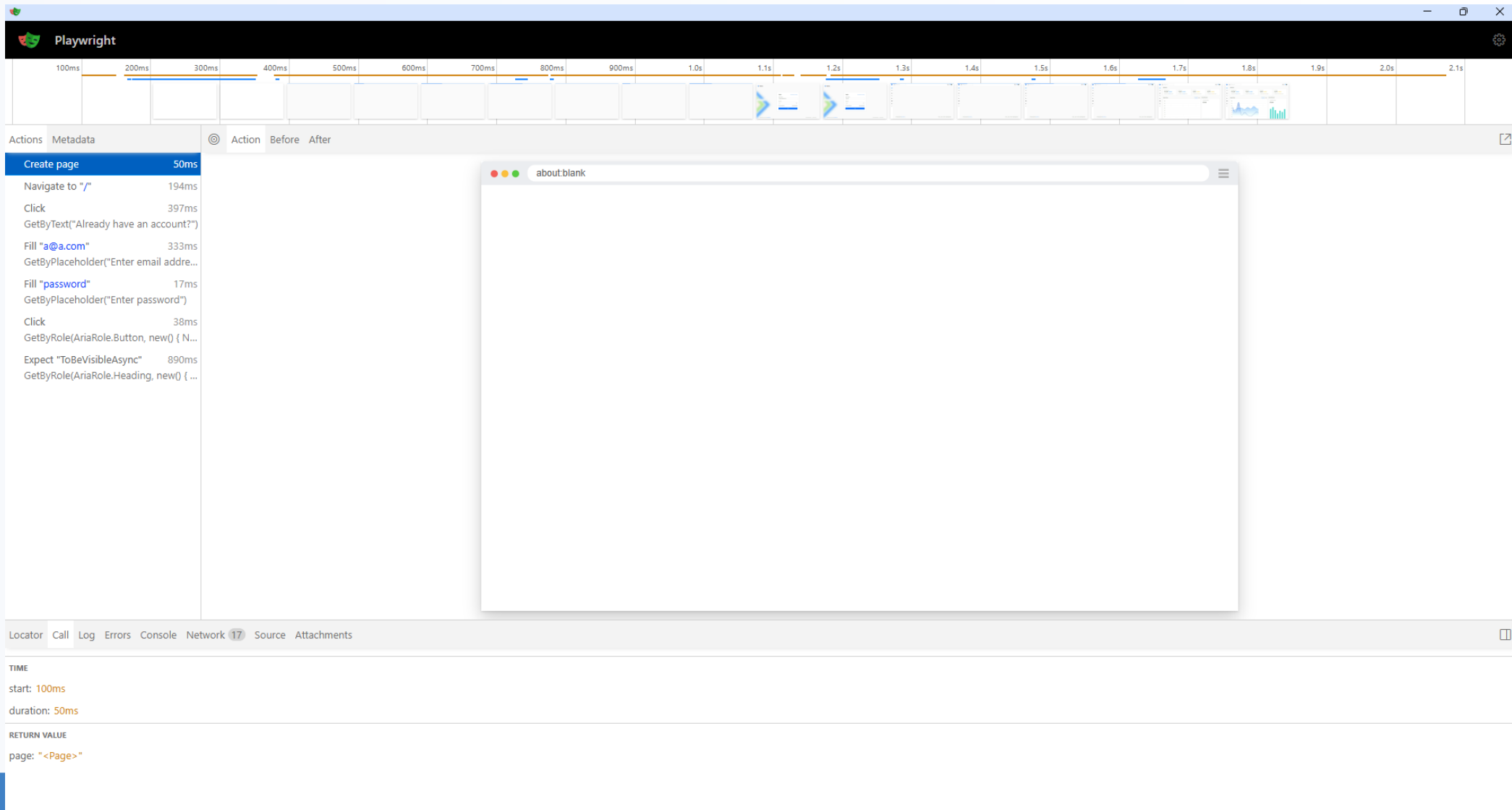


c68b90594547a96
adb9d4ab494aa6
ae8.webm



cd5310d0c9e8a15
d467fd8fa1b124f
1d.webm

Show-Trace pwsh bin/Debug/net10.0/playwright.ps1 show-trace .\traces\trace-20260528-152818.zip



Playwright

100ms 200ms 300ms 400ms 500ms 600ms 700ms 800ms 900ms 1.0s 1.1s 1.2s 1.3s 1.4s 1.5s 1.6s 1.7s 1.8s 1.9s 2.0s 2.1s

Actions Metadata

Click 397ms
GetByText("Already have an account?")

Create page 50ms
Navigate to "/" 194ms
Fill "a@a.com" 333ms
GetByPlaceholder("Enter email address")
Fill "password" 17ms
GetByPlaceholder("Enter password")
Click 38ms
GetByRole(AriaRole.Button, new) { N...
Expect "ToBeVisibleAsync" 890ms
GetByRole(AriaRole.Heading, new) { ...

Locator Call Log Errors Console Network 17 Source Attachments

TIME
start: 378ms
duration: 397ms

PARAMETERS
locator: GetByText("Already have an account?")
timeout: 30000
strict: true

http://localhost:3000/register

Mantis

Sign up

Already have an account?

First Name*
John

Last Name*
Doe

Company
Demo Inc.

Email Address*
demo@company.com

Password

Poor

By Signing up, you agree to our [Terms of Service](#) and [Privacy Policy](#)

Create Account

© Made with love by Team CodedThemes

Terms and Conditions Privacy Policy

Playwright

100ms 200ms 300ms 400ms 500ms 600ms 700ms 800ms 900ms 1.0s 1.1s 1.2s 1.3s 1.4s 1.5s 1.6s 1.7s 1.8s 1.9s 2.0s 2.1s

Actions Metadata

Create page 50ms

Navigate to "/" 194ms

Click 397ms
GetByText("Already have an account?")

Fill "a@a.com" 333ms
GetByPlaceholder("Enter email address...")

Fill "password" 17ms
GetByPlaceholder("Enter password")

Click 38ms
GetByRole(AriaRole.Button, new) { N...

Expect "ToBeVisibleAsync" 890ms
GetByRole(AriaRole.Heading, new) { ...

⌕ Action Before After

http://localhost:3000/register

Mantis

Sign up

Already have an account?

First Name* Last Name*

John Doe

Company

Demo Inc.

Email Address*

demo@company.com

Password

Poor

By Signing up, you agree to our [Terms of Service](#) and [Privacy Policy](#)

Create Account

© Made with love by Team CodedThemes

Terms and Conditions Privacy Policy

Locator Call Log Errors Console Network 17 Source Attachments

TIME

start: 378ms

duration: 397ms

PARAMETERS

locator: GetByText("Already have an account?")

timeout: 30000

strict: true

Playwright

100ms 200ms 300ms 400ms 500ms 600ms 700ms 800ms 900ms 1.0s 1.1s 1.2s 1.3s 1.4s 1.5s 1.6s 1.7s 1.8s 1.9s 2.0s 2.1s

Actions Metadata Action Before After

Create page	50ms
Navigate to "/"	194ms
Click	397ms
GetByText("Already have an account?")	
Fill "a@a.com"	333ms
GetByPlaceholder("Enter email address")	
Fill "password"	17ms
GetByPlaceholder("Enter password")	
Click	38ms
GetByRole(AriaRole.Button, new) { N...	
Expect "ToBeVisibleAsync"	890ms
GetByRole(AriaRole.Heading, new) { ...	

http://localhost:3000/login

Mantis

Login

Don't have an account?

Email Address

a@a.com

Password

☐ Keep me sign in

Forgot Password?

Login

© Made with love by Team CodedThemes

Terms and Conditions Privacy Policy

Locator Call Log Errors Console Network 17 Source Attachments

TIME

start: 1.1s

duration: 17ms

PARAMETERS

locator: GetByPlaceholder("Enter password")

value: "password"

timeout: 30000

Playwright

100ms200ms300ms400ms500ms600ms700ms800ms900ms1.0s1.1s1.2s1.3s1.4s1.5s1.6s1.7s1.8s1.9s2.0s2.1s

ActionsMetadata

Create page50ms

Navigate to "/"194ms

Click397ms

GetByText("Already have an account?")

Fill "a@a.com"333ms

GetByPlaceholder("Enter email addre...")

Fill "password"17ms

GetByPlaceholder("Enter password")

Click38ms

GetByRole(AriaRole.Button, new) { N...

Expect "ToBeVisibleAsync"890ms

GetByRole(AriaRole.Heading, new) { ...

⌕ ActionBeforeAfter

http://localhost:3000/login

Mantis

Login

Don't have an account?

Email Address

a@a.com

Password

☐ Keep me sign in

Forgot Password?

© Made with love by Team CodedThemes

Terms and ConditionsPrivacy Policy

LocatorCallLogErrorsConsoleNetwork17SourceAttachments

TIME

start: 1.1s

duration: 38ms

PARAMETERS

locator: GetByRole(AriaRole.Button, new) { Name = "Login" }

timeout: 30000

strict: true

Playwright

100ms 200ms 300ms 400ms 500ms 600ms 700ms 800ms 900ms 1.0s 1.1s 1.2s 1.3s 1.4s 1.5s 1.6s 1.7s 1.8s 1.9s 2.0s 2.1s

Actions Metadata

⌕ Action Before After

Create page 50ms

Navigate to "/" 194ms

Click 397ms

GetByText("Already have an account?")

Fill "a@a.com" 333ms

GetByPlaceholder("Enter email address")

Fill "password" 17ms

GetByPlaceholder("Enter password")

Click 38ms

GetByRole(AriaRole.Button, new) { Name = "Sign Up" }

Expect "ToBeVisibleAsync" 890ms

GetByRole(AriaRole.Heading, new) { Name = "Dashboard" }

Locator Call Log Errors Console Network 17 Source Attachments

Expect "ToBeVisibleAsync"

TIME

start: 1.2s

duration: 890ms

PARAMETERS

locator: GetByRole(AriaRole.Heading, new) { Name = "Dashboard" }

expression: "to.be.visible"

http://localhost:3000/dashboard/default

Dashboard

Total Page Views
4,42,236 59.3%

Total Users
78,250 70.5%

Total Order
18,800 27.4%

Total Sales
35,078 27.4%

Unique Visitor

Income Overview

This Week Statistics

\$7,650

Mo Tu We Th Fr Sa Su

Module 11

Test Project Structure and Best Practices

Folder structure

- A good folder structure should separate:
 - Tests
 - Pages (POM)
 - Models/Test Data
 - Fixtures
 - Utilities
 - Configuration
 - Evidence (screenshots/videos/traces)

Folder structure

Folder	Purpose
Pages	Page Object Model classes
Tests	Actual xUnit test files
Models	DTO/Test models
TestData	Static factory methods and reusable test data
Fixtures	Shared browser/context lifecycle
Utilities	Reusable helper methods
Evidence	Screenshots/videos/traces
Config	Environment configuration


```
└─ Pages\  
└─ Authentication\  
  └─ MantisLoginPage.cs  
  
└─ Dashboard\  
  └─ MantisDashboardPage.cs  
  
└─ Forms\  
  └─ MantisFormsWizardPage.cs  
  
└─ Tests\  
└─ Authentication\  
  └─ LoginTests.cs  
  
└─ Forms\  
  └─ FormsWizardTests.cs  
  
└─ Debugging\  
  └─ TraceViewerTests.cs  
  
└─ Models\  
└─ ShippingAddressModel.cs  
└─ PaymentModel.cs  
└─ UserModel.cs  
  
└─ TestData\  
  └─ TestDataFactory.cs  
  
└─ Fixtures\  
└─ PlaywrightFixture.cs  
└─ BrowserFixture.cs  
  
└─ Utilities\  
└─ ScreenshotHelper.cs  
└─ TraceHelper.cs  
└─ WaitHelpers.cs  
  
└─ Evidence\  
└─ Screenshots\  
└─ Videos\  
└─ Traces\  
  
└─ Config\  
  └─ appsettings.json
```



Solution Naming

- Use PascalCase and make the purpose obvious.
- **Good Examples**
 - **PlaywrightCourse.slnx**
 - **Activity7.MantisPomAutomation.Tests.slnx**
 - **Activity10.DebuggingEvidence.Tests.slnx**
- **Avoid**
 - **test.slnx**
 - **playwright.slnx**
 - **automation1.slnx**

Project Naming

- Format:
<ProjectName>.Tests
- or
<ActivityName>.Feature.Tests
- Examples
Activity1.BasicPlaywright.Tests
Activity5.MuiDialogAutomation.Tests
Activity8.TestDataUiState.Tests
Activity9.RealisticE2EWorkflowAutomation.Tests

Namespace Naming

- Match the folder structure and project name.
- **Example**
namespace Activity9.RealisticE2EWorkflowAutomation.Tests.Pages.Forms;

Folder Naming

- Use PascalCase.

Pages/

Tests/

Models/

Helpers/

Utilities/

Fixtures/

Data/

Assertions/

Components/

Constants/

Configurations/

Test Class Naming

- Format:
<Feature>Tests
- Examples
LoginTests
DashboardTests
ShoppingCartTests
MantisDialogTests
AuthenticationStateReuseTests

Test Method Naming

- The best convention in xUnit is:
Should_<ExpectedBehavior>
- or
<Action>_<ExpectedResult>

Test Method Naming

- **Recommended**

- Should_Login_Successfully()**

- Should_Display_Validation_Error_For_Invalid_Email()**

- Should_Create_New_User()**

- Should_Filter_Table_By_Status()**

- Should_Reuse_Authentication_State_Without_Login()**

- **Longer Enterprise-Style**

- Should_Display_Error_When_Invalid_Credentials_Are_Entered()**

- Should_Navigate_To_Dashboard_After_Successful_Login()**

- Should_Save_Profile_Changes_When_Form_Is_Valid()**

Page Object Naming (POM)

- Format

<PageName>Page

- Examples

LoginPage

DashboardPage

ProfilePage

CheckoutPage

MantisFormsWizardPage

MantisFilteringTablePage

Component Naming

- Format:
 <ComponentName>Component
- Examples
 SidebarComponent
 HeaderComponent
 NavigationMenuComponent
 UserTableComponent

Model Naming

- Format:
<EntityName>Model
- Examples
LoginModel
ShippingAddressModel
UserRegistrationModel
PaymentDetailsModel

Test Data Factory Naming

- Format:
<EntityName>Factory
- or
TestDataFactory
- Examples
UserFactory
ShippingAddressFactory
OrderFactory
TestDataFactory

Variable Naming

- Use camelCase.
- Good
 - loginButton**
 - shippingAddress**
 - dashboardHeading**
 - submitButton**
- Avoid
 - LoginButton**
 - Btn1**
 - X**
 - obj**

Constant Naming

- Use PascalCase for const.
- **Example**
`public const string BaseUrl = "https://mantisdashboard.com";`

Async Method Naming

- Always suffix with Async.

- **Correct**

LoginAsync()

GotoAsync()

FillShippingAddressAsync()

WaitForDashboardAsync()

- **Wrong**

Login()

Goto()

FillForm()

Screenshot Naming

- Use timestamps or scenario names.
- Examples
 - login-failure.png**
 - dashboard-loaded.png**
 - checkout-error-20260528.png**

Trace/Video Naming

- Use test scenario names.
- Examples
 - Should_Login_Successfully.zip**
 - Should_Filter_Table_By_Status.webm**

Configuration Naming

- **Files**

playwright.config.json

appsettings.json

testsettings.json

Environment Variable Naming

- Use uppercase with underscore.

- Examples

BASE_URL

TEST_USERNAME

TEST_PASSWORD

HEADLESS

Example Full Naming Combination

```
namespace Activity9.RealisticE2EWorkflowAutomation.Tests.Pages.Forms;

public sealed class MantisFormsWizardPage
{
    private readonly IPage _page;

    public MantisFormsWizardPage(IPage page)
    {
        _page = page;
    }

    public async Task FillShippingAddressAsync(
        ShippingAddressModel shippingAddress)
    {
        await _page
            .GetByPlaceholder("First Name")
            .FillAsync(shippingAddress.FirstName);
    }
}
```



Test isolation

- Every test should run independently and should not rely on another test's data, browser state, cookies, login session, or execution order.
- A test must still pass even if:
 - It runs alone
 - It runs first
 - It runs last
 - Other tests fail
 - Tests run in parallel

Reusable fixtures

- **Important rule**
- Share the **browser**, but create a new **context** and **page** per test.
 - await using IBrowserContext context =
 await _fixture.Browser.NewContextAsync();

 IPage page = await context.NewPageAsync();
- This keeps tests isolated while avoiding repeated browser startup cost.

Test cleanup

- **Test cleanup** means removing anything created or changed by a test so the next test starts clean.
 - Closing the IPage
 - Closing the IBrowserContext
 - Deleting test data created during the test
 - Clearing temporary files like screenshots, videos, or downloaded files when needed

Test cleanup

- Playwright recommends using separate browser contexts for isolation, because each context is an independent browser session.
- xUnit supports async setup and cleanup through **IAsyncLifetime**, using **InitializeAsync()** and **DisposeAsync()**.

Test and Cleanup Rule

- Create Browser once if possible.
- Create BrowserContext per test.
- Create Page per test.
- Close Context after each test.

Maintainability guidelines

1. Follow the Page Object Model (POM)

➤ Separate:

- test logic
- UI interaction logic

Bad Example

```
[Fact]
public async Task LoginTest()
{
    await page.GotoAsync("https://example.com");

    await page.GetByPlaceholder("Email").FillAsync("admin@test.com");
    await page.GetByPlaceholder("Password").FillAsync("password");

    await page.GetByRole(
        AriaRole.Button,
        new() { Name = "Login" }
    ).ClickAsync();

    await Assertions.Expect(
        page.GetByText("Dashboard")
    ).ToBeVisibleAsync();
}
```

Better Example: Login Page

```
public sealed class LoginPage
{
    private readonly IPage _page;
    public LoginPage(IPage page)
    {
        _page = page;
    }
    public async Task GotoAsync()
    {
        await _page.GotoAsync("https://example.com");
    }
    public async Task LoginAsync(string email, string password)
    {
        await _page.GetByPlaceholder("Email").FillAsync(email);
        await _page.GetByPlaceholder("Password").FillAsync(password);
        await _page.GetByRole(
            AriaRole.Button,
            new() { Name = "Login" }
        ).ClickAsync();
    }
}
```



Test

```
[Fact]
public async Task Should_Login_Successfully()
{
    LoginPage loginPage = new(page);

    await loginPage.GotoAsync();

    await loginPage.LoginAsync(
        "admin@test.com",
        "password"
    );
}
```

Maintainability guidelines

2. Keep Tests Focused

- Each test should validate ONE business behavior.

Bad

```
[Fact]
public async Task Everything_Test()
{
    // login
    // create user
    // delete user
    // logout
}
```

Maintainability guidelines

2. Keep Tests Focused

- Each test should validate ONE business behavior.

Better

[Fact]

```
public async Task Should_Login()
```

[Fact]

```
public async Task Should_Create_User()
```

[Fact]

```
public async Task Should_Delete_User()
```

Maintainability guidelines

3. Avoid Hardcoded Waits

➤ Never rely on:

`await Task.Delay(5000);`

➤ Problems:

- slow tests
- flaky tests
- unreliable timing

➤ Use Auto-Waiting

**`await Assertions.Expect(
page.GetByRole(AriaRole.Alert)
)toBeVisibleAsync();`**

Maintainability guidelines

4. Prefer Stable Locators

- Priority order:
 - Test IDs
 - Roles
 - Labels
 - Text
 - CSS/XPath (last resort)

Maintainability guidelines

5. Centralize Test Data - Do not hardcode data everywhere.

- Bad

- `await loginPage.LoginAsync(
 "admin@test.com",
 "Password123"
);`

Maintainability guidelines

- Better

```
public static class TestUsers  
{  
    public const string AdminEmail = "admin@test.com";  
  
    public const string AdminPassword = "Password123";  
}
```

- Usage:

```
await loginPage.LoginAsync(  
    TestUsers.AdminEmail,  
    TestUsers.AdminPassword  
);
```

Maintainability guidelines

6. Use Reusable Fixtures - it help reduce duplication.

```
public sealed class BrowserFixture : IAsyncLifetime
{
    public IBrowser Browser { get; private set; }
    private IPlaywright _playwright;
    public async Task InitializeAsync()
    {
        _playwright = await Playwright.CreateAsync();
        Browser = await _playwright.Chromium.LaunchAsync(
            new BrowserTypeLaunchOptions
            {
                Headless = false
            });
    }
    public async Task DisposeAsync()
    {
        await Browser.CloseAsync();
        _playwright.Dispose();
    }
}
```



Maintainability guidelines

7. Use Meaningful Naming Conventions

- **Good Test Names**

- Should_Login_With_Valid_Credentials()
- Should_Display_Validation_Error_When_Email_Is_Invalid()

- **Bad Test Names**

- Test1()
- LoginTest()

Maintainability guidelines

8. Separate Layers Properly

Activity/



Maintainability guidelines

9. Keep Assertions Inside Tests (Mostly)

- Page objects should:
 - interact with UI
- Tests should:
 - validate business expectations

```
// Better

// Page
public async Task ClickSaveAsync()
{
    await _page.GetByRole(
        AriaRole.Button,
        new() { Name = "Save" }
    ).ClickAsync();
}

// Test
await page.ClickSaveAsync();
await Assertions.Expect(
    notification
).ToContainTextAsync("Saved");
```


Maintainability guidelines

10. Avoid Duplicate Selectors

- If a selector changes, you should update ONE location only.

Good

```
private ILocator SaveButton =>  
    _page.GetByRole(  
        AriaRole.Button,  
        new() { Name = "Save" }  
    );
```

- Usage:

```
await SaveButton.ClickAsync();
```

Maintainability guidelines

11. Use Models for Complex Forms

➤ Benefits:

- reusable
- scalable
- cleaner methods

➤ Example

```
public sealed class ShippingAddressModel  
{  
    public string FirstName { get; set; }  
    public string LastName { get; set; }  
    public string Address { get; set; }  
}
```

➤ Usage:

```
await wizardPage.FillShippingAddressAsync(shipping);
```

Maintainability guidelines

12. Design for UI Changes

- Your automation WILL face:
 - renamed buttons
 - changed layouts
 - updated MUI structure
- Good maintainability minimizes update effort.

Maintainability guidelines

13. Use Helper Methods

- Common operations should be reusable.

```
public static class WaitHelpers
{
    public static async Task WaitForSpinnerAsync(IPage page)
    {
        await Assertions.Expect(
            page.GetText("Loading...")
        ).ToBeHiddenAsync();
    }
}
```

Maintainability guidelines

14. Keep Tests Independent

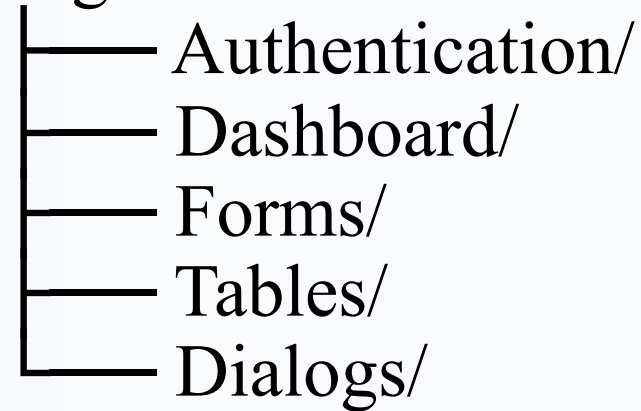
- Each test must:
 - create its own state
 - clean up after itself
 - never depend on execution order
- This is critical for:
 - CI/CD
 - parallel execution
 - reliability

Maintainability guidelines

15. Organize by Feature

- Better than organizing by locator type.

Pages/

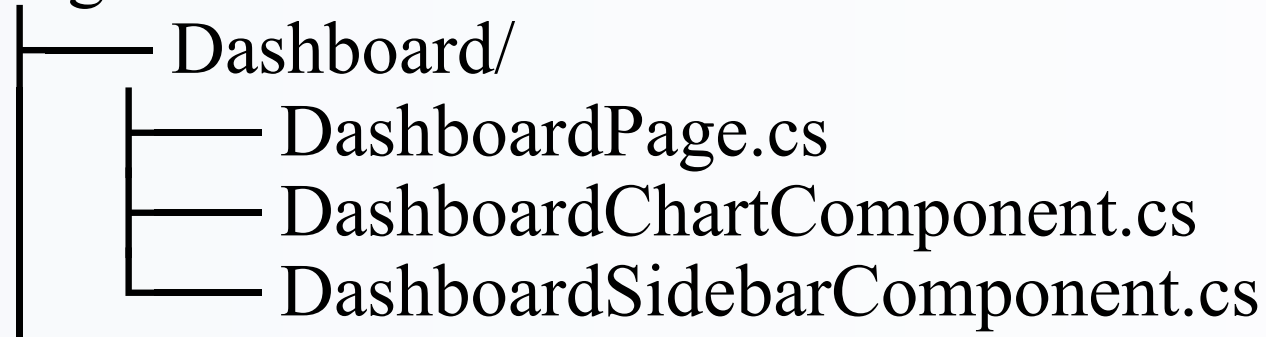


Maintainability guidelines

16. Prefer Composition Over Huge Page Classes

- Avoid giant page classes with 100+ methods.

Pages/



Maintainability guidelines

17. Record Evidence Automatically

- For debugging and long-term maintenance:
 - screenshots
 - videos
 - traces
- Example:

```
await context.Tracing.StartAsync(  
    new TracingStartOptions  
    {  
        Screenshots = true,  
        Snapshots = true  
    });
```


Maintainability guidelines

18. Make Tests Read Like Documentation

- Good automation should explain business behavior clearly.
A non-technical person should almost understand it.
- **Example**
[Fact]
public async Task Should_Prevent_Checkout_When_Cart_Is_Empty()

Module 12

CI/CD Overview for UI Tests

Running Playwright tests with dotnet test

1. Run All Tests

dotnet test

Running Playwright tests with dotnet test

2. Run a Specific Test Class

`dotnet test --filter "FullyQualifiedName~LoginTests"`

Suppose you have:

```
public sealed class LoginTests
{
    [Fact]
    public async Task Should_Login_Successfully()
    {
    }
}
```

Running Playwright tests with dotnet test

3. Run a Specific Test Method

```
dotnet test --filter "Should_Login_Successfully"
```

Or:

```
dotnet test --filter "FullyQualifiedName~Should_Login_Successfully"
```

Running Playwright tests with dotnet test

4. Run Tests in Headed Mode

➤ Normally

```
await playwright.Chromium.LaunchAsync(  
    new BrowserTypeLaunchOptions  
    {  
        Headless = true  
    });
```

➤ For debugging

```
await playwright.Chromium.LaunchAsync(  
    new BrowserTypeLaunchOptions  
    {  
        Headless = false  
    });
```

Running Playwright tests with dotnet test

5. Run Tests with Detailed Logs

```
dotnet test --logger "console;verbosity=detailed"
```

- Useful when troubleshooting failures.

Running Playwright tests with dotnet test

6. Run Tests from a Solution

- For your Playwright course projects:
 - D:\PlaywrightActivities\Module7\Activity7\
- Run:
 - dotnet test Activity7.MantisPomAutomation.Tests.slnx
- or simply:
 - dotnet test
- from the solution folder.

Running Playwright tests with dotnet test

7. Generate a Test Results File

`dotnet test --logger trx`

➤ Produces:

**TestResults\
*.trx**

➤ Useful for:

- Azure DevOps
- Jenkins
- GitLab CI
- Test reporting

Running Playwright tests with dotnet test

8. Run Tests in Parallel can execute tests in parallel.

➤ Example:

`dotnet test -m:4`

➤ Meaning:

Maximum 4 test workers

Running Playwright tests with dotnet test

9. Run Tests with Code Coverage

```
dotnet test --collect:"XPlat Code Coverage"
```

➤ Coverage report:

**TestResults\
coverage.cobertura.xml**

➤ Useful for CI/CD pipelines.

Running Playwright tests with dotnet test

10. Common Playwright Workflow

```
# Restore packages
dotnet restore

# Build project
dotnet build

# Run tests
dotnet test

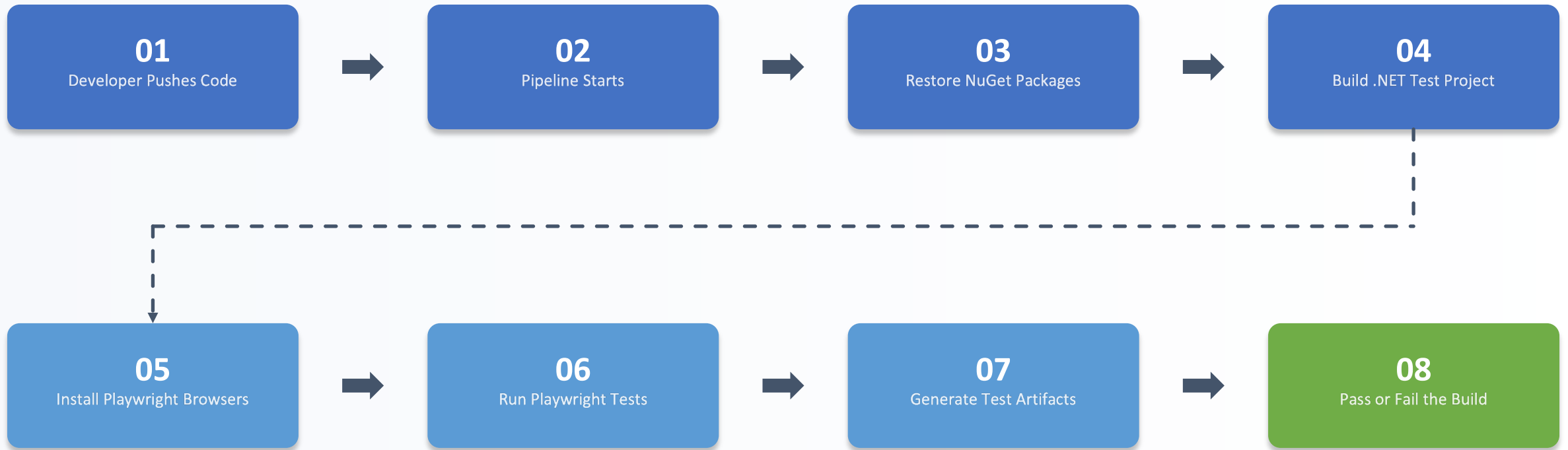
# Detailed logs
dotnet test --logger "console;verbosity=detailed"

# Generate TRX report
dotnet test --logger trx

# Generate coverage
dotnet test --collect:"XPlat Code Coverage"
```



Pipeline execution concept



common pipeline commands

1. `dotnet restore`
2. `dotnet build`
3. `pwsh bin/Debug/net10.0/playwright.ps1 install`
4. `dotnet test`

Example GitHub Actions concept

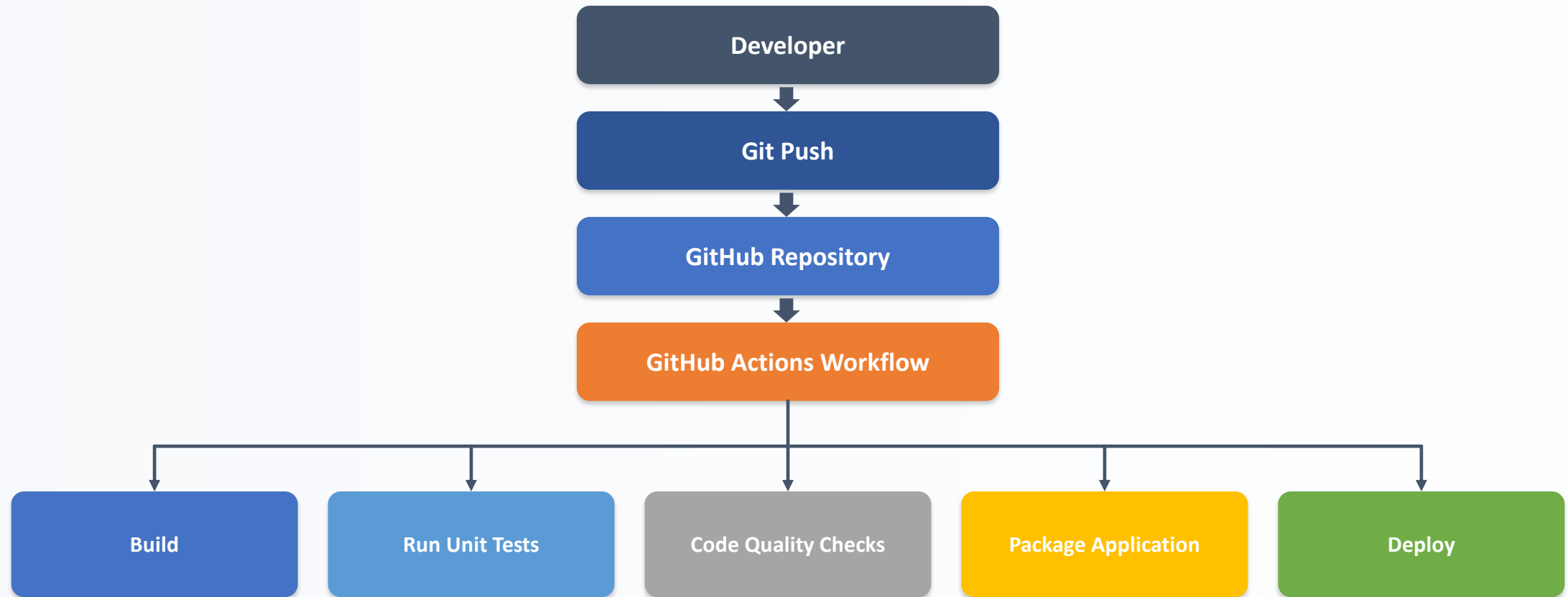
```
name: Playwright .NET Tests
on:
  push:
    branches: [ main ]
jobs:
  ui-tests:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'
      - name: Restore
        run: dotnet restore
      - name: Build
        run: dotnet build --no-restore
      - name: Install Playwright Browsers
        run: pwsh bin/Debug/net10.0/playwright.ps1 install
      - name: Run Tests
        run: dotnet test --no-build
```



GitHub Actions overview

- GitHub Actions is a CI/CD (Continuous Integration and Continuous Delivery/Deployment) platform built directly into GitHub.
- It allows developers to automatically:
 - Build applications
 - Run tests
 - Perform code analysis
 - Create Docker images
 - Deploy applications
 - Automate workflows

Typical CI/CD Workflow

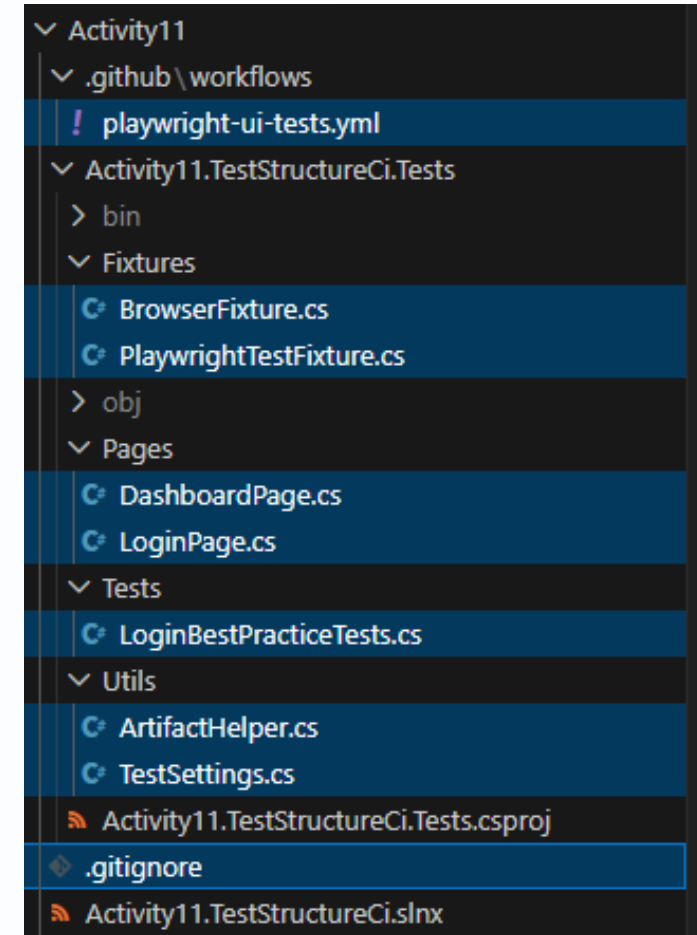


Activity 11 - Test Project Structure and Best Practices

- In this activity you will learn
 - How to organize a Playwright project so it remains maintainable as it grows from 10 tests to 1,000+ tests.
 - Naming Conventions
 - Test Isolation
 - Reusable Fixtures
 - Test Cleanup
 - Maintainability Guidelines
 - GitHub Actions Overview
 - Test Artifacts

Activity 11 - Test Project Structure and Best Practices

- Project Structure
- <https://github.com/crisperando/Playwright>



Activity 11 - Test Project Structure and Best Practices

1. Initialize Git inside Activity11

git init

git status

Activity 11 - Test Project Structure and Best Practices

2. Add .gitignore and ignore the following

```
.vs/  
*.user  
*.suo  
*.userosscache  
*.sln.docstates  
bin/  
obj/  
TestResults/  
*.trx  
artifacts/  
screenshots/  
videos/  
traces/
```

Activity 11 - Test Project Structure and Best Practices

2. Add .gitignore and ignore the following

```
playwright.ps1
playwright.sh
coverage/
coverage-report/
*.coverage
*.coveragexml
.idea/
_ReSharper*/
*.DotSettings.user
.vscode/
Thumbs.db
Desktop.ini
.DS_Store
```

Activity 11 - Test Project Structure and Best Practices

2. Add ./Activity11/.gitignore and ignore the following

```
*.log  
tmp/  
temp/  
packages/  
node_modules/  
.env  
.env.*
```

Activity 11 - Test Project Structure and Best Practices

3. Add `./.github/workflows/playwright-ui-tests.yml`

```
name: Playwright UI Tests

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:
```


Activity 11 - Test Project Structure and Best Practices

3. Add `./.github/workflows/playwright-ui-tests.yml`

```
jobs:
  ui-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Setup .NET 10 SDK
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'

      - name: Restore dependencies
        run: dotnet restore Activity11.TestStructureCi.slnx

      - name: Build solution
        run: dotnet build Activity11.TestStructureCi.slnx --no-restore
```



Activity 11 - Test Project Structure and Best Practices

3. Add `./.github/workflows/playwright-ui-tests.yml`

```
- name: Install Playwright browsers
  working-directory: Activity11.TestStructureCi.Tests
  run: pwsh bin/Debug/net10.0/playwright.ps1 install --with-deps

- name: Run Playwright tests
  run: dotnet test Activity11.TestStructureCi.slnx --no-build --logger
      "trx;LogFileName=test-results.trx"

- name: Upload test artifacts
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: playwright-artifacts
    path: |
      **/TestResults/*.trx
      **/artifacts/screenshots/**
      **/artifacts/videos/**
      **/artifacts/traces/**
```



Activity 11 - Test Project Structure and Best Practices

4. Add all files to repository

git add .

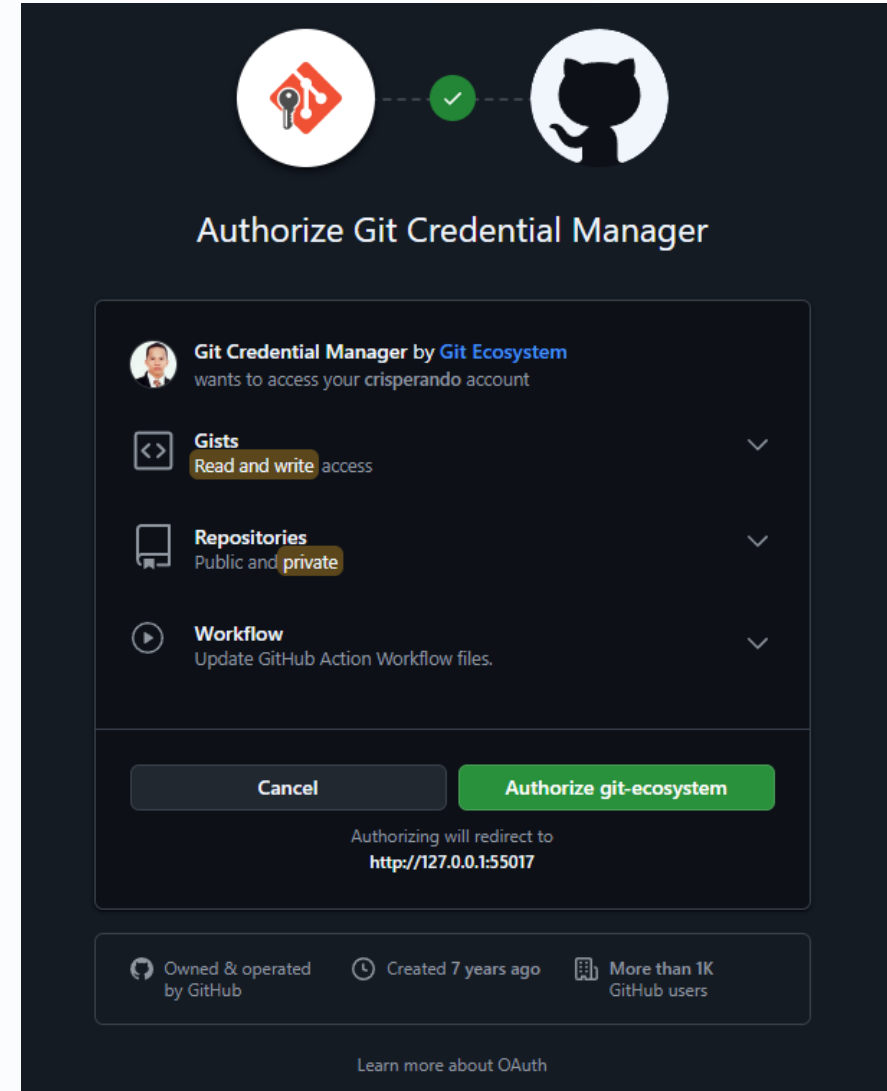
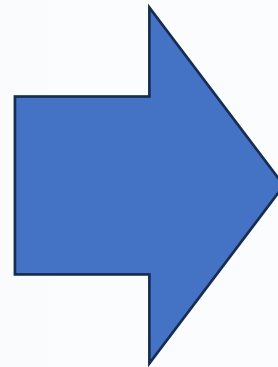
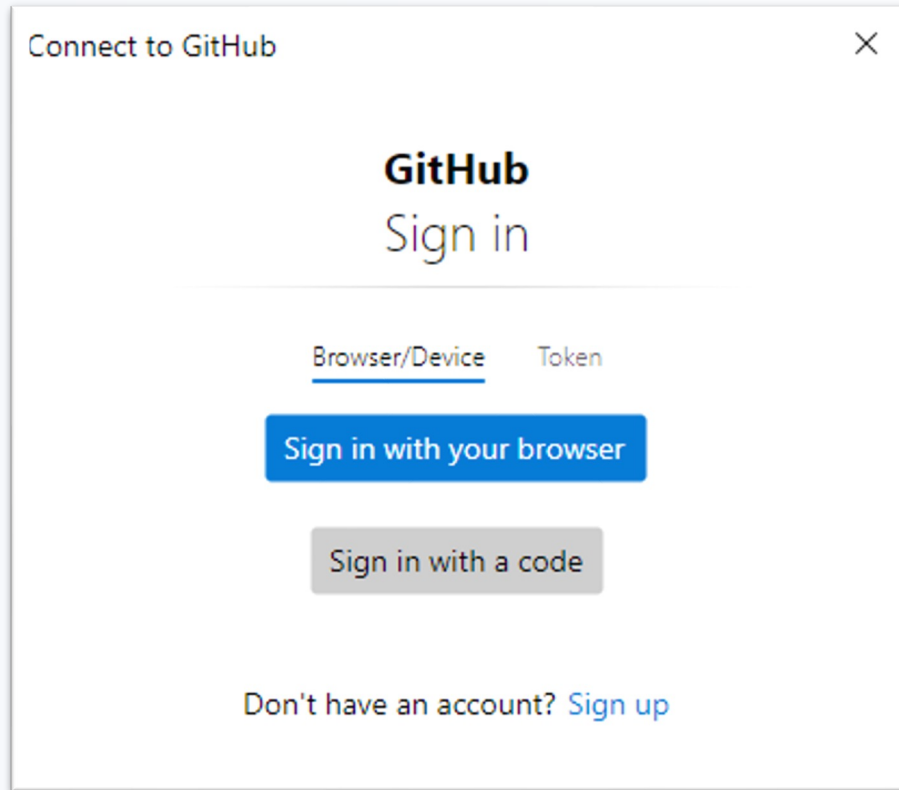
git commit -m "first commit"

git branch -M main

git remote add origin <https://github.com/crisperando/Playwright.git>

git push -u origin main

```
PowerShell
PS D:\PlaywrightActivities\Activity11> git add .
PS D:\PlaywrightActivities\Activity11> git commit -m "first commit"
On branch master
nothing to commit, working tree clean
PS D:\PlaywrightActivities\Activity11> git branch -M main
PS D:\PlaywrightActivities\Activity11> git remote add origin https://github.com/crisperando/Playwright.git
PS D:\PlaywrightActivities\Activity11> git push -u origin main
```



```
PowerShell
PS D:\PlaywrightActivities\Activity11> git add .
PS D:\PlaywrightActivities\Activity11> git commit -m "first commit"
On branch master
nothing to commit, working tree clean
PS D:\PlaywrightActivities\Activity11> git branch -M main
PS D:\PlaywrightActivities\Activity11> git remote add origin https://github.com/crisperando/Playwright.git
PS D:\PlaywrightActivities\Activity11> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 32 threads
Compressing objects: 100% (20/20), done.
Writing objects: 100% (21/21), 4.77 KiB | 348.00 KiB/s, done.
Total 21 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/crisperando/Playwright.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\PlaywrightActivities\Activity11> |
```

github.com/crisperando/Playwright

crisperando / Playwright

Type to search

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Playwright Public

Pin Watch 0

main 1 Branch 0 Tags

Go to file Add file Code

crisperando Initial Activity11 Playwright framework 25e5f48 · 9 hours ago 1 Commit

.github/workflows	Initial Activity11 Playwright framework	9 hours ago
Activity11.TestStructureCi.Tests	Initial Activity11 Playwright framework	9 hours ago
.gitignore	Initial Activity11 Playwright framework	9 hours ago
Activity11.TestStructureCi.slnx	Initial Activity11 Playwright framework	9 hours ago

README

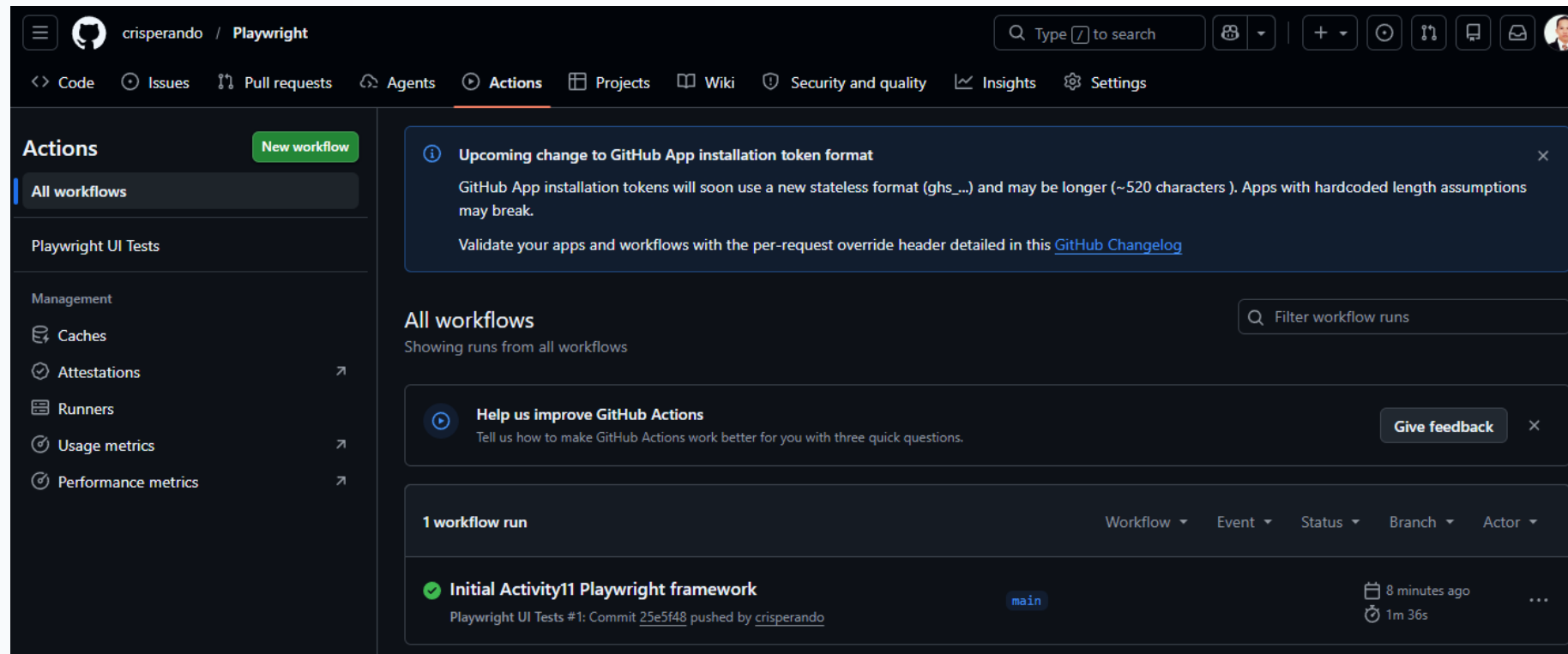
Add a README

Help people interested in this repository understand your project.

Add a README

Activity 11 - Test Project Structure and Best Practices

5. Click Actions <https://github.com/crisperando/Playwright/actions>



6. Under Workflow Run > Click the latest workflow

The screenshot shows the GitHub Actions interface for a workflow named "Initial Activity11 Playwright framework #1". The workflow is in a "Success" state, triggered by a push to the main branch. The total duration is 1m 36s, and there is 1 artifact.

Summary

Triggered via push 9 minutes ago

Triggered via	Status	Total duration	Artifacts
crisperando pushed -> 25e5f48 main	Success	1m 36s	1

playwright-ui-tests.yml

on: push

ui-tests 1m 32s

Annotations

1 warning

ui-tests

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-...

[Show more](#)

Artifacts

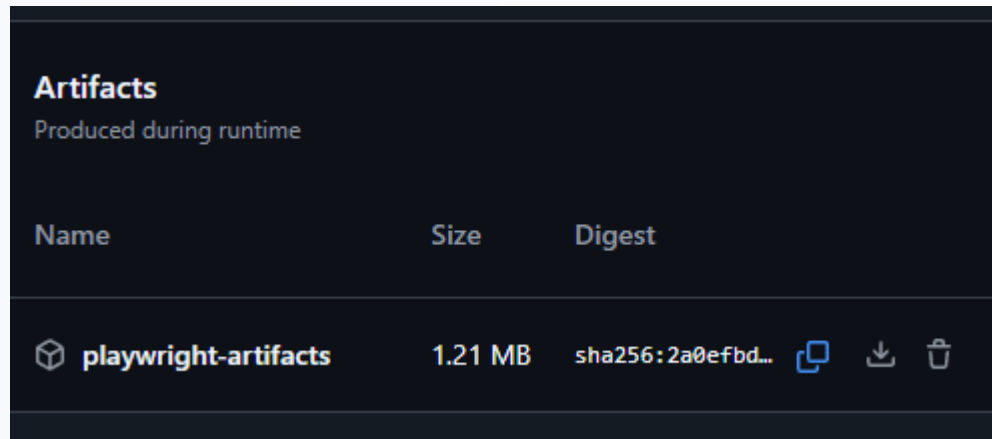
Produced during runtime

Name	Size	Digest
playwright-artifacts	1.21 MB	sha256:2a0efbd20db8f7caa6f06e1b4350746d96bee9...

7. Let say, the test was failed, you can download the artifacts and investigate what makes your test fails.

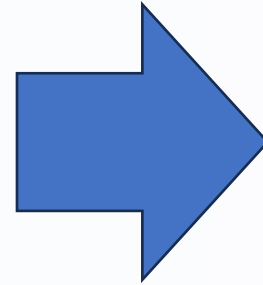
Artifacts			
Produced during runtime			
Name	Size	Digest	
 playwright-artifacts	1.21 MB	sha256:2a0efbd20db8f7caa6f06e1b4350746d96bee9...	  

7. Let say, the test was failed, you can download the artifacts and investigate what makes your test fails.

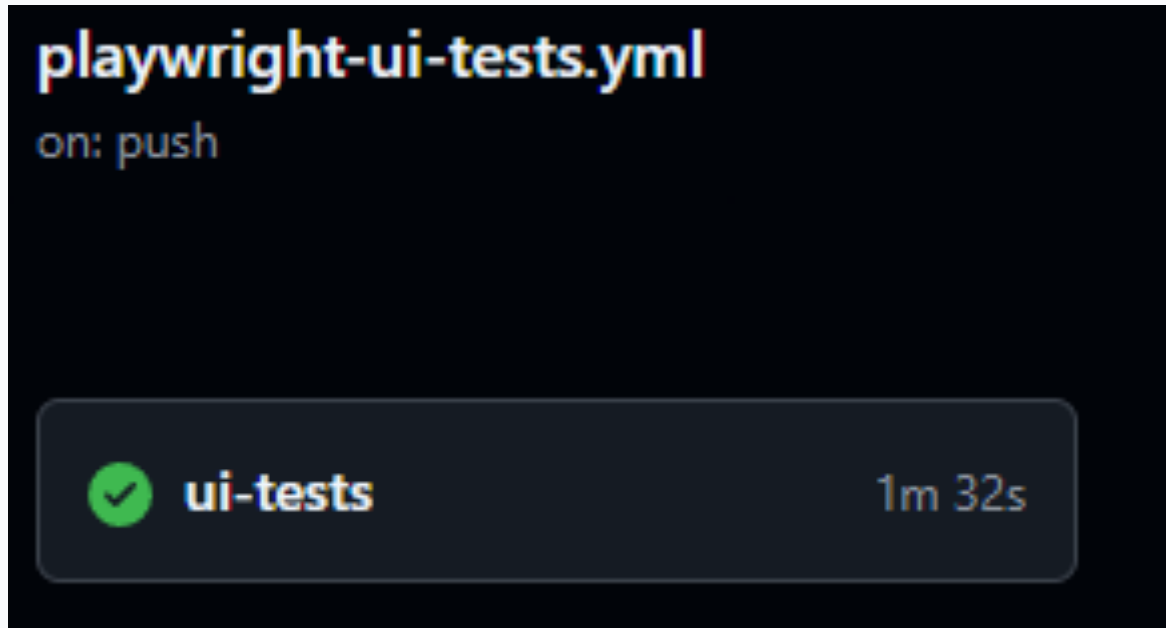


The screenshot shows the 'Artifacts' section of a Playwright test runner. It indicates that artifacts were 'Produced during runtime'. Below this, there is a table with columns for 'Name', 'Size', and 'Digest'. A single artifact is listed: 'playwright-artifacts' with a size of '1.21 MB' and a SHA256 digest starting with 'sha256:2a0efbd...'. To the right of the digest are three icons: a copy icon, a download icon, and a trash icon.

Name	Size	Digest
 playwright-artifacts	1.21 MB	sha256:2a0efbd...   



8. You can also investigate the details of steps and action taken during test
9. Click the latest test take in in the workflows



← Playwright UI Tests

✓ Initial Activity11 Playwright framework #1

Re-run all jobs

...

Summary

All jobs

ui-tests

Run details

Usage

Workflow file

> Annotations

1 warning

ui-tests

succeeded 24 minutes ago in 1m 32s

Search logs

↺

⚙

> ✓ Set up job

2s

> ✓ Checkout source code

1s

> ✓ Setup .NET 10 SDK

0s

> ✓ Restore dependencies

13s

> ✓ Build solution

6s

> ✓ Install Playwright browsers

57s

> ✓ Run Playwright tests

7s

> ✓ Upload test artifacts

1s

> ✓ Post Setup .NET 10 SDK

1s

> ✓ Post Checkout source code

0s

> ✓ Complete job

0s

